

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Analiza imaginilor cu fund de ochi pentru ajutor în
diagnosticul retinopatiei diabetice**

Lucrare de dizertație

Prezentată ca cerință parțială pentru obținerea
titlului de *Master*
în domeniul *Electronică, Telecomunicații și Tehnologia Informației*
programul de studii *Tehnici Avansate de Imagistică Digitală*

Conducător științific
Conf. dr. ing. Laura Maria FLOREA

Absolvent
Ing. Stancovici Marian

Anul 2026

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Programul de masterat **TAID**

Anexa 2

TEMA LUCRĂRII DE DISERTAȚIE
a masterandului **STANCOVICI L.I. Marian, 421-TAID**

1. Titlul temei: Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice

2. Descrierea temei:

Retinopatia diabetică este o boală oculară cronică progresivă asociată unui grup de probleme oculare ca o complicație a diabetului. Această boală poate provoca pierdere severă a vederii sau chiar orbire. Specialiștii analizează imagini de fund de ochi pentru a diagnostica boala și a oferi tratamente specifice. Clasificarea corectă a imaginilor cu fund de ochi depinde de abilitatea și experiența specialiștilor, precum și de calitatea imaginilor. Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

3. Contribuția originală:

1. Se vor analiza metode deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi. 2. Se vor căuta seturi de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice. 3. Se vor analiza seturile de date găsite. 4. Se va propune o metodă bazată pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice. 5. Se va analiza performanța metodei propuse în diverse cazuri de antrenare/testare

4. Resurse și bibliografie:

Alyoubi, W. L., Shalash, W. M., Abulkhair, M. F. (2020). Diabetic retinopathy detection through deep learning techniques: A review. Informatics in Medicine Unlocked, 20, 100377.

5. Data înregistrării temei: 2026-01-13 11:57:24

Conducător(i) lucrare,

Conf. dr. ing. Laura Maria FLOREA

Student,

STANCOVICI L.I. Marian

Responsabil program master,

Prof. dr. ing. Constantin VERTAN

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **f2891db2d4**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul “*Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice*”, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de *Master* în domeniul *Electronică, Telecomunicații și Tehnologia Informației*, programul de studii *Tehnici Avansate de Imagistică Digitală* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, 19.06.2026

Absolvent *Stancovici Marian*



(semnătura în original)

Cuprins

Lista figurilor	iii
Lista tabelelor	v
Lista acronimelor	vii
1. Introducere	1
2. Fundamente teoretice	3
2.1. Diabetul zaharat și complicațiile sale oculare	3
2.2. Anatomia ochiului și retina	4
2.3. Retinopatia diabetică – definiție, stadii de evoluție și simptome	6
2.4. Imagistica de fund de ochi ("fundus photography")	8
2.5. Diagnosticul clinic – procesul actual și limitările sale	9
2.6. Învățarea automată și învățarea profundă în analiza imaginilor medicale	10
2.7. Rețele neuronale convoluționale	12
2.8. Transfer learning și modele pre-antrenate	15
3. Stadiul actual al cercetării	17
3.1. Analiza seturilor de date disponibile	17
3.2. Metoda bazată pe arhitectura Inception-V3 [1]	19
3.3. "Transfer Learning" pentru diagnosticul retinopatiei diabetice [2]	20
3.4. "Multi-task Learning" pentru gradarea retinopatiei diabetice [3]	21
3.5. Analiza arhitecturii "Modified U-Net" pentru segmentarea semantică [4]	22
3.6. Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]	23
3.7. Arhitectura "Dual Branch" [6]	25
3.8. Abordarea "Explainable AI" [7]	25
3.9. Generalizarea în scenarii reale: "Multi-Model Domain Adaptation" [8]	26
3.10. Concluzii	27
4. Seturi de date și construirea dataset-urilor experimentale	29
4.1. Rolul seturilor de date în lucrare	29
4.2. Sursele de date utilizate	30
4.3. Pipeline-ul de prelucrare pentru EyePACS	31
4.4. Pipeline-ul de prelucrare pentru APTOS 2019	32
4.5. Pipeline-ul de prelucrare pentru Diabetic_Balanced_Aug	33
4.6. Verificarea distribuției imaginilor	33
4.7. Separarea originalelor de augmentări în Diabetic_Balanced_Data	34
4.8. Controlul data leakage prin split grupat	34
4.9. Preprocesarea Ben Graham	35
4.10. Formarea dataset-urilor EyePACS pentru sistemul principal	37
4.11. Dataset-urile finale rezultate	38

5. Metodologia propusă	39
5.1. Fluxul general al sistemului	39
5.2. Modelele analizate	41
5.3. Arhitectura ResNet50	42
5.4. Arhitectura EfficientNet-B3	42
5.5. Arhitectura InceptionV3	43
5.6. Arhitectura Inception-ResNet-V2	43
5.7. Funcții de pierdere utilizate	44
5.8. Optimizatorul și hiperparametrii de antrenare	45
5.9. Strategia de evaluare	48
5.10. Implementarea procesului de antrenare	49
6. Rezultate experimentale	51
6.1. Compararea funcțiilor de pierdere pe Diabetic_Balanced_Data	51
6.2. Compararea arhitecturilor pe Diabetic_Balanced_Data	52
6.3. Analiza data leakage în APTOS și Diabetic_Balanced_Data	52
6.4. Configurația experimentală	54
6.5. Istoricul funcției de pierdere	55
6.6. Istoricul acurateții	57
6.7. Istoricul scorului QWK	58
6.8. Precision, recall și F1 pe clase	59
6.9. Matricele de confuzie	60
6.10. Curbele ROC multclasă	61
6.11. Integrarea cu sistemul final în două etape	63
6.12. Analiza calitativă a exemplurilor de inferență	66
7. Aplicația de inferență	71
7.1. Scopul aplicației dezvoltate	71
7.2. Arhitectura aplicației	71
7.3. Componenta backend	71
7.4. Componenta frontend	72
7.5. Fluxul de utilizare	74
7.6. Integrarea modelului antrenat	74
7.7. Limitări practice ale aplicației	75
8. Concluzii și direcții viitoare	77
8.1. Concluzii generale	77
8.2. Rezultatele obținute față de obiectivele inițiale	77
8.3. Limitările lucrării	77
8.4. Direcții viitoare	78
Bibliografie	81
Anexa A. Cod	85

Lista figurilor

2.1.	Structura anatomică a fundului de ochi [9]	5
2.2.	Reprezentare grafică a diferențelor vizuale între imagini cu, respectiv fără retinopatie [10]	7
2.3.	Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională	12
2.4.	Schema generală a unei rețele neuronale convoluționale [11]	13
2.5.	Schema unei arhitecturi ResNet50 [12]	13
2.6.	Schema unei arhitecturi Inception-v3 [13]	14
2.7.	Schema unei arhitecturi EfficientNet-B3 [14]	14
2.8.	Schema unei arhitecturi Inception-ResNet-v2 [15]	15
3.1.	Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]	18
3.2.	Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2]	20
3.3.	Arhitectura U-net modificată [4]	23
3.4.	Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]	24
4.1.	Exemplu de data leakage în <code>Diabetic_Balanced_Data</code> : imaginea originală și augmentările sale apar în subseturi diferite, permițând modelului să recunoască structuri aproape identice în validare sau testare.	35
4.2.	Vizualizarea etapelor de preprocesare Ben Graham: imagine inițială, mască retiniană, estimarea iluminării de fond, filtrare, generarea haloului și imagine finală.	36
5.1.	Fluxul general al lucrării, de la imaginea de fund de ochi și preprocesare până la ramura de experimente și ramura sistemului principal.	40
5.2.	Fluxul sistemului principal în două etape: detector binar pentru separarea imaginilor sănătoase de cele patologice, urmat de clasificarea severității pentru cazurile trimise mai departe.	41
5.3.	Structura blocurilor MBConv în EfficientNet-B3	43
5.4.	Exemplu de hărți Grad-CAM pentru un model ResNet50 antrenat cu Focal Loss și Adam	49
6.1.	Comparația istoricului loss-ului pentru ResNet50 pe <code>Diabetic_Balanced_Data</code> .	52
6.2.	Matrici de confuzie pentru arhitecturile comparate pe <code>Diabetic_Balanced_Data</code> .	52
6.3.	Comparație APTOS între fluxul cu <i>data leakage</i> și fluxul corect fără <i>data leakage</i> : istoricul funcției de pierdere și matricea de confuzie absolută pentru fiecare variantă.	53
6.4.	Comparație pentru <code>Diabetic_Balanced_Data</code> între varianta publică afectată de <i>data leakage</i> și varianta rearanjată prin split grupat: istoricul funcției de pierdere și matricea de confuzie absolută pentru fiecare variantă.	53
6.5.	Istoricul loss-ului pentru ResNet50 cu AdamW pe APTOS, EyePACS și <code>Balanced_Aug</code> .	56
6.6.	Istoricul loss-ului pentru InceptionResNet-v2 pe APTOS, EyePACS și <code>Balanced_Aug</code> .	57

6.7. Istoricul acurateții pentru ResNet50 cu AdamW pe cele nouă variante de experimente.	58
6.8. Istoricul scorului QWK pentru ResNet50 cu AdamW pe cele nouă variante de experimente.	59
6.9. Precision, recall și F1 pe clase pentru cele nouă experimente ResNet50 cu AdamW.	60
6.10. Matricele de confuzie absolute pentru ResNet50 cu AdamW pe cele trei datase-uri și cele trei configurații.	61
6.11. Curbele ROC multclasă pentru cele nouă experimente ResNet50 cu AdamW din experiments.ipynb	62
6.12. Curbele de pierdere și acuratețe pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.	64
6.13. Evoluția AUC pentru detectorul binar și a QWK pentru modelul de severitate 1–4 prin regresie.	65
6.14. Matrici de confuzie absolute pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.	65
6.15. Distribuția indicatorilor precision, recall și F1 pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.	66
6.16. Model experimental: imagine de clasa 2, cu al doilea scor ca mărime la clasa 0.	68
6.17. Pipeline principal: caz sever, cu scoruri apropiate între stadiile 3 și 4.	68
6.18. Pipeline principal: decizie binară corectă pentru clasa 0.	69
6.19. Pipeline principal: exemplu patologic încadrat în stadiul 2.	69
7.1. Interfața aplicației de inferență în modurile dark și light, cu selectorul de model, previzualizarea imaginii și imaginea folosită la inferență	73
7.2. Cardurile de rezultat, recomandări și metadate tehnice afișate după rularea pipeline-ului principal	73
7.3. Meniul dropdown pentru selecția pipeline-ului principal și a modelelor experimentale	74

Lista tabelelor

2.1. Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice	6
3.1. Tabel comparativ între seturile de date disponibile	19
4.1. Dataset-uri construite și rolul lor în lucrare	38

Lista acronimelor

SOTA = State of the Art
DR = Diabetic Retinopathy
CNN = Convolutional Neural Network
AUC = Area Under the Curve
ROC = Receiver Operating Characteristic
CE = Cross-Entropy
BCE = Binary Cross-Entropy
Deep DRiD = Deep Diabetic Retinopathy Image Dataset
APTOS = Asia Pacific Tele-Ophthalmology Society
UoA-DR = University of Auckland Diabetic Retinopathy
IDRiD = Indian Diabetic Retinopathy Image Dataset
DDR = Dataset of Diabetic Retinopathy
DRIVE = Digital Retinal Images for Vessel Extraction
STARE = Structured Analysis of the Retina
QWK = Quadratic Weighted Kappa
Grad-CAM = Gradient-weighted Class Activation Mapping
CAM = Class Activation Mapping
XAI = Explainable Artificial Intelligence
CAD = Computer-Aided Diagnosis
CUDA = Compute Unified Device Architecture
GPU = Graphics Processing Unit
CPU = Central Processing Unit
HTTP = Hypertext Transfer Protocol
JSON = JavaScript Object Notation
RGB = Red Green Blue
CORS = Cross-Origin Resource Sharing

Capitolul 1

Introducere

Contextul și motivația lucrării

Retinopatia diabetică reprezintă o provocare majoră de sănătate, fiind principala cauză a orbirii prevenibile în rândul populației active. Creșterea continuă a numărului de pacienți cu diabet generează presiune asupra sistemelor medicale, depășind adesea capacitatea de diagnosticare a specialiștilor. Din cauză că această afecțiune poate evolua asimptomatic în fazele inițiale, identificarea pacienților la risc este critică înainte ca leziunile retiniene să devină ireversibile, subliniind necesitatea unor soluții de monitorizare scalabile și eficiente.

Complexitatea diagnosticării este dată de necesitatea unei clasificări precise, conform standardelor internaționale care împart boala în cinci clase de severitate [3], [6]. Procesul impune medicului să distingă între stadiile incipiente, marcate doar de microanevrisme discrete [16], și formele avansate, caracterizate de hemoragii și neovascularizație. Această granularitate a diagnosticului este dificilă de menținut constant în programele de "screening" în masă, unde oboseala și subiectivitatea specialiștilor pot duce la erori de clasificare cu impact asupra deciziilor terapeutice.

Pentru a depăși aceste limitări, domeniul imagisticii medicale a adoptat soluții bazate pe inteligență artificială. Trecerea de la metodele clasice de procesare a imaginii [17] la algoritmi de învățare profundă ("Deep Learning") a marcat un punct de inflexiune, validat clinic prin studii extensive [1]. Rețelele neurale convoluționale au eliminat necesitatea definirii manuale a regulilor vizuale [18], având capacitatea de a învăța trăsături complexe direct din datele brute [7] și de a identifica corelații subtile între texturi și patologia ochiului.

Evoluția recentă a acestor tehnologii a permis tranziția de la modele experimentale simple la arhitecturi robuste [5], capabile să opereze în condiții clinice reale. Între timp cercetarea actuală s-a distanțat de clasificarea binară elementară, concentrându-se pe sisteme care pot gestiona diversitatea imaginilor provenite din multiple surse prin tehnici de adaptare la domeniu [8]. Algoritmi moderni demonstrează o flexibilitate crescută la artefacte și zgomot vizual, oferind o consistență a diagnosticului care reușesc să atingă nivelul expertizei umane [7], [2].

Obiectivele lucrării

Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

Aceasta urmărește atingerea următoarelor obiective:

- Analizarea metodelor deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi.

- Căutarea seturilor de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice.
- Analizarea seturilor de date găsite.
- Propunerea unei metode bazate pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice.
- Analizarea performanței pentru metoda propusă în diverse cazuri de antrenare/testare

Structura lucrării

Lucrarea este organizată în opt capitole principale, urmărind trecerea de la contextul medical și stadiul actual al cercetării la metodologia propusă, rezultatele experimentale și aplicația de inferență dezvoltată.

- **Capitolul 1 – Introducere:** Se prezintă contextul medical al retinopatiei diabetice, motivația temei, obiectivele lucrării și modul de organizare a documentului.
- **Capitolul 2 – Fundamente teoretice:** Se introduc noțiunile medicale și tehnice necesare: diabetul zaharat, retina, stadiile retinopatiei diabetice, imaginile de fund de ochi, rețelele neuronale convoluționale și principiile învățării profunde.
- **Capitolul 3 – Stadiul actual al cercetării:** Se analizează metodele existente pentru detecția și clasificarea automată a retinopatiei diabetice, cu accent pe arhitecturi CNN, ”transfer learning” și tehnici de generalizare între seturi de date.
- **Capitolul 4 – Seturi de date și construirea dataset-urilor experimentale:** Se descriu sursele de date utilizate, scripturile de curățare și reorganizare, augmentarea, preprocesarea Ben Graham și verificarea riscului de *data leakage*.
- **Capitolul 5 – Metodologia sistemului propus:** Se prezintă fluxul general al sistemului, arhitecturile analizate, formularea în două etape, funcțiile de pierdere, hiperparametrii, regularizarea și strategia de evaluare.
- **Capitolul 6 – Rezultate experimentale:** Include experimentele preliminare, analiza augmentării și a preprocesării, evaluarea sistemului principal și interpretarea impactului produs de *data leakage*.
- **Capitolul 7 – Aplicația de inferență:** Descrie aplicația web dezvoltată pentru testarea modelului, arhitectura frontend-backend, fluxul de utilizare și integrarea modelului antrenat.
- **Capitolul 8 – Concluzii și direcții viitoare:** Se sintetizează rezultatele obținute, contribuțiile lucrării, limitările identificate și posibilele direcții de dezvoltare ulterioară.

Capitolul 2

Fundamente teoretice

2.1 Diabetul zaharat și complicațiile sale oculare

Diabetul zaharat este o afecțiune metabolică cronică definită prin menținerea unor valori crescute ale glicemiei pe perioade lungi. În mod fiziologic, nivelul glucozei din sânge este reglat în principal prin acțiunea insulinei, hormon produs de pancreas, care facilitează absorbția glucozei de către celule și utilizarea acesteia ca sursă de energie. În cazul diabetului, acest mecanism este perturbat fie prin absența sau producerea insuficientă de insulină, fie prin rezistența țesuturilor la acțiunea acesteia. Din punct de vedere clinic, cele mai cunoscute forme sunt diabetul zaharat de tip 1, asociat cu distrugerea celulelor beta pancreatice și deficit insulinar sever, și diabetul zaharat de tip 2, asociat mai ales cu rezistența la insulină și cu factori metabolici precum obezitatea, sedentarismul și predispoziția genetică [19].

Importanța diabetului pentru această lucrare nu se limitează la caracterul său metabolic, ci derivă în special din complicațiile pe care le produce. Hiperglicemia cronică afectează vasele de sânge de calibru mic și mare, nervii periferici, rinichii, cordul și structurile oculare. Complicațiile microvasculare sunt de interes major în oftalmologie, deoarece retina este un țesut puternic vascularizat și sensibil la variațiile aportului de oxigen și nutrienți. Vasele retiniene au un diametru redus, iar peretele vascular poate fi deteriorat gradual de modificările biochimice asociate diabetului. În timp, apar alterări ale permeabilității vasculare, microanevrisme, hemoragii, exudate, zone ischemice și, în stadiile avansate, vase de neoformație [19], [20].

Retina este una dintre puținele regiuni ale organismului în care microcirculația poate fi observată direct prin imagistică neinvazivă. Această particularitate transformă examinarea fundului de ochi într-un instrument esențial pentru monitorizarea efectelor diabetului asupra sistemului vascular. Din perspectivă medicală, modificările retiniene reflectă atât afectarea oculară locală, cât și severitatea afectării microvasculare. Din perspectivă computațională, imaginile retiniene oferă un suport vizual bogat, în care semnele patologice pot fi analizate prin metode automate de procesare a imaginilor și prin modele de învățare profundă [1], [21].

Complicațiile oculare ale diabetului includ retinopatia diabetică, edemul macular diabetic, cataracta și glaucomul. Dintre acestea, retinopatia diabetică este cea mai importantă în contextul "screening-ului" automat, deoarece evoluează de obicei lent, poate fi asimptomatică în fazele inițiale și poate produce pierdere ireversibilă de vedere dacă nu este identificată și tratată la timp [20]. Edemul macular diabetic este o complicație strâns legată de retinopatie și apare atunci când permeabilitatea vasculară crescută determină acumularea de lichid în zona maculei, regiunea responsabilă pentru vederea centrală fină [20]. Chiar și atunci când retinopatia nu este în stadiu proliferativ, afectarea maculară poate reduce semnificativ acuitatea vizuală.

În contextul acestei lucrări, cele mai relevante complicații oculare pot fi sintetizate astfel:

- **Retinopatia diabetică** – afectează vasele retiniene și reprezintă principala țintă a sistemului automat de clasificare.

- **Edemul macular diabetic** – implică acumularea de lichid în zona maculei și poate reduce vederea centrală.
- **Cataracta și glaucomul** – nu sunt obiectivul direct al clasificării, dar pot influența calitatea imaginii și interpretarea clinică.

Din punctul de vedere al sistemelor automate, retinopatia diabetică este o problemă dificilă deoarece semnele bolii au dimensiuni, forme și contraste diferite, iar severitatea trebuie estimată pe o scară ordinală, nu doar printr-o decizie binară sănătos/bolnav. Această particularitate justifică abordarea multiclasă detaliată în secțiunea următoare, unde sunt prezentate stadiile bolii și semnele vizuale asociate fiecărui nivel de severitate [1], [3], [22].

2.2 Anatomia ochiului și retina

Ochiul uman este un sistem optic complex, specializat în captarea luminii și transformarea acesteia în semnale nervoase interpretate de creier. Din punct de vedere funcțional, componentele principale implicate în formarea imaginii sunt corneea, cristalinul, umoarea apoasă, corpul vitros și retina. Corneea și cristalinul focalizează lumina, iar retina funcționează ca un strat fotosensibil situat în partea posterioară a globului ocular. În contextul analizei automate a imaginilor, retina este structura de interes, deoarece aici pot fi observate vasele sanguine, discul optic, macula și leziunile asociate retinopatiei diabetice [20], [16].

Retina este formată din mai multe straturi celulare, incluzând fotoreceptori, celule bipolare, celule ganglionare și celule de suport. Fotoreceptorii transformă stimulii luminoși în semnale electrice, iar informația este transmisă prin nervul optic către cortexul vizual. Regiunea centrală a retinei, numită maculă, este responsabilă pentru vederea detaliată, necesară cititului, recunoașterii fețelor și percepției fine a culorilor. În centrul maculei se află foveea, zona cu cea mai mare densitate de conuri și cu acuitatea vizuală maximă. Din acest motiv, afectarea maculei prin edem sau exudate poate avea consecințe majore asupra calității vieții pacientului [23], [20].

În imaginile de fund de ochi, câteva repere anatomice sunt esențiale pentru interpretare. Discul optic apare de obicei ca o regiune circulară sau ovalară, mai luminoasă, din care pornesc vasele retiniene principale. Vasele se ramifică progresiv și formează o rețea vasculară care acoperă suprafața retiniană vizibilă. Macula se observă ca o zonă mai întunecată, situată lateral față de discul optic, iar foveea reprezintă centrul acesteia. Aceste detalii se pot observa în figura de mai jos. Pentru algoritmi de analiză, aceste structuri au rol dublu: pe de o parte oferă informație anatomică utilă, iar pe de altă parte pot produce confuzii. De exemplu, discul optic este o regiune luminoasă care poate fi confundată cu exudatele dacă modelul nu învață contextul anatomic corect.

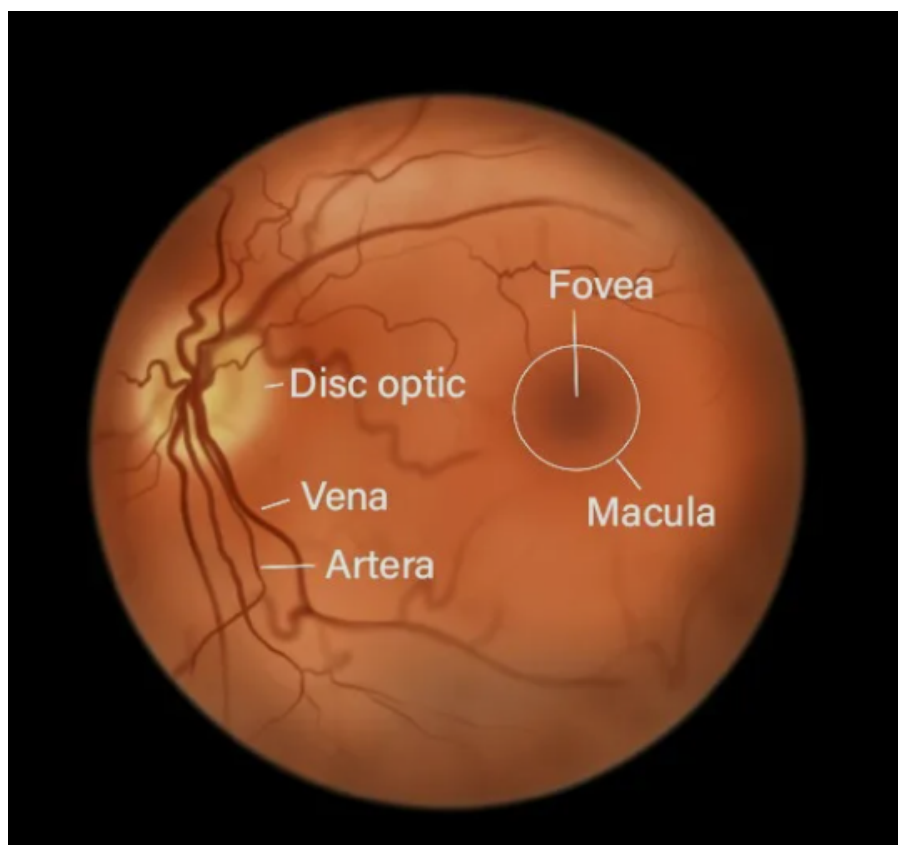


Figura 2.1: Structura anatomică a fundului de ochi [9]

Principalele repere anatomice urmărite într-o imagine de fund de ochi sunt:

- **Discul optic** – regiune luminoasă din care pornesc vasele retiniene principale.
- **Macula** – zonă responsabilă pentru vederea centrală, importantă în evaluarea edemului macular.
- **Foveea** – centrul maculei, asociat cu acuitatea vizuală maximă.
- **Rețeaua vasculară** – structură relevantă pentru identificarea hemoragiilor, microanevrismelor și modificărilor ischemice.

Vasele retiniene au o importanță deosebită în detectarea modificărilor patologice. În diabet, afectarea microvasculară determină fragilizarea pereților capilari și modificări ale fluxului sanguin. Microanevrismele apar ca mici dilatații ale capilarelor, fiind printre cele mai timpurii semne vizibile ale retinopatiei diabetice. Hemoragiile sunt rezultatul ruperii sau scurgerii vaselor deteriorate, iar exudatele apar prin depunerea de lipide și proteine în urma creșterii permeabilității vasculare [23]. În stadiile avansate, ischemia retiniană poate stimula formarea de vase noi, fragile și anormale, fenomen numit neovascularizație [20].

Din perspectiva prelucrării imaginilor, retina prezintă o combinație de structuri globale și detalii locale fine. Structurile globale includ forma câmpului vizual, poziția discului optic și distribuția vaselor principale, iar detaliile locale includ microanevrismele, punctele hemoragice, exudatele mici sau modificările subtile de textură [24], [17].

Un alt aspect relevant este diversitatea aspectului retinei: pigmentarea fundului de ochi, diametrul vaselor și poziția discului optic pot varia între pacienți și între achiziții. Modelele

automate trebuie astfel să distingă între variații anatomice normale și semne patologice reale, iar influența artefactelor de imagine este discutată separat în secțiunea dedicată fotografiei fundus.

2.3 Retinopatia diabetică – definiție, stadii de evoluție și simptome

Retinopatia diabetică este o complicație microvasculară a diabetului zaharat, caracterizată prin afectarea progresivă a vaselor de sânge retiniene. Mecanismele patologice includ lezarea endoteliului vascular, pierderea pericitelor, creșterea permeabilității capilare, ocluzii vasculare și ischemie retiniană. În timp, aceste procese duc la apariția unor leziuni vizibile în imaginile fundului de ochi. În fazele incipiente, pacientul poate să nu observe nicio modificare a vederii, ceea ce face ca ”screening-ul” periodic să fie esențial. În fazele avansate, boala poate produce vedere încetoșată, pete în câmpul vizual, scăderea acuității vizuale și, în cazuri severe, pierderea vederii [20].

Clasificarea clinică a retinopatiei diabetice este de obicei organizată pe niveluri de severitate. În lucrările de învățare automată, inclusiv în seturile de date publice folosite frecvent, se utilizează adesea o scară cu cinci clase: [1], [3], [22]

- Clasa 0 – fără retinopatie diabetică
- Clasa 1 – retinopatie diabetică ușoară
- Clasa 2 – retinopatie diabetică moderată
- Clasa 3 – retinopatie diabetică severă
- Clasa 4 – retinopatie diabetică proliferativă

Această formulare este utilă pentru antrenarea modelelor de clasificare, deoarece transformă evaluarea clinică într-o problemă de predicție multiclasă.

Tabela 2.1: Interpretarea orientativă a claselor de severitate folosite în clasificarea retinopatiei diabetice

Clasa	Grad de severitate	Semne vizuale urmărite în imagine
0	Fără retinopatie diabetică	Nu sunt observate leziuni specifice; imaginea poate fi considerată normală din punct de vedere al retinopatiei diabetice.
1	Retinopatie ușoară	Microanevrisme izolate, de obicei greu vizibile și ușor de confundat cu zgomot sau artefacte de achiziție.
2	Retinopatie moderată	Număr mai mare de leziuni, cu posibile hemoragii, exudate dure și modificări vasculare vizibile.
3	Retinopatie severă	Afectare vasculară extinsă, numeroase hemoragii și risc crescut de progresie către forma proliferativă.
4	Retinopatie proliferativă	Neovascularizație și risc major de complicații severe, precum hemoragie vitroasă sau dezlipire de retină.

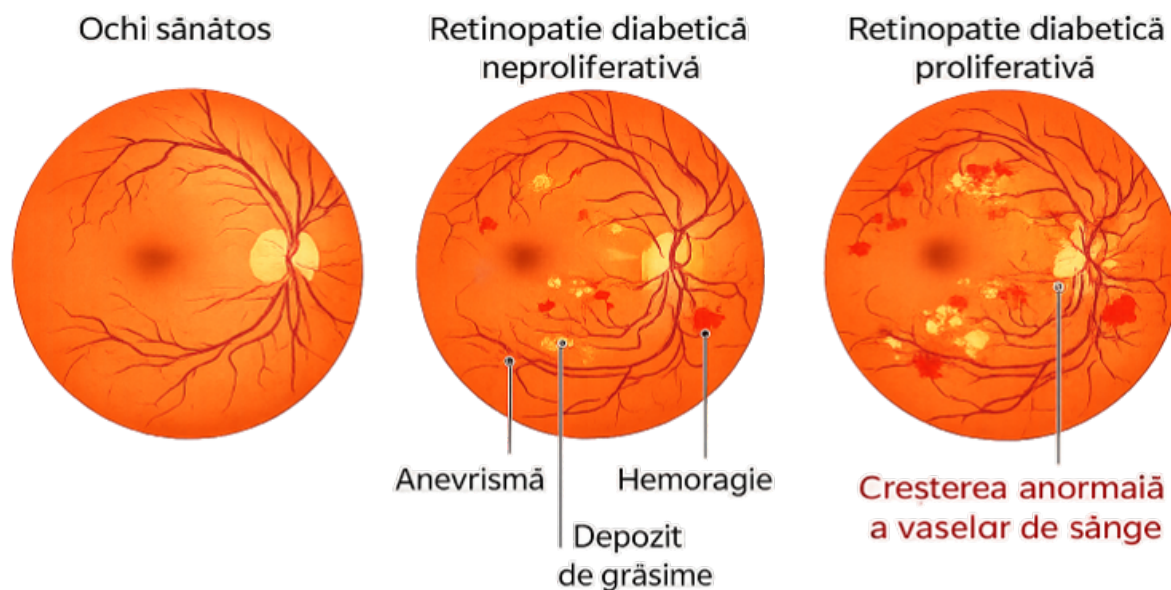


Figura 2.2: Reprezentare grafică a diferențelor vizuale între imagini cu, respectiv fără retinopatie [10]

Clasele 0 și 1 sunt de obicei cele mai subtile din punct de vedere vizual. Absența semnelor evidente într-o fotografie nu garantează absența completă a riscului, deoarece detectabilitatea leziunilor depinde de calitatea imaginii și de câmpul vizual acoperit. În același timp, retinopatia ușoară poate fi reprezentată doar de câteva microanevrisme, motiv pentru care confuziile între clasele 0, 1 și 2 sunt frecvente în modelele automate [1], [22].

Clasele 3 și 4 sunt asociate cu afectare vasculară mai evidentă și cu risc clinic mai ridicat. Imaginea devine, în general, mai evidentă din punct de vedere patologic, iar modelele automate tind să obțină performanțe mai bune pentru aceste stadii decât pentru cele incipiente. Totuși, identificarea corectă a claselor avansate este critică, deoarece subestimarea severității poate întârzia intervenția medicală. Într-un sistem de screening, o eroare de tip fals negativ pentru o formă proliferativă este mult mai gravă decât o confuzie între două clase adiacente [21].

În afara clasificării pe stadii, este importantă și natura leziunilor observabile. Cele mai importante semne patologice sunt:

- **Microanevrismele** – printre primele semne ale bolii, vizibile ca puncte roșii foarte mici.
- **Hemoragiile** – pot avea forme variate, de la punctiforme la pete mai extinse.
- **Exudatele dure** – regiuni galben-albicioase, bine delimitate, asociate cu scurgeri lipidice.
- **Exudatele moi** – pete vătămate care indică zone de ischemie în stratul fibrelor nervoase.
- **Neovascularizația** – semn al trecerii către forma proliferativă.

Metodele automate pot aborda aceste semne fie prin clasificare globală a imaginii, fie prin segmentarea leziunilor, fie prin combinații multi-task, în care modelul învață simultan mai multe sarcini legate de severitate și localizare [24], [17], [23], [20], [3], [5].

2.4 Imagistica de fund de ochi ("fundus photography")

Fotografia de fund de ochi reprezintă una dintre cele mai utilizate metode de examinare imagistică a retinei. Aceasta permite captarea unei imagini color a suprafeței retiniene, incluzând discul optic, vasele sanguine, macula și eventualele leziuni patologice. Spre deosebire de metode mai complexe, precum angiografia cu fluoresceină sau tomografia în coerență optică, fotografia fundus este relativ rapidă, neinvazivă și potrivită pentru programe de "screening" la scară largă [1]. Din acest motiv, majoritatea seturilor de date utilizate în cercetarea privind detecția automată a retinopatiei diabetice se bazează pe astfel de imagini [7], [22].

În mod obișnuit, o imagine fundus este obținută cu o cameră specializată care iluminează retina și captează lumina reflectată prin pupilă. Achiziția poate fi realizată cu sau fără dilatarea pupilei, iar câmpul vizual poate varia în funcție de echipament. Imaginile pot avea rezoluții diferite, niveluri diferite de contrast și grade variabile de claritate. În plus, pot apărea artefacte precum reflexii luminoase, umbre, zone întunecate, margini circulare, praf pe lentilă sau defocalizare. Toate aceste elemente influențează atât evaluarea umană, cât și performanța algoritmilor de clasificare [22], [8].

Pentru analiza automată, imaginea fundus prezintă mai multe provocări. Prima este variația de iluminare. Unele imagini sunt supraexpuse, iar leziunile luminoase pot fi greu de distins de fundal. Altele sunt subexpuse, iar leziunile roșii sau vasele fine pot deveni slab vizibile. A doua provocare este variația cromatică: tonul general al retinei diferă între pacienți și între camere. A treia provocare este rezoluția foarte mare a imaginilor originale. Leziunile mici pot ocupa doar câțiva pixeli, iar redimensionarea agresivă poate elimina informație relevantă. În același timp, procesarea directă a imaginilor la rezoluție completă poate fi costisitoare computațional [1], [6].

Cele mai frecvente dificultăți întâlnite în imaginile de fund de ochi sunt:

- variații de iluminare și contrast între imagini;
- diferențe cromatice între pacienți și între camere de achiziție;
- rezoluții diferite și dimensiuni mari ale imaginilor originale;
- artefacte precum reflexii, blur, margini întunecate sau câmp vizual incomplet;
- dimensiunea foarte mică a unor leziuni, în special a microanevrismelor.

Preprocesarea imaginilor are rolul de a reduce aceste variații și de a oferi modelului date mai consistente. Etapele frecvente includ decuparea marginilor negre, redimensionarea la o dimensiune standard, normalizarea valorilor pixelilor, corectarea iluminării și aplicarea unor transformări de augmentare. În unele abordări, se utilizează metode pentru evidențierea vaselor sau pentru îmbunătățirea contrastului local. În altele, modelul primește imaginea aproape brută, iar rețeaua neuronală învață singură trăsăturile relevante. Alegerea depinde de dimensiunea setului de date, de arhitectura folosită și de obiectivul experimentului [17], [16], [7].

Într-un flux tipic de lucru, preprocesarea poate include:

- decuparea zonelor negre din jurul câmpului circular al imaginii;

- redimensionarea imaginilor la dimensiunea cerută de model;
- normalizarea intensităților pixelilor;
- corectarea parțială a iluminării neuniforme;
- augmentări precum rotații, flip-uri, modificări de luminozitate sau contrast.

Din punct de vedere practic, preprocesarea trebuie să păstreze leziunile fine, nu doar să uniformizeze aspectul general al imaginii. O redimensionare prea agresivă poate elimina microanevrismele, iar o corecție de contrast nepotrivită poate accentua artefactele. De aceea, etapele de preprocesare sunt alese în funcție de modelul folosit și de rezoluția minimă la care semnele patologice rămân vizibile.

Aceste observații sunt importante pentru proiectarea fluxului experimental: imaginile trebuie aduse la o formă comparabilă fără a elimina informația patologică fină. Diferența dintre abordările bazate pe trăsături proiectate manual și cele bazate pe învățare profundă este discutată în secțiunea următoare [24], [17], [18].

Un element important în folosirea imaginilor fundus pentru clasificare este raportul dintre informația globală și cea locală. O imagine poate conține semne patologice mici, dar decizia finală depinde și de distribuția acestora în câmpul retinian. Un model care analizează doar fragmente locale poate detecta leziuni, dar poate pierde contextul global. Un model care analizează imaginea întreagă poate învăța distribuția generală, dar poate rata detalii fine după redimensionare. Din acest motiv, literatura recentă explorează arhitecturi care combină informații la mai multe scări, ramuri multiple sau mecanisme de atenție [6], [5].

2.5 Diagnosticul clinic – procesul actual și limitările sale

Diagnosticul clinic al retinopatiei diabetice se bazează pe evaluarea imaginilor retiniene de către medici oftalmologi sau specialiști instruiți în screening. În cadrul unui flux standard, pacientului i se realizează una sau mai multe fotografii ale fundului de ochi, iar imaginile sunt analizate pentru identificarea leziunilor specifice. În funcție de severitate, pacientul poate fi încadrat într-o categorie de risc și poate primi recomandări pentru monitorizare, investigații suplimentare sau tratament. În programele de screening, scopul principal este identificarea pacienților care necesită evaluare medicală mai rapidă, înainte de apariția pierderii ireversibile de vedere [1], [21].

Procesul clinic are însă mai multe limitări [1], [22]:

- **Disponibilitatea specialiștilor:** Pe măsură ce numărul pacienților cu diabet crește, volumul imaginilor care trebuie evaluate devine tot mai mare.
- **Variabilitatea opiniei între observatori:** Doi specialiști pot evalua diferit aceeași imagine, mai ales în cazurile de graniță dintre clase.
- **Variabilitatea intra-observator:** Același specialist poate avea discrepanțe în decizie în funcție de oboseală, experiență sau calitatea imaginii.
- **Calitatea imaginilor:** Imaginile slab focalizate, întunecate sau parțial acoperite de artefacte sunt greu de interpretat.

În practică, problema calității imaginii rămâne deosebit de importantă. În unele cazuri, imaginea trebuie repetată, ceea ce crește timpul și costul examinării. În alte cazuri, imaginea poate fi evaluată cu incertitudine, ceea ce afectează consistența diagnosticului. Din acest motiv, unele sisteme moderne includ nu doar clasificarea retinopatiei, ci și estimarea calității imaginii, astfel încât imaginile neinterpretabile să fie semnalate separat [22].

Sistemele automate bazate pe inteligență artificială nu înlocuiesc medicul, dar pot funcționa ca instrumente de suport. Într-un scenariu de "screening", un algoritm poate analiza rapid un număr mare de imagini și poate prioritiza cazurile suspecte. De asemenea, poate oferi o a doua opinie sau poate reduce timpul necesar pentru trierea imaginilor fără semne evidente. Studiile moderne au demonstrat că modelele de învățare profundă pot atinge performanțe ridicate în detectarea retinopatiei diabetice pe imagini fundus [1], [21]. Totuși, integrarea clinică necesită validare riguroasă, interpretabilitate, robustețe la imagini din surse diferite și control al erorilor critice.

O dificultate specifică diagnosticului automat este faptul că nu toate erorile au aceeași importanță. În clasificarea multiclasă, o predicție greșită cu o clasă adiacentă poate fi tolerabilă în anumite contexte, dar o predicție care subestimează severitatea bolii poate întârzia tratamentul. De exemplu, clasificarea unei imagini severe ca fiind fără retinopatie este o eroare mult mai periculoasă decât clasificarea unei imagini moderate ca ușoare. Din acest motiv, evaluarea unui model nu trebuie să se bazeze exclusiv pe acuratețe globală, ci pe un set mai larg de metrici [21], [22]:

- **Sensibilitate** (*Sensitivity / Recall*) – exprimă proporția cazurilor patologice detectate corect de model. În context medical, această metrică este importantă deoarece un scor scăzut indică existența unor cazuri de boală ratate (*false negatives*), ceea ce poate afecta diagnosticul precoce.
- **Specificitate** (*Specificity*) – exprimă proporția cazurilor sănătoase identificate corect. O specificitate ridicată indică faptul că modelul produce puține alarme false și nu clasifică eronat pacienții sănătoși ca fiind bolnavi.
- **Scorul AUC** (*Area Under the ROC Curve*) – evaluează capacitatea modelului de a separa clasele pentru diferite praguri de decizie. Un scor AUC apropiat de 1 indică o separare foarte bună, iar un scor apropiat de 0.5 sugerează o performanță slabă, comparabilă cu alegerea aleatoare.
- **Matricea de confuzie** – oferă o reprezentare detaliată a relației dintre clasele reale și cele prezise. Aceasta permite identificarea tipurilor de erori produse de model și evidențierea claselor care sunt confundate frecvent.
- **Performanța pe fiecare clasă** – este necesară mai ales în cazul seturilor de date dezechilibrate. Prin raportarea valorilor de *precision*, *recall* și *F1-score* pentru fiecare clasă, se poate evalua dacă modelul funcționează bine doar pentru clasele majoritare sau și pentru cele rare, dar importante clinic.

2.6 Învățarea automată și învățarea profundă în analiza imaginilor medicale

Învățarea automată este un domeniu al inteligenței artificiale care urmărește dezvoltarea unor algoritmi capabili să învețe modele din date. În loc ca regulile să fie definite explicit de

programator, sistemul ajustează parametri interni pe baza unor exemple. În cazul clasificării imaginilor medicale, exemplele sunt imagini asociate cu etichete clinice, iar obiectivul modelului este să învețe o funcție care primește o imagine și produce o predicție. Pentru retinopatia diabetică, această predicție poate fi binară, de tipul retinopatie prezentă/absentă, sau multclasă, corespunzătoare severității bolii [1], [21].

Metodele clasice de învățare automată necesitau de obicei o etapă separată de extragere a trăsăturilor. În imaginile retiniene, aceste trăsături puteau include culoarea, textura, forma leziunilor, grosimea vaselor sau distribuția spațială a exudatelor. După extragerea trăsăturilor, un clasificator precum SVM, k-NN sau Random Forest putea fi antrenat pentru a lua decizia finală. Avantajul acestor metode este interpretabilitatea relativ mai mare a unor trăsături proiectate manual. Dezavantajul este dependența puternică de calitatea acestor trăsături și dificultatea de a acoperi toată aria și diversitatea imaginilor reale [24], [17], [16].

Într-o abordare clasică, "pipeline"-ul era de obicei împărțit în etape distincte:

- preprocesarea imaginii și corectarea iluminării;
- extragerea manuală a trăsăturilor relevante;
- selectarea sau reducerea dimensionalității trăsăturilor;
- antrenarea unui clasificator clasic;
- validarea rezultatului pe imagini nefolosite la antrenare.

Învățarea profundă schimbă această paradigmă prin folosirea unor rețele neuronale cu multe straturi, capabile să învețe reprezentări ierarhice [25]. Într-o rețea convoluțională, primele straturi învață filtre simple, precum muchii, contraste locale sau texturi. Straturile intermediare combină aceste filtre în forme mai complexe, iar straturile profunde pot reprezenta structuri relevante pentru sarcina finală. Această ierarhie este potrivită pentru imagini medicale, deoarece semnele patologice apar la scări diferite și în contexte anatomice variate [25], [18].

În procesul de antrenare supervizată, modelul primește perechi imagine-etichetă și produce o predicție. Diferența dintre predicție și eticheta reală este măsurată printr-o funcție de pierdere, iar parametrii rețelei sunt actualizați prin algoritmi de optimizare, de obicei pe baza propagării inverse a gradientului. Pentru clasificarea multclasă, se pot utiliza mai multe tipuri de funcții de pierdere, fiecare abordând problema dintr-o perspectivă diferită:

- **Cross-Entropy (CE)** – este funcția standard pentru problemele de clasificare multclasă. Aceasta măsoară divergența dintre distribuția de probabilitate prezisă de model și distribuția reală a etichetelor. Funcționează optim atunci când setul de date este echilibrat, dar poate favoriza clasele majoritare în scenarii dezechilibrate.
- **Weighted Cross-Entropy (WCE)** – reprezintă o extensie a funcției standard, introdusă pentru a combate dezechilibrul claselor. Prin asocierea unei ponderi mai mari erorilor provenite din clasele rare (minoritare), modelul este penalizat mai sever dacă greșește în aceste cazuri, asigurând o învățare echitabilă și performanțe superioare pentru clasele critice.

- **Focal Loss** – este o funcție avansată concepută special pentru a rezolva problema dezechilibrului extrem și a exemplor greu de separat. Aceasta modifică ecuația standard prin introducerea unui factor de modulare care reduce semnificativ contribuția exemplorlor clasificate ușor și forțează modelul să se concentreze prioritar pe exemplele dificile (*hard examples*) [26].

Dezechilibrul claselor este o problemă frecventă în retinopatia diabetică. În multe seturi de date, clasa fără retinopatie este mult mai numeroasă decât clasele severe. Dacă modelul este antrenat fără corecții, acesta poate obține acuratețe bună prin favorizarea clasei dominante, dar poate avea sensibilitate slabă pentru clasele rare. Din acest motiv, evaluarea trebuie să urmărească performanța pe fiecare clasă, nu doar media globală. Tehnicile de reechilibrare includ ponderarea funcției de pierdere, eșantionarea echilibrată, augmentare mai puternică a claselor minoritare sau folosirea unor metrici robuste la dezechilibru [26], [22].

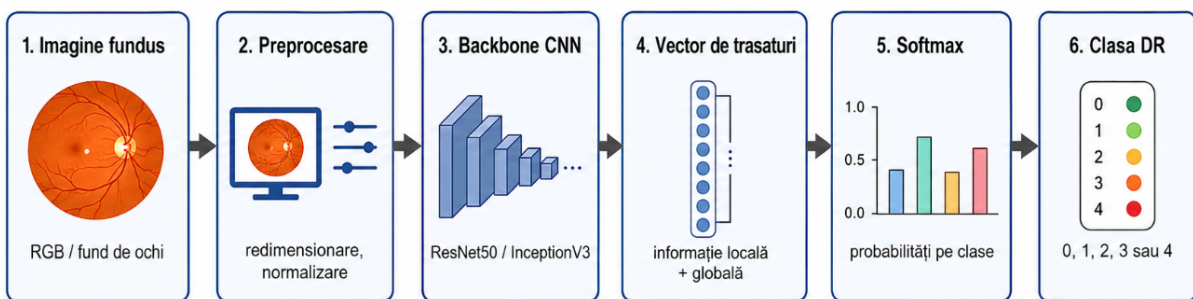


Figura 2.3: Flux conceptual pentru clasificarea imaginilor de fund de ochi cu o rețea neuronală convoluțională

În analiza imaginilor medicale, o altă provocare este dimensiunea limitată a datelor adnotate. Obținerea etichetelor clinice necesită timp și expertiză, iar datele pot fi afectate de restricții etice și de confidențialitate. Una dintre soluțiile folosite frecvent pentru această problemă este transfer learning-ul, prezentat separat la finalul capitolului [2].

2.7 Rețele neuronale convoluționale

Rețelele neuronale convoluționale, cunoscute sub abrevierea CNN, reprezintă una dintre cele mai importante arhitecturi pentru analiza imaginilor. Ele sunt concepute pentru a exploata structura spațială a datelor vizuale. Spre deosebire de o rețea complet conectată, care ar trata fiecare pixel independent de poziția sa, o rețea convoluțională aplică filtre locale pe imagine și păstrează relațiile spațiale dintre pixeli. Această proprietate este esențială pentru imagini retiniene, deoarece semnele patologice sunt definite de configurații locale de culoare, formă și textură [18], [7].

Operația de convoluție constă în deplasarea unui filtru peste imagine și calcularea unui răspuns local. Fiecare filtru învață să detecteze un anumit tipar vizual. În primele straturi, filtrele pot detecta margini, pete sau contraste. În straturile mai profunde, combinațiile de filtre pot detecta structuri mai complexe, precum vase, exudate sau zone hemoragice. Parametrii filtrelor sunt învățați automat în timpul antrenării, ceea ce elimină necesitatea proiectării manuale a descriptorilor vizuali [25], [18].

Pe lângă straturile convoluționale, CNN-urile includ mai multe componente recurente, așa cum se poate observa și în figura 2.4:

- **funcții de activare**, care introduc neliniaritate și permit modelului să aproximeze relații complexe;
- **operații de pooling**, care reduc dimensiunea spațială a hărților de trăsături;
- **straturi de normalizare**, care pot stabiliza antrenarea și accelera convergența;
- **straturi complet conectate sau globale**, care transformă trăsăturile extrase într-o predicție finală;
- **strat softmax**, folosit frecvent pentru a obține probabilități pe clase în clasificarea mult-clasă.

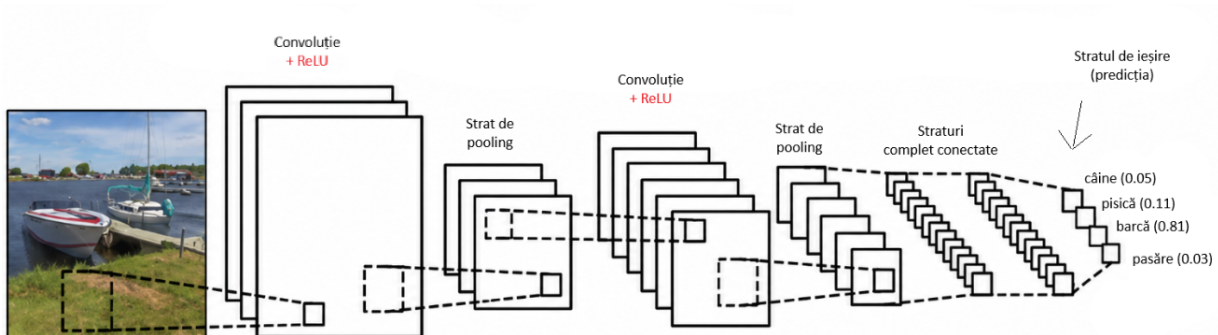


Figura 2.4: Schema generală a unei rețele neuronale convoluționale [11]

Arhitecturile CNN moderne diferă prin adâncime, tipul blocurilor folosite și modul în care gestionează propagarea informației. Dintre acestea, câteva arhitecturi s-au remarcat în mod special în analiza imaginilor medicale:

- **ResNet (Residual Networks)** – introduce conexiuni reziduale (“skip connections”), care permit transmiterea directă a informației peste unul sau mai multe straturi [27]. Această inovație atenuează problema dispariției gradientului și previne degradarea performanței în rețelele foarte adânci, facilitând învățarea unor reprezentări vizuale complexe.

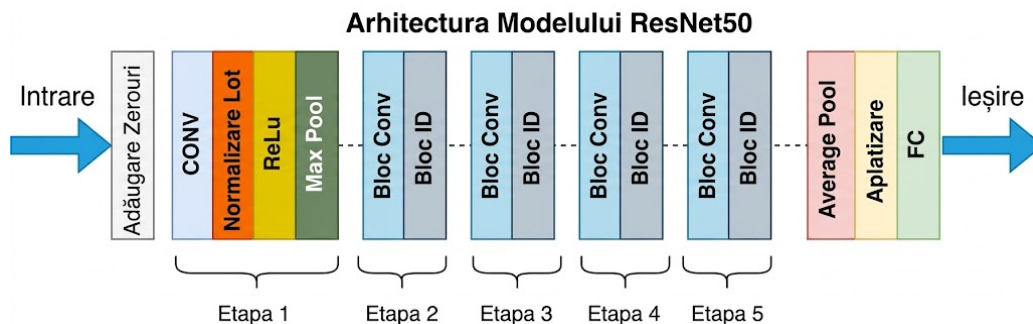


Figura 2.5: Schema unei arhitecturi ResNet50 [12]

- **Inception** – utilizează module care aplică simultan filtre convoluționale de dimensiuni diferite (ex. 1×1 , 3×3 , 5×5) pe aceeași intrare, capturând astfel informații la scări spațiale multiple [28]. Acest mecanism este crucial în retinopatia diabetică, unde patologia variază

de la leziuni minuscule (microanevrisme punctiforme) la zone masive (exudate extinse sau hemoragii).

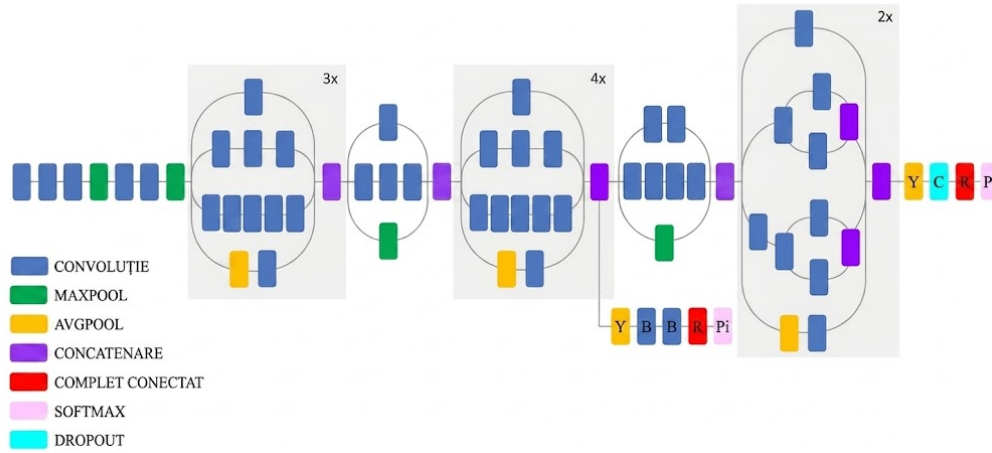


Figura 2.6: Schema unei arhitecturi Inception-v3 [13]

- **EfficientNet-B3** – face parte din familia EfficientNet, arhitecturi care folosesc o strategie de scalare compusă (*compound scaling*) pentru a crește simultan adâncimea, lățimea și rezoluția rețelei într-un mod echilibrat. Varianta B3 oferă un compromis între acuratețe și cost computațional, fiind potrivită pentru imagini medicale unde detaliile fine trebuie păstrate fără a crește excesiv complexitatea modelului. În contextul retinopatiei diabetice, această arhitectură poate ajuta la identificarea leziunilor subtile, precum microanevrismele sau exudatele mici, menținând în același timp capacitatea de a surprinde contextul global al imaginii fundus [29].

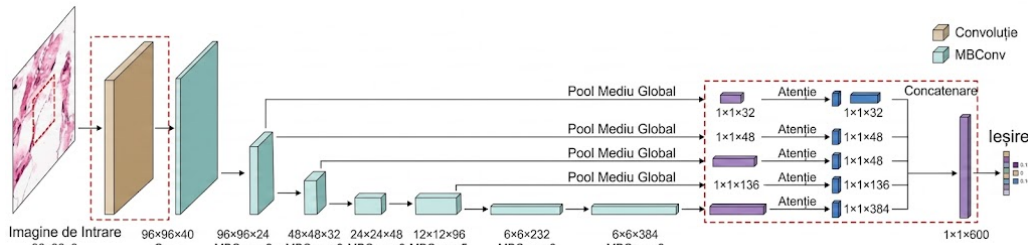


Figura 2.7: Schema unei arhitecturi EfficientNet-B3 [14]

- **Inception-ResNet** – reprezintă o arhitectură hibridă care combină eficiența de procesare multi-scală a modulelor Inception cu propagarea robustă a gradientului oferită de conexiunile reziduale. Această fuziune crește viteza de convergență și oferă o capacitate superioară de generalizare, aspect esențial pentru imaginile cu patologii retiniene complexe [2], [6].

Arhitectura Rețelei Inception Resnet V2

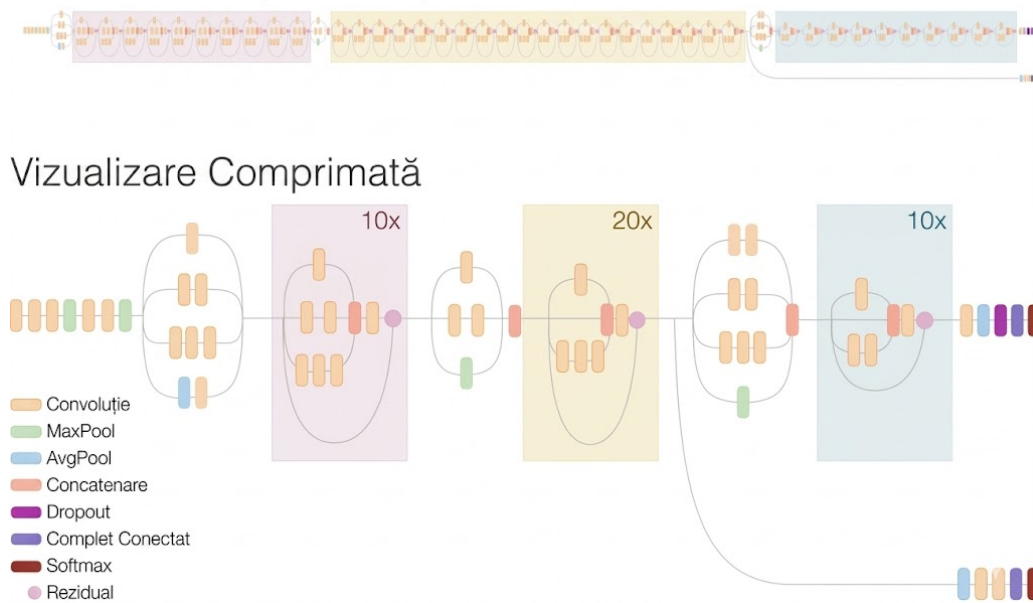


Figura 2.8: Schema unei arhitecturi Inception-ResNet-v2 [15]

În retinopatia diabetică, CNN-urile pot fi folosite în mai multe moduri:

- **clasificare globală**, în care modelul primește fotografia fundus și produce gradul de severitate;
- **detectie de leziuni**, în care modelul indică prezența unor semne patologice;
- **segmentare semantică**, în care modelul identifică regiunile afectate la nivel de pixel;
- **modele hibride sau multi-task**, care învață simultan clasificarea, localizarea și interpretarea leziunilor.

Segmentarea poate oferi informație mai detaliată și poate susține interpretabilitatea, dar necesită etichete "pixel-level", mai greu de obținut. Metodele hibride combină ramuri globale și locale pentru a integra contextul cu detaliile patologice [3], [5], [6].

Un aspect important al CNN-urilor este interpretabilitatea. Deși performanța lor poate fi ridicată, modelele profunde sunt adesea percepute ca sisteme de tip "cutie neagră". În medicină, această problemă este critică, deoarece deciziile trebuie să fie verificabile și explicabile. Tehnici precum hărțile de activare a claselor pot evidenția regiunile care au influențat predicția, oferind medicului o indicație vizuală asupra motivului pentru care modelul a ales o anumită clasă. Astfel de metode nu garantează corectitudinea deciziei, dar pot crește transparența și pot ajuta la detectarea cazurilor în care modelul se bazează pe artefacte sau regiuni irelevante [7].

2.8 Transfer learning și modele pre-antrenate

Transfer learning-ul este o tehnică prin care cunoștințele învățate într-o sarcină sunt reutilizate pentru o altă sarcină. În contextul imaginilor medicale, această tehnică este foarte importantă deoarece seturile de date adnotate sunt mai greu de obținut decât în domeniile generale ale vederii computerizate. Un model pre-antrenat pe milioane de imagini poate învăța

filtre generale pentru muchii, texturi, forme și compoziții vizuale. Chiar dacă imaginile naturale diferă de imaginile retiniene, primele straturi ale rețelei pot rămâne utile, iar straturile finale pot fi adaptate la clasificarea medicală [2], [25].

Există mai multe moduri de utilizare a transfer learning-ului:

- **Extractor de trăsături fix.** Straturile convoluționale sunt păstrate fixe, iar doar clasificatorul final este antrenat pe datele medicale.
- **Fine-tuning parțial.** Se ajustează doar ultimele blocuri ale rețelei, pentru a adapta reprezentările la imaginile retiniene.
- **Fine-tuning complet.** Toți parametrii modelului sunt actualizați, variantă utilă când există suficiente date și regularizare adecvată.

Fine-tuning-ul poate oferi performanțe mai bune, dar necesită atenție la rata de învățare, dimensiunea setului de date, regularizare și optimizatorul folosit, Adam fiind una dintre alegerile frecvente în antrenarea rețelelor profunde [2], [30].

În aplicații de retinopatie diabetică, arhitecturile pre-antrenate pot fi adaptate prin înlocuirea ultimului strat cu un clasificator pentru cele cinci clase de severitate și prin continuarea antrenării pe imaginile fundus disponibile [18], [2]. Detaliile arhitecturilor utilizate experimental sunt prezentate ulterior în metodologia propusă.

Transfer learning-ul nu elimină însă toate problemele. Dacă datele de antrenare sunt dezechilibrate sau conțin artefacte sistematice, modelul poate învăța corelații nedorite. De exemplu, dacă imaginile severe provin predominant de la o anumită cameră sau au o anumită rezoluție, modelul poate asocia caracteristicile tehnice cu boala. Această problemă este cunoscută ca bias de domeniu și afectează capacitatea de generalizare. Din acest motiv, este importantă evaluarea pe seturi de validare și testare reprezentative, precum și analiza erorilor prin matrice de confuzie și curbe ROC [8], [22].

În această lucrare, fundamentele prezentate în capitolul curent susțin partea experimentală ulterioară. Retinopatia diabetică este o problemă medicală cu impact ridicat, imaginile de fund de ochi oferă suportul vizual pentru screening automat, iar rețelele convoluționale permit învățarea directă a trăsăturilor relevante. Transfer learning-ul facilitează adaptarea arhitecturilor moderne la date medicale, iar metricile de evaluare permit compararea configurațiilor de antrenare. Împreună, aceste elemente definesc cadrul teoretic necesar pentru metodologia propusă și pentru interpretarea rezultatelor experimentale.

Capitolul 3

Stadiul actual al cercetării

3.1 Analiza seturilor de date disponibile

Performanța și capacitatea de generalizare a modelelor de învățare profundă sunt intrinsec dependente de calitatea, volumul și diversitatea datelor utilizate în procesul de antrenare. În contextul oftalmologiei computaționale, disponibilitatea bazelor de date publice a permis tranziția de la metodele clasice de procesare a imaginii la arhitecturi neuronale complexe, oferind un teren comun pentru evaluarea obiectivă a algoritmilor.

- **Seturi de referință pentru clasificarea globală (“Grading”)**

- **EyePACS** (Kaggle Diabetic Retinopathy Detection) [1], [7] rămâne cea mai vastă resursă publică disponibilă, conținând aproximativ 88.702 de imagini. Importanța sa constă în caracterul eterogen al imaginilor (variații de expunere, focalizare, artefacte), fiind critic pentru antrenarea unor rețele backbone robuste capabile să gestioneze fenomenul de “domain shift” [8] și să generalizeze pe imagini de calitate slabă
- **APTOS 2019 Blindness Detection** [7] reprezintă standardul modern de calitate. Colectat de Aravind Eye Hospital, acest set de 3.662 de imagini este utilizat frecvent în literatura recentă pentru “fine-tuning”, oferind o vizibilitate superioară a detaliilor retiniene comparativ cu seturile mai vechi
- **MESSIDOR-2** [7] este considerat în prezent „Benchmark-ul de Aur” pentru testarea algoritmilor. Extinzând setul original Messidor la 1.748 de imagini, acesta se distinge prin consistența diagnostică și este utilizat ca domeniu țintă (Target Domain) în studiile de adaptare [8] și pentru validarea sistemelor explicabile (XAI) [7], oferind o metrică fidelă a performanței reale pe date curate.
- **DeepDRiD** [22] aduce o inovație necesară prin structurarea celor 2.000 de imagini în perechi binoculare și includerea notelor de calitate, simulând fluxul de triaj clinic.
- **UoA-DR** [7] este o resursă istorică (200 imagini), utilizată complementar în studiile comparative pentru a asigura diversitatea demografică a validării.

- **Seturi specializate pentru segmentarea leziunilor și analiza detaliată**

- **IDRiD** [7] este setul critic pentru sarcinile de segmentare semantică fină [5]. Pentru 81 dintre imagini, experții au trasat manual măști binare la nivel de pixel, făcându-l indispensabil pentru antrenarea arhitecturilor complexe de tip Transformer și pentru validarea localizării leziunilor în sistemele end-to-end [7].
- **DDR** [21] a revoluționat segmentarea pe scară largă, oferind 13.673 de imagini. Este esențial pentru antrenarea rețelelor adânci de segmentare (precum RTNet) fără a risca “overfitting-ul”, oferind adnotări “pixel-wise” pentru leziuni multiple [5].

- **E-Ophtha** [7] este o bază de date vitală pentru reducerea alarmelor false în metodele de detecție a exudatelor [17]. Valoarea sa unică constă în includerea imaginilor cu patologii non-diabetice care pot fi confundate cu DR
- **DiaRetDB1** [7] este standardul istoric pentru calibrarea sensibilității la leziuni roșii (microanevrisme) în lucrările de detecție automată [24]. Complementar, HEI-MED [23] oferă 169 de imagini focalizate exclusiv pe segmentarea zonelor de Edem Macular Diabetic.

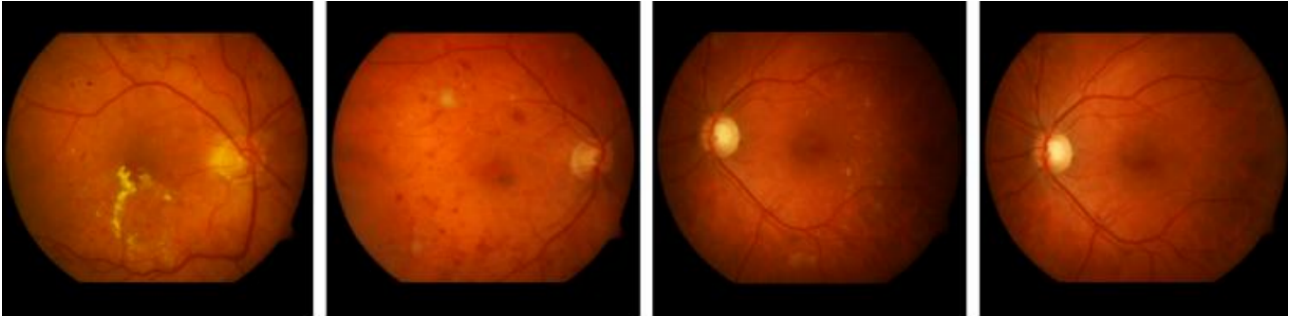


Figura 3.1: Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]

- **Seturi standard pentru preprocesarea vasculară**

- **DRIVE** [16] și **STARE** [7] sunt pilonii etapei de preprocesare. Deși conțin un număr redus de imagini, ele sunt utilizate obligatoriu pentru validarea algoritmilor de segmentare a vaselor de sânge, cum ar fi arhitecturile U-Net modificate [4] sau metodele clasice de extragere a trăsăturilor [24].

Tabelul de mai jos sintetizează caracteristicile tehnice esențiale ale seturilor de date analizate, evidențiind diferențele de volum, rezoluție și tipul de adnotare disponibil, elemente care dictează utilizarea lor în diversele etape ale proiectului.

Tabela 3.1: Tabel comparativ între seturile de date disponibile

Set de date	Număr imagini (total)	Rezoluție tipică (px)	Tip adnotare	Utilizare principală
EyePACS	88.702	Variabilă, până la 4000+	Globală, clase 0–4	Antrenare backbone și generalizare
DDR	13.673	Variabilă	Pixel-wise, măști și bounding box	Segmentare la scară largă
APTOS 2019	3.662	Înaltă, standardizată	Globală, clase 0–4	Fine-tuning și validare
DeepDRiD	2.000	512×512, reconstruit	Globală, calitate și binocular	Analiza calității imaginii
MESSIDOR-2	1.748	1440×960 – 2240×1488	Globală și edem	Benchmark și testare
IDRiD	516	4288×2848	Pixel-wise pentru 81 imagini și globală	Segmentare semantică fină
E-Ophtha	381 EX / 381 MA	Variabilă, medie/mare	Contur leziuni EX/MA	Reducerea alarmelor false pozitive
UoA-DR	~200	Variabilă	Globală, clase 0–4	Validare încrucișată
HEI-MED	169	2196×1958	Măști edem	Detecție edem macular
DiaRetDB1	89	1500×1152	Coordonate punctuale	Calibrare microanevrisme
DRIVE	40	565×584	Măști vase binare	Preprocesare și segmentare vase de sânge
STARE	20	700×605	Măști vase binare	Preprocesare și segmentare vase de sânge

3.2 Metoda bazată pe arhitectura Inception-V3 [1]

Studiul realizat de Gulshan et al. [1] este considerat un reper major în literatura de specialitate, marcând una dintre primele validări clinice ale utilizării rețelelor neuronale convoluționale pentru screening-ul automat al retinopatiei diabetice. Lucrarea demonstrează că un sistem de deep learning poate atinge și chiar depăși performanța medie a medicilor oftalmologi în detectarea retinopatiei diabetice, contribuind decisiv la acceptarea inteligenței artificiale în aplicații medicale reale [1].

Arhitectura utilizată este Inception-v3, o rețea convoluțională profundă caracterizată prin utilizarea modulelor Inception, care combină în paralel convoluții de dimensiuni diferite (1×1, 3×3, 5×5) și operații de pooling. Această strategie permite captarea informațiilor la multiple scări spațiale și reduce costul computațional prin utilizarea convoluțiilor 1×1 pentru diminuarea dimensionalității. Datorită acestei structuri, modelul este capabil să învețe reprezentări vizuale complexe, esențiale pentru identificarea leziunilor subtile specifice retinopatiei diabetice.

Modelul a fost antrenat pe un set de date extins, alcătuit din peste 128.000 de imagini fundus color, etichetate independent de mai mulți medici oftalmologi certificați în 2015. Utilizarea consensului între experți a contribuit la reducerea subiectivității etichetelor și la creșterea fiabilității procesului de antrenare. Sarcina principală a rețelei a fost clasificarea imaginilor în funcție de prezența sau absența retinopatiei diabetice referabile, conform standardelor clinice acceptate internațional [1].

Preprocesarea imaginilor a inclus decuparea automată a regiunii retiniene, redimensionarea la o rezoluție compatibilă cu Inception-v3 și normalizarea intensității pixelilor. În plus, au

fost aplicate tehnici extensive de augmentare a datelor, precum rotații, translații și ajustări de luminozitate și contrast, pentru a îmbunătăți generalizarea și a reduce supraînvățarea [1] [2].

Evaluarea performanței a fost realizată pe seturile de date EyePACS-1 și Messidor-2 [7], unde modelul a obținut o arie sub curba ROC (AUC) de aproximativ 0.99 pentru detectarea retinopatiei diabetice referabile, cu valori ridicate de sensibilitate, comparabile cu performanța medicilor oftalmologi experimentați [1] [18].

3.3 “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]

Lucrarea propusă de Jabbar et al. [2] abordează problema diagnosticului automat al retinopatiei diabetice dintr-o perspectivă practică, concentrându-se pe utilizarea tehnicilor de “transfer learning” pentru a compensa limitările impuse de dimensiunea redusă a seturilor de date medicale etichetate. Studiul evidențiază faptul că modelele CNN pre-antrenate pe seturi de date generale de mari dimensiuni, precum ImageNet, pot fi adaptate eficient pentru analiza imaginilor fundus, obținând performanțe competitive cu un cost de antrenare semnificativ redus [2].

Autorii investighează mai multe arhitecturi consacrate de rețele neuronale convoluționale, printre care VGG16, VGG19, ResNet50, Inception-v3 și DenseNet, utilizate ca extractoare de caracteristici. Strategia de transfer learning constă în inițializarea rețelelor cu ponderi pre-antrenate, urmată de ajustarea straturilor superioare pentru sarcina specifică de clasificare a retinopatiei diabetice. În unele experimente, straturile convoluționale inferioare sunt înghețate pentru a preveni supraînvățarea, în timp ce în altele este aplicat “fine-tuning” progresiv pentru a îmbunătăți adaptarea la domeniul imagisticii medicale. Una din abordările propuse poate fi observată în figura următoare.

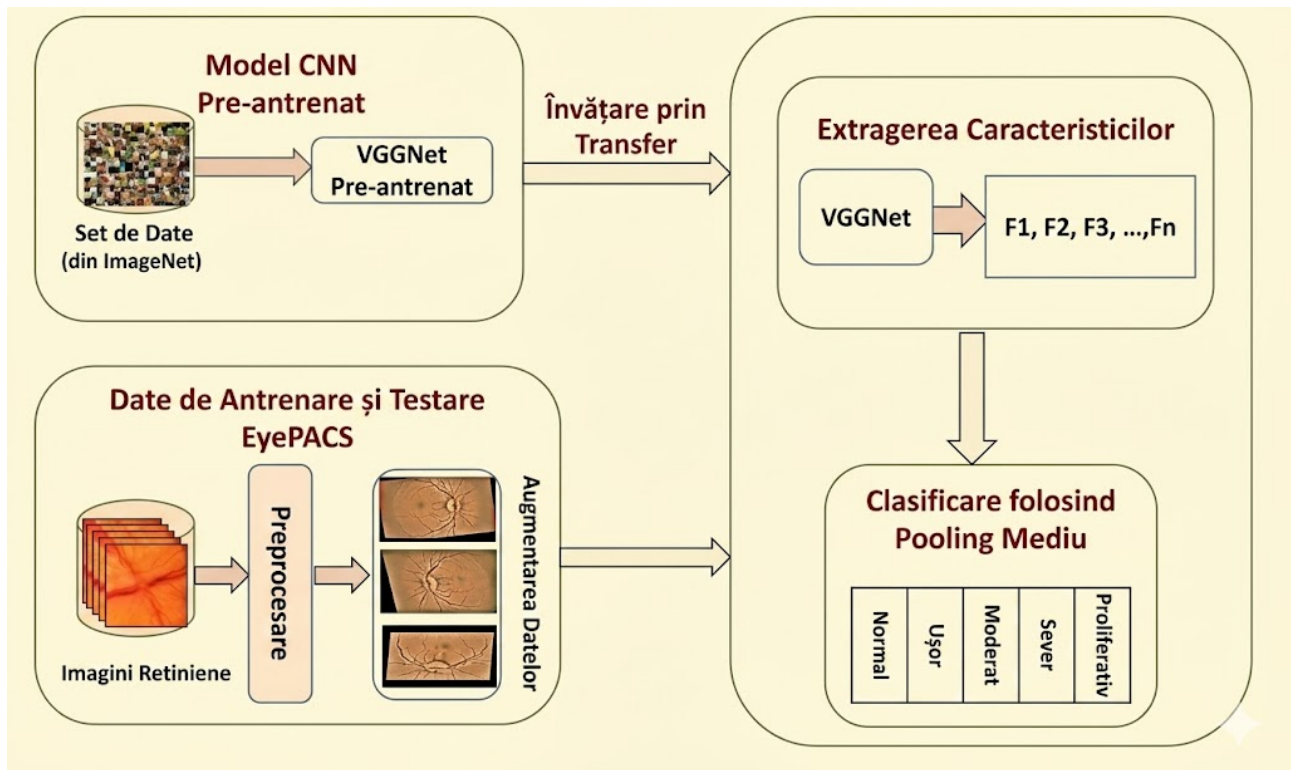


Figura 3.2: Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2]

Seturile de date utilizate includ imagini fundus publice din Kaggle EyePACS [7], preprocesate prin redimensionare, normalizare și augmentare extensivă a datelor. Tehnicile de augmentare, precum rotațiile, flip-urile orizontale și ajustările de luminosități, joacă un rol central în creșterea diversității artificiale a datelor și în îmbunătățirea capacității de generalizare a modelelor. Această etapă este esențială în contextul “transfer learning-ului”, unde diferențele de distribuție dintre datele naturale și imaginile medicale pot afecta performanța rețelelor [2].

Evaluarea performanței este realizată folosind metrici standard, precum acuratețea, sensibilitatea, specificitatea și scorul F1, permițând o comparație riguroasă între diferitele arhitecturi analizate. Rezultatele experimentale indică faptul că modelele bazate pe DenseNet și ResNet obțin performanțe superioare față de arhitecturile mai vechi, confirmând avantajele reutilizării caracteristicilor vizuale generale în sarcini medicale specializate. Studiul subliniază că transfer learning-ul reprezintă o soluție viabilă și eficientă pentru implementarea rapidă a sistemelor automate de screening în contexte cu resurse limitate [2].

3.4 “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]

Lucrarea propusă de Wang et al. [3] introduce o abordare de tip “deep multi-task learning” pentru gradarea automată a retinopatiei diabetice, având ca obiectiv principal exploatarea relației clinice dintre severitatea bolii și prezența leziunilor retiniene specifice. Spre deosebire de metodele clasice de clasificare “single-task”, autorii demonstrează că învățarea simultană a mai multor sarcini corelate clinic conduce la reprezentări vizuale mai discriminative și la performanțe superioare în sarcina principală de gradare [3].

Arhitectura propusă este alcătuită dintr-un backbone convoluțional comun, bazat pe o rețea CNN profundă (ResNet18), utilizat pentru extracția caracteristicilor globale din imaginile fundus. Peste acest extractor comun sunt construite mai multe ramuri specializate (task-specific branches), fiecare corespunzătoare unei sarcini distincte. Sarcina principală constă în clasificarea severității retinopatiei diabetice în mai multe stadii, în timp ce sarcinile auxiliare vizează detectarea prezenței diferitelor tipuri de leziuni, precum microanevrismele, exudatele dure, exudatele moi și hemoragiile, considerate “markeri” clinici relevanți ai progresiei bolii [3].

Un element central al arhitecturii îl reprezintă mecanismul de partajare controlată a caracteristicilor între sarcini. În locul unei separări rigide între ramuri, modelul permite propagarea informației relevante între sarcina de gradare și cele de detecție a leziunilor, încurajând învățarea unor reprezentări comune sensibile la semnalele patologice fine. Funcția de pierdere globală este definită ca o combinație ponderată a pierderilor individuale asociate fiecărei sarcini:

$$L_{isr} = \lambda_{img} \cdot L_{img_isr} + \lambda_{tv} \cdot L_{tv_isr} + \lambda_{sa} \cdot L_{seg-aware} + \lambda_{ca} \cdot L_{cls-aware}$$

,unde λ_{img} , λ_{tv} , λ_{sa} și λ_{ca} sunt hiperparametri care controlează contribuția pierderilor “intra-task”, “segmentation-aware” și “classification-aware”. Această formulare permite echilibrarea influenței fiecărei sarcini și stabilizează procesul de optimizare [3].

Modelul este antrenat și evaluat pe seturi de date publice de imagini fundus etichetate (DDR [21] și EyePACS [7]) atât cu stadiul retinopatiei diabetice, cât și cu adnotări la nivel de imagine privind prezența leziunilor. Imaginile sunt supuse unor etape standard de preprocesare, incluzând decuparea regiunii retiniene, redimensionarea și augmentarea datelor, pentru a reduce variațiile de iluminare și pentru a îmbunătăți capacitatea de generalizare. Prin includerea sarcinilor auxiliare, modelul beneficiază de un efect de regularizare implicită, reducând riscul

de supraînvăţare, aspect observat şi în lucrările anterioare bazate pe CNN-uri “single-task” [1], [18].

Rezultatele experimentale raportate în articol indică îmbunătăţiri consistente ale performanţei faţă de modelele “single-task”, atât în termeni de acurateţe, cât şi de scor F1 pentru sarcina de gradare a retinopatiei diabetice. În plus, autorii evidenţiază o creştere a stabilităţii predicţiilor şi o sensibilitate mai bună la stadiile intermediare ale bolii, considerate dificil de clasificat. Aceste rezultate confirmă faptul că integrarea informaţiilor despre leziuni în procesul de antrenare contribuie direct la o evaluare mai precisă a severităţii retinopatiei diabetice şi validează eficienţa paradigmei “multi-task learning” în context clinic [3].

3.5 Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]

Segmentarea semantică a structurilor retiniene, în special a reţelei vasculare şi a leziunilor asociate retinopatiei diabetice, reprezintă o etapă critică în diagnosticarea automată, deoarece modificările morfologice ale vaselor de sânge sunt adesea primii indicatori ai bolii. Deşi arhitectura UNet, introdusă iniţial de Ronneberger et al. [31] pentru segmentarea biomedicală, a devenit standardul de aur în domeniu datorită structurii sale simetrice de tip “encoder-decoder”, aceasta prezintă limitări în detectarea vaselor capilare foarte fine, pierzând informaţii spaţiale esenţiale în procesul de subeşantionare.

În studiul “Modified U-Net architecture for semantic segmentation of diabetic retinopathy images”, Sambyal et al [4]. propun o optimizare arhitecturală semnificativă pentru a contracara aceste deficienţe. Modelul propus, denumit Modified U-Net, păstrează structura în formă de „U” a reţelei originale, dar inovează fundamental la nivelul blocurilor de procesare.

Autorii au înlocuit blocurile convoluţionale standard cu blocuri reziduale (residual blocks), inspirate din arhitecturile de tip ResNet. Această modificare este crucială din punct de vedere tehnic: în reţelele adânci, apare frecvent problema dispariţiei gradientului (vanishing gradient), ceea ce face antrenarea dificilă şi duce la pierderea detaliilor fine. Prin introducerea conexiunilor reziduale (“skip connections” la nivel de bloc), informaţia originală este propagată mai eficient prin straturi, permiţând reţelei să înveţe caracteristici complexe fără a degrada performanţa pe detaliile de fineţe, precum microanevrismele sau vasele subţiri. Aceste detalii se pot observa şi în figura următoare.

Importanţa acestei abordări constă în capacitatea de generalizare comparativ cu metodele clasice de procesare a imaginii, precum cele bazate pe “clustering fuzzy” propuse anterior de Sopharak et al. [17]. În timp ce metodele de tip “clustering” sunt sensibile la zgomot şi la variaţiile de iluminare (frecvente în imaginile de fund de ochi), arhitectura Modified U-Net demonstrează o robusteţe sporită prin învăţarea contextuală a trăsăturilor

Pentru validarea performanţei, metoda a fost testată pe seturile de date publice standardizate DRIVE [16] şi STARE [7], recunoscute pentru dificultatea lor în segmentarea vaselor.

Rezultatele experimentale raportate de Sambyal et al. [4] au indicat o îmbunătăţire clară faţă de U-Net-ul standard. Pe setul de date DRIVE, autorii au obţinut o acurateţe maximă de 96.94 sub curba ROC (AUC) de 0.98, valori care plasează metoda în topul soluţiilor de segmentare. Mai mult, analiza vizuală a rezultatelor arată o continuitate mult mai bună a vaselor de sânge segmentate şi o reducere a alarmelor false în zonele cu patologii, demonstrând că integrarea învăţării reziduale în arhitectura de segmentare este o direcţie esenţială pentru analiza detaliată a retinopatiei diabetice.

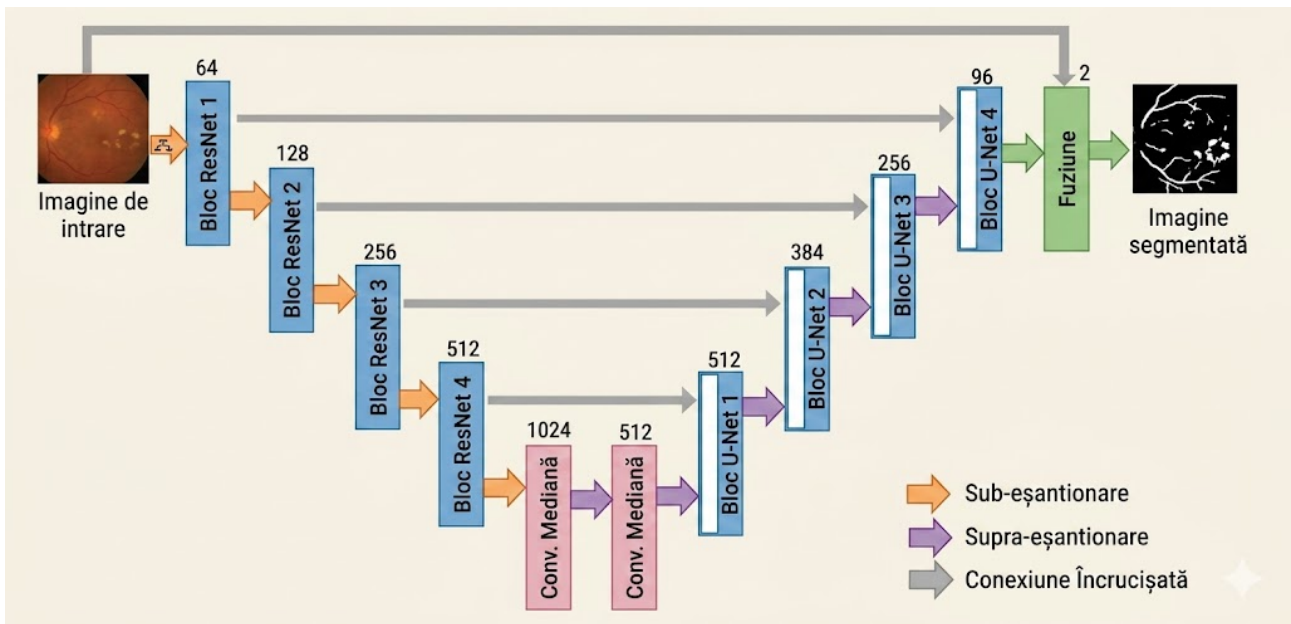


Figura 3.3: Arhitectura U-net modificată [4]

3.6 Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]

Dacă modelele bazate pe U-Net, precum cel discutat anterior [4], excelează în capturarea detaliilor locale, ele suferă totuși de o limitare inerentă a rețelelor convoluționale: incapacitatea de a modela dependențe la distanță mare (long-range dependencies). În patologia retiniană, leziunile nu apar izolat; există adesea o corelație spațială între poziția vaselor de sânge și apariția microanevrismelor sau a exudatelor. Pentru a adresa această lipsă de „context global”, Huang et al. [5] au propus în 2022 o schimbare de paradigmă prin introducerea RTNet (Relation Transformer Network), o arhitectură care marchează trecerea către mecanisme de atenție.

Inovația centrală a RTNet constă în înlocuirea parțială a operațiilor convoluționale standard cu blocuri de tip “Transformer”, denumite Relation Transformer Blocks (RTB). Spre deosebire de CNN-uri care „văd” doar o fereastră mică de pixeli la un moment dat, RTB-ul utilizează două mecanisme de atenție complementare:

1. “Self-Attention” (Atenție proprie): Permite modelului să înțeleagă relațiile dintre leziunile dispersate pe toată suprafața retinei (de exemplu, grupuri de exudate dure care semnalează o zonă de edem).
2. “Cross-Attention” (Atenție încrucișată): Autorii au observat că leziunile vasculare sunt strâns legate de structura vaselor de sânge. Astfel, mecanismul de “cross-attention” folosește harta vaselor de sânge ca un „ghid” pentru a localiza mai precis leziunile fine, transferând informație structurală dinspre ramura vasculară către ramura de detectare a leziunilor.

Pentru validare, metoda a fost testată pe seturile de date IDRiD [7] și DDR [21], care conțin adnotări la nivel de pixel pentru multiple tipuri de leziuni. Rezultatele cantitative au demonstrat că RTNet depășește metodele existente anterioare (inclusiv diverse variante de U-Net și DeepLab), obținând cele mai bune scoruri de segmentare pentru Exudate (EX), Microanevrisme (MA) și Exudate Moi (SE).

Deși inovația teoretică a arhitecturii RTNet (Relation Transformer Network) propusă de Huang et al. [5] este evidentă prin integrarea mecanismelor de atenție, validarea practică a acesteia necesită o demonstrație riguroasă a faptului că fiecare modul propus contribuie activ la îmbunătățirea segmentării. În acest sens, autorii au realizat un studiu extensiv de ablație (ablation study), analizând impactul incremental al componentelor arhitecturale asupra capacității de învățare și generalizare a rețelei.

Această analiză comparativă este sintetizată în graficele de performanță care monitorizează evoluția funcției de pierdere (Loss) și curbele Precision-Recall (PR) pentru cele patru tipuri principale de leziuni asociate retinopatiei diabetice: microanevrisme (MA), exudate (EX), hemoragii (HE) și exudate moi (SE).

Comparatia pornește de la un model “Baseline” (o rețea convoluțională standard de tip U-Net fără mecanisme de atenție) și adaugă progresiv componentele specifice RTNet, regasite în figura de mai jos:

1. GTB (Global Transformer Block): Adăugarea capacității de procesare globală.
2. SAH (Self-Attention Head): Introducerea atenției proprii pentru corelarea leziunilor similare.
3. RAH (Relation Attention Head / Cross-Attention): Modulul critic care leagă leziunile de structura vaselor de sânge.
4. RTB (Relation Transformer Block): Arhitectura completă propusă.

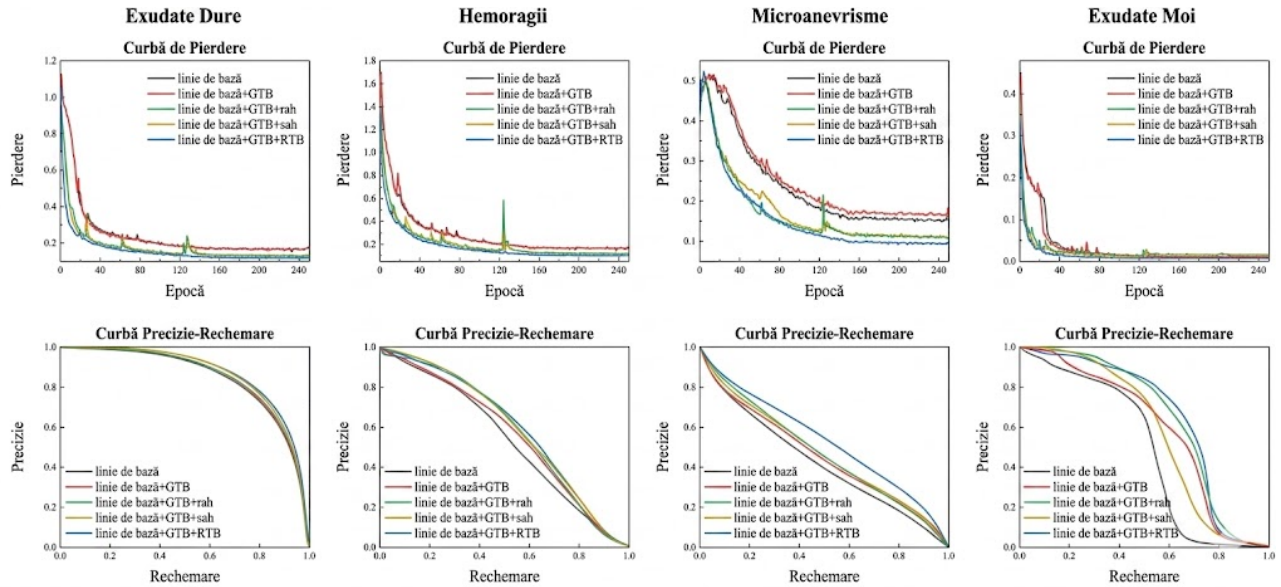


Figura 3.4: Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]

Lucrarea lui Huang et al. [5] validează ipoteza că integrarea informației globale prin Transformere este pasul necesar pentru a depăși plafonul de performanță atins de rețelele convoluționale pure în segmentarea medicală complexă.

3.7 Arhitectura “Dual Branch” [6]

Deși progresele în segmentarea semantică, ilustrate prin metodele bazate pe U-Net [4] sau pe Transformers [5], au permis o delimitare precisă a leziunilor, translatarea acestor detalii într-un diagnostic global (grading) rămâne o provocare majoră din cauza naturii subtile a diferențelor dintre stadiile retinopatiei diabetice. Rețelele convoluționale standard utilizate în studiile fundamentale, precum cel al lui Gulshan et al. [1], tind să piardă detaliile fine, cum ar fi microanevrismele incipiente, în procesul de redimensionare a imaginii necesar pentru analiza globală. Pentru a depăși acest compromis inerent între contextul vizual și rezoluția detaliilor, Shakibania et al. [6] au propus o arhitectură inovatoare de tip “Dual Branch Deep Learning Network”, care restructurează procesul de învățare prin decuplarea extragerii trăsăturilor în două fluxuri paralele distincte.

Conceptul central al acestei abordări constă în imitarea procesului cognitiv uman de diagnosticare, procesând imaginea simultan pe o „ramură globală” și o „ramură locală”. Ramura globală analizează imaginea completă a fundului de ochi pentru a captura structura generală a vaselor și geometria retinei, o strategie similară cu metodele clasice de “Transfer Learning” [2]. În paralel, ramura locală operează pe decupaje de înaltă rezoluție, fiind specializată în identificarea leziunilor microscopice care ar fi invizibile pentru rețeaua globală. Elementul de noutate adus de această lucrare față de abordările “multi-task” anterioare [3], este mecanismul de fuziune a celor două fluxuri, care utilizează tehnici de atenție pentru a pondera importanța trăsăturilor locale în decizia finală, asigurând că un detaliu critic nu este ignorat în favoarea unei imagini de ansamblu aparent sănătoase.

Eficiența acestei arhitecturi hibride este validată prin rezultatele experimentale obținute pe seturile de date standard, unde modelul “Dual Branch” a demonstrat o superioritate clară în clasificarea stadiilor intermediare ale bolii (ușor și moderat). Analiza performanței indică faptul că integrarea informației locale crește semnificativ sensibilitatea modelului și reduce confuzia între clasele adiacente, oferind o robustețe superioară la variațiile de calitate a imaginii comparativ cu arhitecturile “singlestream”. Astfel, lucrarea confirmă faptul că performanța actuală în 2024 nu derivă doar din adâncimea rețelei, ci din capacitatea arhitecturală de a reuni viziunea macroscopică cu cea microscopică într-un diagnostic coerent.

3.8 Abordarea “Explainable AI” [7]

În timp ce arhitecturile precum “Dual Branch Network” [6] sau Transformer-ele relaționale [5] au reușit să împingă limitele acurateței și ale segmentării fine, integrarea acestor sisteme în fluxul clinic real se lovește de o barieră fundamentală: lipsa de încredere a medicilor în algoritmi de tip „black-box”. Validarea clinică inițiată de Gulshan et al. [1] a demonstrat potențialul diagnostic, însă pentru ca un sistem de inteligență artificială să fie acceptat ca asistent de încredere, acesta trebuie să își poată justifica deciziile. În acest context, lucrarea publicată de Chetoui și Akhloufi [7], reprezintă un pilon esențial al stadiului actual al cunoașterii, adresând simultan două probleme critice: interpretabilitatea (Explainable AI - XAI) și robustețea pe seturi de date eterogene.

Metodologia propusă de autori se bazează pe o arhitectură de tip CNN modificată (utilizând backbone-uri precum DenseNet121 sau ResNet50), în care straturile clasice complet conectate sunt înlocuite cu Global Average Pooling (GAP). Această modificare tehnică permite generarea Hărților de Activare a Claselor (CAM), oferind medicului o confirmare vizuală a regiunilor patologice care au determinat diagnosticul.

Relevanța clinică a acestei abordări extensive este evidențiată în rezultatele vizuale, unde

hărțile de saliență generate de algoritm se suprapun cu precizie peste semnele patologice reale, indiferent de setul de date din care provine imaginea. Zonele marcate cu roșu intens pe hărțile CAM corespund anatomic cu microanevrismele și exudatele, confirmând că rețeaua neuronală a învățat trăsăturile intrinseci ale bolii și nu caracteristicile specifice ale camerei foto (bias-ul echipamentului). Prin urmare, modelul propus [7] validează faptul că un sistem trebuie să fie nu doar precis, ci și independent de sursa datelor și transparent în decizii.

3.9 Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]

Ultima frontieră în dezvoltarea sistemelor de diagnostic asistat de calculator (CAD) pentru retinopatia diabetică nu mai este legată strict de acuratețea obținută pe un singur set de date de antrenament, ci de capacitatea algoritmului de a funcționa corect într-un mediu clinic eterogen. În realitate, imaginile de fund de ochi variază drastic în funcție de modelul camerei, setările de iluminare, etnia pacientului sau protocolul de achiziție, fenomen cunoscut sub numele de “domain shift”. Modelele antrenate pe un „domeniu sursă” (de exemplu, imagini de înaltă calitate de la EyePACS [7]) tind să aibă performanțe scăzute atunci când sunt aplicate pe un „domeniu țintă” (imagini clinice noi, cu caracteristici vizuale diferite), ceea ce limitează sever aplicabilitatea lor universală.

Pentru a soluționa această problemă critică de implementare, Zhang et al. [8] au propus în 2022 o metodă inovatoare denumită “Multi-Model Domain Adaptation” (MMDA). Spre deosebire de abordările anterioare care încearcă să alinieze domeniile folosind un singur model, autorii introduc o arhitectură complexă bazată pe o rețea siameză, utilizând două modele distincte pentru a extrage trăsături complementare și pentru a crește robustețea deciziei.

Inovația tehnică centrală a lucrării constă în integrarea simultană a trei mecanisme de optimizare care lucrează concertat pentru a forța modelul să ignore diferențele stilistice dintre seturile de date și să se concentreze exclusiv pe patologie. Astfel, arhitectura combină o pierdere de clasificare pentru a asigura acuratețea diagnostică pe domeniul sursă etichetat, cu o pierdere de discrepanță care maximizează diferența dintre predicțiile celor două clasificatoare pe domeniul țintă pentru a detecta exemplele aflate la granița de decizie. În plus, se utilizează o pierdere de adaptare adversarială care, prin intermediul unui discriminator, minimizează distanța dintre distribuția trăsăturilor din sursă și cea din țintă, făcând ca imaginile provenite din spitale diferite să fie reprezentate similar de către rețeaua neuronală.

Validarea experimentală a metodei a fost realizată folosind setul de date EyePACS [7] ca domeniu sursă și Messidor-2 [7] ca domeniu țintă, simulând un scenariu realist de transfer tehnologic de la imagini de screening la imagini clinice de referință. Rezultatele au demonstrat că abordarea MMDA depășește semnificativ metodele standard de antrenare și alte tehnici de adaptare cunoscute, obținând o îmbunătățire substanțială a metricilor de performanță pe domeniul țintă fără a necesita etichete pentru acesta. Această lucrare încheie logic analiza stadiului actual al cunoașterii, subliniind faptul că un sistem complet nu trebuie doar să segmenteze leziuni fine sau să fie explicabil, ci trebuie să fie capabil să iasă din mediul controlat al laboratorului și să ofere o performanță robustă și generalizabilă în diversele condiții întâlnite în practica medicală globală.

3.10 Concluzii

Analiza literaturii de specialitate relevă faptul că detectarea automată a retinopatiei diabetice a depășit stadiul de experiment teoretic, devenind o direcție de cercetare matură, cu rezultate clinice valide. Totuși, problema rămâne una de o complexitate ridicată, care depășește simpla clasificare a imaginilor. Dificultatea majoră nu constă doar în volumul masiv de date necesar, ci în subtilitatea semnelor patologice incipiente, precum microanevrismele, care pot ocupa doar câțiva pixeli dintr-o imagine de înaltă rezoluție, fiind ușor de confundat cu artefactele sau zgomotul de fond. Această granularitate fină impune algoritmilor o capacitate de discriminare vizuală care, în anumite scenarii limită, trebuie să depășească acuitatea ochiului uman.

În ciuda acestor obstacole, soluțiile algoritmice existente demonstrează o evoluție remarcabilă. Trecerea de la rețelele convoluționale standard la arhitecturi sofisticate, capabile să integreze informația globală cu cea locală prin mecanisme de tip “dual-branch” [6], a permis o creștere semnificativă a sensibilității în detectarea stadiilor intermediare ale bolii. Mai mult, rezultatele obținute prin tehnici de adaptare la domeniu [8] sugerează că bariera variabilității echipamentelor medicale începe să fie depășită, deschizând calea pentru sisteme robuste, capabile să funcționeze corect indiferent de spitalul în care sunt implementate.

Un alt aspect promițător este schimbarea de paradigmă de la modelele de tip „cutie neagră” la sistemele explicabile (Explainable AI). Așa cum evidențiază studiile recente [7], capacitatea modelelor de a genera hărți de activare precise transformă inteligența artificială dintr-un simplu clasificator statistic într-un partener transparent pentru medic, facilitând validarea rapidă a deciziei. Deși provocările legate de dezechilibrul claselor și de generalizarea pe populații diverse persistă, performanțele atinse de metodele actuale, validate pe seturi de date riguroase precum MESSIDOR-2 [7], confirmă potențialul acestor tehnologii de a eficientiza screening-ul în masă și de a preveni orbirea la scară globală.

Capitolul 4

Seturi de date și construirea dataset-urilor experimentale

4.1 Rolul seturilor de date în lucrare

Clasificarea automată a retinopatiei diabetice depinde puternic de calitatea imaginilor, de distribuția claselor și de modul în care datele sunt separate înainte de antrenare. În literatură, diferențele dintre seturi precum EyePACS, APTOS, DDR, IDRiD sau Messidor sunt discutate frecvent în raport cu robustețea modelelor și cu problema schimbării domeniului vizual (*domain shift*) [1], [8], [7], [22], [21]. Din acest motiv, în această lucrare seturile de date nu au fost tratate ca simple directoare de imagini, ci ca o componentă de metodologie centrală a sistemului.

O parte importantă a lucrării a constatat în construirea, curățarea și verificarea mai multor variante de seturi de date. Imaginile au fost reorganizate local, filtrate, împărțite în subseturi, augmentate și preprocesate prin fluxuri reproductibile. Acest efort a fost necesar deoarece rezultate aparent foarte bune pot fi obținute artificial atunci când imagini derivate din aceeași sursă ajung simultan în antrenare și în validare sau testare. În consecință, capitolul prezintă nu doar sursele de date, ci și munca practică realizată pentru a transforma aceste surse în seturi experimentale controlate.

Pașii urmăriți în construirea dataset-urilor au fost:

- identificarea și descărcarea surselor publice relevante pentru clasificarea retinopatiei diabetice;
- verificarea structurii inițiale a fiecărui set și numărarea imaginilor pe clase;
- reorganizarea imaginilor într-un format comun `train/val/test`, fiecare subset fiind împărțit în clasele 0-4;
- curățarea imaginilor invalide, foarte neclare sau fără regiune retiniană detectabilă;
- aplicarea augmentărilor doar pe subsetul de antrenare;
- generarea variantelor preprocesate prin metoda Ben Graham;
- verificarea riscului de *data leakage*, mai ales în seturile care conțin augmentări preexistente.

Au fost folosite mai multe surse de date: EyePACS, APTOS 2019, `Diabetic_Balanced_Data` și `Diabetic_Balanced_Aug`. Modelul principal al lucrării este construit pe EyePACS, într-un pipeline local CUDA, iar celelalte seturi au fost folosite pentru experimente preliminare, comparații între arhitecturi și analiza robustă a protocolului de evaluare. Toate variantele finale au fost aduse, pe cât posibil, la o structură comună de tip:

```
dataset/
  train/0  train/1  train/2  train/3  train/4
  val/0    val/1    val/2    val/3    val/4
  test/0   test/1   test/2   test/3   test/4
```

Clasele numerice corespund celor cinci niveluri clinice ale retinopatiei diabetice: clasa 0 reprezintă absența bolii, clasa 1 forma ușoară, clasa 2 forma moderată, clasa 3 forma severă, iar clasa 4 forma proliferativă.

4.2 Sursele de date utilizate

Setul EyePACS a fost utilizat ca sursă principală pentru sistemul final. Acesta conține un număr mare de imagini de fund de ochi, dar are o distribuție puternic dezechilibrată: clasa 0 este dominantă, în timp ce clasele severe sunt mult mai puțin reprezentate. Dezechilibrul este o caracteristică întâlnită frecvent în screening-ul pentru retinopatie diabetică, unde imaginile normale sau cazurile moderate apar mai des decât formele severe [1], [21]. Tocmai din acest motiv, EyePACS este relevant pentru scenarii apropiate de practica reală, dar impune atenție la metricile folosite și la strategia de antrenare.

Setul APTOS 2019 a fost folosit pentru experimente comparative și pentru analiza efectului augmentării și al preprocesării Ben Graham. Imaginile APTOS provin dintr-o distribuție diferită de EyePACS, astfel încât ele permit observarea sensibilității modelelor la schimbarea domeniului vizual. Această diferență de domeniu este importantă deoarece modelele antrenate pe un singur set pot învăța particularități ale camerei, iluminării sau rezoluției, nu doar semne clinice ale bolii [8], [7].

Din punct de vedere experimental, APTOS este valoros deoarece are un volum mai redus decât EyePACS, dar imagini relativ bine standardizate și etichete pe aceeași scară de severitate 0–4, făcându-l fiabil pentru experimente rapide și eficiente. Prin urmare, el a fost folosit ca set de test pentru deciziile de preprocesare și augmentare: dacă o strategie funcționează doar pe un dataset mare, dar nu se păstrează pe APTOS, atunci există riscul ca modelul să fi învățat particularități ale sursei, nu trăsături generale ale retinopatiei diabetice.

Setul *Diabetic_Balanced_Data*, disponibil public pe Kaggle sub denumirea *Diabetic Retinopathy Balanced* [32], a fost folosit în experimentele inițiale de comparație între arhitecturi, funcții de pierdere și optimizatori. Ulterior, analiza acestui set a devenit importantă metodologic, deoarece s-a observat că organizarea sa poate permite apariția unui *data leakage* între subseturi.

Setul *Diabetic_Balanced_Aug* a fost folosit ca sursă mare, dezechilibrată, din care au fost construite mai multe variante: split inițial, variantă curățată, variantă augmentată și variantă augmentată cu preprocesare Ben Graham. Aceste variante au fost generate prin scripturile din directorul `Scripts/Diabetic_Balanced_Aug`.

Prin folosirea acestor surse complementare, lucrarea a urmărit două obiective practice:

- obținerea unui set principal suficient de mare pentru antrenarea modelelor CNN prin *transfer learning*;
- testarea deciziilor de preprocesare și organizare pe seturi cu distribuții și niveluri de calitate diferite.

4.3 Pipeline-ul de prelucrare pentru EyePACS

Pentru EyePACS au fost create mai multe variante experimentale prin scripturile din `Scripts/EyePACS`. Primul pas este realizat de notebook-ul `split_eyepacs.ipynb`, care citește fișierele CSV cu etichete, caută imaginile existente în directorul `Images`, păstrează doar intrările care pot fi mapate corect și creează o împărțire stratificată în subseturile `train`, `val` și `test`. Imaginile sunt apoi copiate în directoare numerotate de la 0 la 4, corespunzătoare claselor.

Al doilea pas este curățarea imaginilor, implementată în `clean_eyepacs.ipynb`. Scriptul verifică fiecare imagine printr-un set de criterii simple de calitate: citirea corectă a fișierului, claritatea estimată prin varianța Laplacianului, existența unei regiuni retiniene detectabile prin threshold și contur, precum și aria regiunii principale. Imaginile valide sunt copiate în `eyepacs_cleaned`.

Algoritmul de curățare a fost construit ca un filtru automat, aplicat înainte de augmentare și preprocesare. Logica urmărită este următoarea:

- **Redimensionarea pentru analiză:** imaginea este adusă temporar la o dimensiune standard, astfel încât pragurile de claritate și de arie să fie comparabile între fișiere.
- **Conversia în grayscale:** imaginea este convertită în tonuri de gri pentru a analiza intensitatea globală și pentru a separa mai ușor retina de fundal.
- **Estimarea clarității:** se calculează varianța Laplacianului; valori foarte mici indică imagini încețoșate, din care modelul poate învăța trăsături instabile.
- **Detectarea regiunii retiniene:** se aplică un prag de intensitate pentru separarea fundului de ochi de fundalul negru, apoi se caută contururile rezultate.
- **Validarea conturului principal:** este păstrat cel mai mare contur, presupus a fi regiunea retiniană; imaginile fără contur sau cu o arie prea mică sunt respinse.

Prin această succesiune de pași, curățarea nu modifică diagnosticul și nu introduce imagini noi, ci elimină intrările care ar putea afecta negativ antrenarea: imagini neclare, imagini aproape goale, imagini cu fund de ochi foarte mic sau fișiere imposibil de citit. Această etapă a fost aplicată înaintea augmentării tocmai pentru ca imaginile sintetizate să pornească din exemple valide, nu din imagini problematice.

Această etapă a avut rolul de a elimina cazurile în care imaginea nu putea fi folosită în mod fiabil de model: fișiere corupte, imagini aproape complet întunecate, imagini cu fund de ochi foarte mic în cadru sau imagini în care conturul retinian nu putea fi separat suficient de fundal. În analiza imaginilor medicale, calitatea intrării influențează direct capacitatea modelului de a învăța trăsături utile [22].

Pentru reducerea dezechilibrului de clase, notebook-ul `form_eyepacs_augmented.ipynb` generează o variantă augmentată. Scriptul copiază imaginile originale și aplică augmentări doar pe subsetul de antrenare: flip, transformări afine, blur gaussian, variații moderate de luminosități/contrast și zgomot. Scopul nu este modificarea diagnosticului, ci creșterea diversității vizuale pentru clasele minoritare.

În alegerea augmentărilor s-a urmărit păstrarea sensului clinic al imaginii. Astfel, transformările au fost limitate la modificări care pot simula variații obișnuite de achiziție, fără a

introduce artefacte agresive. Pentru retinopatia diabetică, această precauție este importantă deoarece leziunile relevante pot fi mici, iar o augmentare excesivă poate estompa microanevrismele, hemoragiile punctiforme sau exudatele [24], [17].

Augmentarea a avut trei roluri practice. În primul rând, a crescut numărul de exemple pentru clasele slab reprezentate, reducând tendința modelului de a favoriza clasa majoritară. În al doilea rând, a expus rețeaua la variații de poziție, scară, luminozitate și claritate asemănătoare celor întâlnite între camere sau sesiuni diferite de achiziție. În al treilea rând, a permis compararea între variante de dataset fără a schimba arhitectura modelului, astfel încât efectul preprocesării și al augmentării să poată fi analizat separat. Pentru a păstra validitatea evaluării, imaginile din validare și test nu au fost augmentate artificial.

Ultima variantă este generată de `form_eyepacs_augmented_bengraham.ipynb`, care aplică preprocesarea Ben Graham peste imaginile augmentate. Astfel se obține un set în care fundalul, contrastul și regiunea retiniană sunt mai uniformizate.

4.4 Pipeline-ul de prelucrare pentru APTOS 2019

Setul APTOS 2019 a fost prelucrat printr-un flux asemănător cu cele folosite pentru EyePACS și Diabetic_Balanced_Aug. Imaginile au fost reorganizate pe clase, apoi au fost construite variante experimentale cu augmentare și cu preprocesare Ben Graham. Scopul nu a fost doar creșterea numerică a setului, ci obținerea unui banc de testare mai controlat pentru efectul preprocesărilor.

În cazul APTOS, augmentările au fost importante deoarece setul este mai mic decât EyePACS, iar clasele severe au mai puține exemple disponibile. Au fost folosite transformări moderate: flip-uri, rotații mici, scalări, modificări de luminozitate și contrast, blur ușor și zgomot gaussian. Aceste transformări simulează variații reale ale achiziției, dar evită deformarea patologiei. Astfel, modelul este încurajat să recunoască structuri retiniene relevante, nu poziții fixe sau caracteristici accidentale ale imaginilor.

După augmentare, au fost generate și variante preprocesate prin metoda Ben Graham. Această etapă a uniformizat iluminarea, a redus influența fundalului și a făcut mai vizibile structurile retiniene. Prin folosirea aceluiași tip de preprocesare ca în celelalte dataset-uri, APTOS a devenit util pentru experimente comparative: se poate observa dacă modelul reacționează similar pe o distribuție diferită sau dacă performanța depinde de sursa imaginilor.

APTOS a fost, așadar, un dataset experimental potrivit pentru lucrare din patru motive:

- dimensiunea sa mai redusă, dar totuși în același timp echilibrată și cu imagini curate
- are aceeași formulare multiclasă 0–4 ca celelalte seturi folosite;
- provine dintr-un domeniu vizual diferit, util pentru testarea robusteții;
- permite analiza separată a efectului augmentării și al preprocesării Ben Graham pe un set mai mic și mai controlabil.

4.5 Pipeline-ul de prelucrare pentru Diabetic_Balanced_Aug

Pentru `Diabetic_Balanced_Aug`, fluxul este similar și este implementat în `Scripts/Diabetic_Balanced_Aug`. Notebook-ul `split_balanced_aug.ipynb` citește `trainLabels.csv`, potrivește etichetele cu imaginile din `resized_train` și creează un split stratificat pe clase.

Notebook-ul `clean_balanced_aug.ipynb` construiește varianta `balanced_aug_cleaned`. Acesta aplică filtre de calitate asemănătoare cu cele folosite pentru EyePACS: verificarea citirii imaginii, redimensionarea pentru analiză, estimarea clarității, segmentarea aproximativă a fundului de ochi și validarea conturului principal.

Pentru acest set, curățarea a fost utilă deoarece imaginile provin dintr-o colecție deja prelucrată și nu toate fișierele au aceeași calitate vizuală. Prin aplicarea aceluiași criterii ca în cazul EyePACS, s-a obținut un protocol comparabil între dataset-uri. Această uniformizare nu elimină complet diferențele dintre surse, dar reduce variațiile produse de organizarea locală a fișierelor.

În practică, același algoritm de curățare a avut două efecte importante asupra acestui dataset:

- a separat etapa de control al calității de etapa de augmentare, ceea ce a permis compararea clară între dataset-ul curățat și dataset-ul augmentat;
- a redus riscul ca imaginile cu fundal dominant, contur incorect sau blur puternic să fie multiplicat prin augmentări și să influențeze disproporționat antrenarea.

Notebook-ul `form_balanced_augmented.ipynb` creează varianta `balanced_augmented`. Pentru fiecare clasă, imaginile originale din train sunt păstrate, iar clasele minoritare sunt completate prin augmentări generate dinamic. Transformările includ flip, rotații și scalări mici, blur, variații de culoare și zgomot gaussian. Subseturile de validare și testare sunt copiate fără augmentări suplimentare, pentru a păstra evaluarea stabilă.

Pentru acest dataset, augmentarea a fost tratată ca o etapă de construcție a datelor, nu ca o operație aplicată întâmplător în timpul evaluării. Prin generarea variantelor augmentate offline, s-a putut verifica explicit câte imagini apar în fiecare clasă și în fiecare subset. Această abordare a făcut mai ușoară compararea dintre setul curățat, setul augmentat și setul augmentat cu Ben Graham.

În final, `form_balanced_augmented_bengraham.ipynb` aplică metoda Ben Graham și generează `balanced_augmented_bengraham`. Această variantă a fost folosită pentru a observa dacă uniformizarea iluminării și accentuarea contrastului local ajută modelele pe un set mai mare și dezechilibrat.

4.6 Verificarea distribuției imaginilor

Pe lângă generarea efectivă a dataset-urilor, a fost realizată și o etapă de verificare a distribuției imaginilor. În notebook-ul `Notebooks/teste_si_incerari.ipynb` au fost incluse funcții pentru numărarea imaginilor pe clase și subseturi, aplicate succesiv pe variantele intermediare și finale. Această verificare a fost importantă deoarece o eroare de copiere sau o distribuție foarte diferită între `train`, `val` și `test` poate afecta direct interpretarea rezultatelor, dând avantaj clasei majoritare.

Pentru fiecare dataset au fost urmărite următoarele aspecte:

- existența tuturor claselor în fiecare subset;
- numărul de imagini pe clasă după curățare și augmentare;
- păstrarea validării și testării fără augmentări generate artificial;
- compararea vizuală a imaginilor originale cu imaginile preprocesate;
- identificarea variantelor de dataset care pot fi folosite pentru experimente preliminare și a celor potrivite pentru sistemul final.

4.7 Separarea originalelor de augmentări în Diabetic_Balanced_Data

Un pas distinct a fost realizat pentru setul `Diabetic_Balanced_Data`. Scriptul `Scripts/DIabetic_Balanced_Data/split_diabetic_balanced_augmented.py` separă imaginile originale de cele augmentate pe baza markerului `_aug_` din numele fișierului. Rezultatul constă în două colecții separate: una cu imagini originale și una cu imagini augmentate. Această separare a fost necesară pentru a înțelege mai bine compoziția setului și pentru a investiga dacă imaginile augmentate pot apărea în subseturi diferite față de imaginea originală.

Din punct de vedere experimental, această etapă a fost una dintre cele mai importante, deoarece a mutat accentul de la simpla antrenare a modelelor la validitatea datelor folosite. Într-un dataset care include deja imagini augmentate, fișierele pot părea exemple independente, deși ele provin din aceeași imagine inițială. Dacă aceste versiuni sunt distribuite aleator în subseturi diferite, modelul poate recunoaște structuri aproape identice în validare sau testare.

4.8 Controlul data leakage prin split grupat

În ultimele experimente realizate pe `Diabetic_Balanced_Data` s-a observat o problemă metodologică importantă: setul public de pe Kaggle poate produce rezultate artificiale dacă imaginile originale și augmentările lor sunt împărțite între subseturi diferite. Această situație reprezintă *data leakage*, deoarece modelul poate vedea în antrenare o variantă foarte apropiată de o imagine care apare ulterior în validare sau testare.

Pentru verificarea acestei ipoteze a fost creat scriptul `Scripts/DIabetic_Balanced_Data/grouped_split_diabetic_balanced_data.py`. Scriptul grupează fiecare imagine originală împreună cu augmentările sale, apoi împarte grupurile, nu fișierele individuale, în `train`, `val` și `test`, folosind raportul 70%/20%/10%. După copiere, scriptul validează că niciun grup nu apare în mai multe subseturi și salvează un raport în `grouped_split_report.json`.

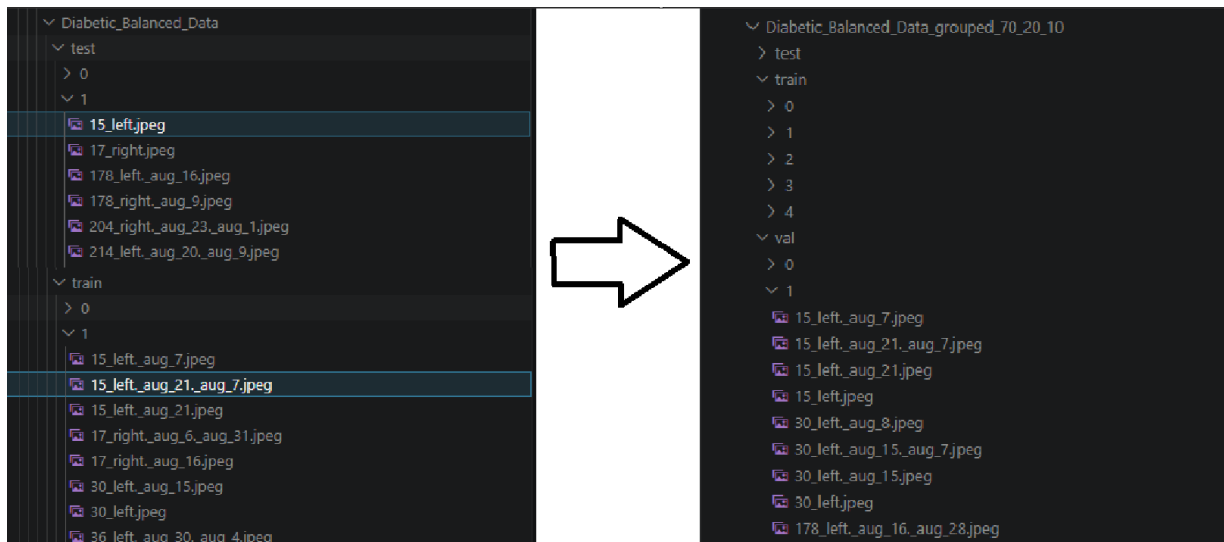


Figura 4.1: Exemplu de data leakage în `Diabetic_Balanced_Data`: imaginea originală și augmentările sale apar în subseturi diferite, permițând modelului să recunoască structuri aproape identice în validare sau testare.

Logica scriptului poate fi rezumată astfel:

- se extrage numele imaginii de bază prin eliminarea markerilor de augmentare;
- toate fișierele care provin din aceeași imagine sunt plasate în același grup;
- împărțirea se face la nivel de grup, nu la nivel de fișier individual;
- după copiere se verifică intersecția grupurilor între subseturi;
- se salvează un raport care documentează numărul de imagini și grupuri rezultate.

Această descoperire este o contribuție practică a lucrării. Ea arată că performanțele obținute pe seturi publice trebuie interpretate împreună cu modul exact de împărțire a datelor. În capitolul de rezultate se arată că, după eliminarea acestui risc de *data leakage*, performanța scade semnificativ, ceea ce confirmă că evaluarea inițială era prea optimistă.

4.9 Preprocesarea Ben Graham

Preprocesarea Ben Graham este folosită pentru reducerea variațiilor de iluminare și pentru accentuarea structurilor retiniene. În scripturile de construire a dataset-urilor, metoda aplică redimensionare, conversie în tonuri de gri, detectarea regiunii retiniene prin threshold, extragerea conturului principal, eroziunea măștii, blur gaussian și combinația ponderată dintre imaginea originală și imaginea blurată.

Pentru a documenta vizual fiecare etapă, în `Notebooks/teste_si_incerari.ipynb` a fost creată funcția `procesare_ben_graham_detaliata`. Aceasta afișează secvențial transformările aplicate unei imagini de fund de ochi: imaginea originală, conversia în tonuri de gri, masca retiniană, blur-ul gaussian folosit pentru estimarea iluminării de fond, filtrul Ben Graham, masca exterioară, harta distanțelor, haloul exterior și imaginea finală. Figura 4.2 sintetizează această analiză vizuală.

Etapile principale ale preprocesării sunt:

- izolarea regiunii retiniene prin prag de intensitate și contur principal;
- erodarea măștii pentru reducerea marginilor instabile;
- estimarea iluminării de fond prin blur gaussian cu deviație proporțională cu dimensiunea imaginii;
- aplicarea formulei de tip `addWeighted`, care accentuează diferența dintre imagine și fundalul blurat;
- uniformizarea exteriorului retinei printr-un halou gradual, pentru a evita margini dure;
- salvarea imaginii procesate pentru antrenare offline.

În figura următoare se poate observa cum fiecare etapă a preprocesării Ben Graham contribuie la transformarea imaginii inițiale într-o versiune în care fundalul este uniformizat, iar structurile retiniene sunt mai vizibile. Această vizualizare detaliată ajută la înțelegerea modului în care preprocesarea influențează datele de intrare și, implicit, performanța modelelor antrenate pe aceste date.

Procesarea Secvențială: Extragere Contur, Transformare Ben Graham și Halou

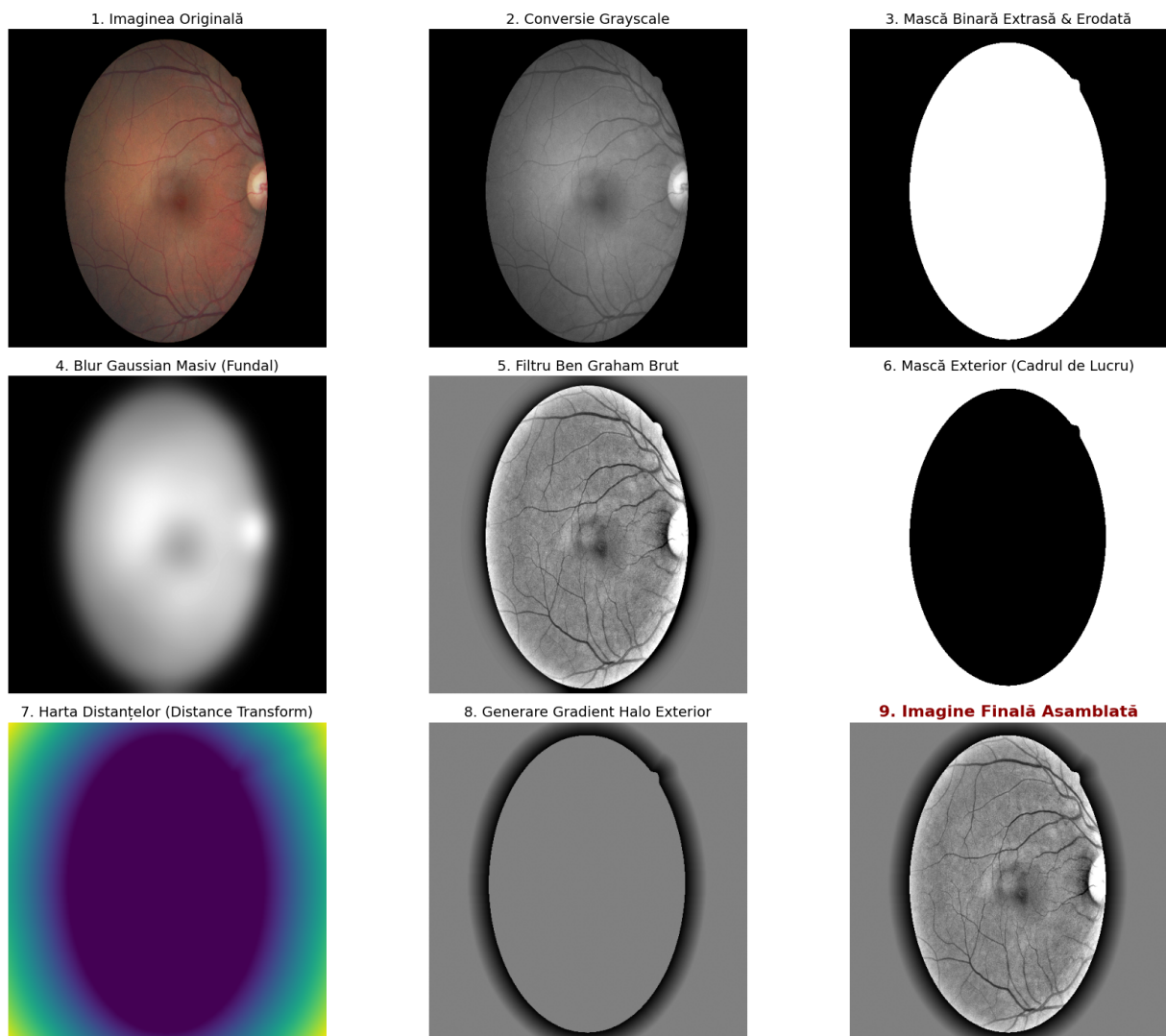


Figura 4.2: Vizualizarea etapelor de preprocesare Ben Graham: imagine inițială, mască retiniană, estimarea iluminării de fond, filtrare, generarea haloului și imagine finală.

În modelul principal local CUDA, preprocesarea este realizată prin metoda Ben Graham. Imaginile sunt redimensionate, regiunea retiniană este mascată, iar fundalul este uniformizat pentru a reduce variațiile de iluminare înainte de utilizarea lor în antrenare.

Preprocesarea Ben Graham a fost aleasă deoarece reduce variațiile globale de iluminare și face mai vizibile structurile retiniene relevante, fără a introduce o etapă suplimentară de ajustare locală a contrastului. Totuși, această preprocesare nu a fost presupusă automat ca fiind optimă pentru toate seturile. Ea a fost testată experimental pe variantele construite, iar rezultatele sunt discutate în capitolele următoare. Această abordare este importantă deoarece literatura arată că modelele pot reacționa diferit la schimbări de preprocesare și de domeniu vizual [8], [7].

4.10 Formarea dataset-urilor EyePACS pentru sistemul principal

Pe lângă variantele EyePACS folosite în experimentele comparative, pentru sistemul principal în două etape au fost construite două dataset-uri specializate: `eyepacs_binar` și `eyepacs_1_4`. Acestea au fost generate separat deoarece pipeline-ul final nu rezolvă direct o singură clasificare multclasă 0–4, ci împarte decizia în două probleme mai controlabile: detecția bolii și gradarea severității pentru cazurile patologice.

Dataset-ul `eyepacs_binar` este construit în notebook-ul `Scripts/EyePACS/create_eyepacs_binar.ipynb`. Scriptul pornește de la fișierul `all_labels.csv` și de la imaginile originale EyePACS, verifică existența fiecărui fișier și transformă etichetele originale 0–4 într-o problemă binară: clasa 0 devine *sănătos*, iar clasele 1–4 sunt grupate în clasa *bolnav*. Imaginile sunt preprocesate prin metoda Ben Graham în momentul copierii în dataset-ul final. Împărțirea urmărește raportul 70%/15%/15%, iar setul de test este construit explicit 50/50 între imagini sănătoase și imagini patologice, pentru evaluarea clară a detectorului binar.

După split, subsetul de antrenare al dataset-ului binar este echilibrat prin augmentare offline. Augmentările sunt aplicate doar în `train`, astfel încât validarea și testarea să rămână nealterate. Varianta finală conține 96.664 imagini în `train`, câte 48.332 pentru fiecare clasă, 13.304 imagini în validare și 13.304 imagini în test. Setul de test conține 6.652 imagini *sănătos* și 6.652 imagini *bolnav*.

Dataset-ul `eyepacs_1_4` este construit în notebook-ul `Scripts/EyePACS/create_eyepacs_1-4.ipynb`. În această variantă sunt păstrate doar imaginile cu etichete patologice, adică nivelurile 1, 2, 3 și 4. Scopul său este antrenarea celei de-a doua etape a sistemului principal, care primește doar cazurile considerate bolnave de detectorul binar și estimează gradul de severitate. Split-ul inițial este stratificat pe clasele patologice, cu raport 70% pentru antrenare, 15% pentru validare și 15% pentru test.

Pentru a reduce dezechilibrul puternic dintre stadiile patologice, subsetul de antrenare din `eyepacs_1_4` este echilibrat prin augmentare offline până la 10.000 imagini pentru fiecare clasă. Rezultatul final are 40.000 imagini în `train`, distribuite egal între stadiile 1–4, 3.503 imagini în validare și 3.504 imagini în test. Validarea și testarea păstrează distribuția naturală a cazurilor patologice, tocmai pentru ca evaluarea modelului de severitate să rămână realistă.

Prin separarea acestor două dataset-uri, sistemul principal poate optimiza diferit cele două obiective. Primul model este antrenat pentru sensibilitate ridicată în separarea *sănătos*–*bolnav*, iar al doilea model este antrenat doar pe diferențele mai fine dintre stadiile patologice. Această organizare reduce conflictul dintre clasa majoritară 0 și clasele de boală, problemă frecventă în clasificarea directă pe scala completă 0–4.

4.11 Dataset-urile finale rezultate

În urma acestor pași au rezultat mai multe variante experimentale și două dataset-uri dedicate sistemului principal. Tabelul 4.1 sintetizează rolul fiecăreia.

Tabela 4.1: Dataset-uri construite și rolul lor în lucrare

Dataset	Script / sursă	Rol
eyepacs_split	split_eyepacs.ipynb	Split stratificat inițial pentru EyePACS.
eyepacs_cleaned	clean_eyepacs.ipynb	Variantă filtrată calitativ.
eyepacs_augmented	form_eyepacs_augmented.ipynb	Variantă cu augmentare pentru reducerea dezechilibrului.
eyepacs_augmented_bengraham	form_eyepacs_augmented_bengraham.ipynb	EyePACS augmentat și preprocesat Ben Graham.
eyepacs_binar	create_eyepacs_binar.ipynb	Dataset pentru prima etapă a sistemului principal: clasificare <i>sănătos-bolnav</i> , cu train echilibrat și test 50/50.
eyepacs_1_4	create_eyepacs_1-4.ipynb	Dataset pentru a doua etapă a sistemului principal: gradarea cazurilor patologice în clasele 1–4, cu train echilibrat la 10.000 imagini/clasă.
aptos_augmented	form_aptos_augmented.ipynb	Variantă APTOS construită pentru experimente cu augmentare controlată.
aptos_augmented_bengraham	form_aptos_augmented+b_g.ipynb	APTOS augmentat și preprocesat Ben Graham, folosit pentru comparații între domenii vizuale.
balanced_aug_cleaned	clean_balanced_aug.ipynb	Variantă curățată din dataset-ul Diabetic_Balanced_Aug.
balanced_augmented	form_balanced_augmented.ipynb	Variantă echilibrată prin augmentare.
balanced_augmented_bengraham	form_balanced_augmented_bengraham.ipynb	Variantă augmentată și preprocesată Ben Graham.
Diabetic_Balanced_Data_grouped_70_20_10	grouped_split_diabetic_balanced_data.py	Split grupat fără leakage între originale și augmentări.

Capitolul 5

Metodologia propusă

5.1 Fluxul general al sistemului

Metodologia propusă urmărește construirea unui flux complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Sistemul pornește de la seturi de date reorganizate în directoare de antrenare, validare și testare. Pentru experimentele comparative sunt folosite EyePACS, APTOS și *Diabetic_Balanced_Aug*, iar pentru sistemul principal sunt construite două seturi EyePACS specializate: **EyePACS_binar**, pentru separarea imaginilor bolnave de cele sănătoase, și **EyePACS_1_4**, pentru gradarea severității bolii. Fluxul ajunge la salvarea unor modele antrenate care pot fi folosite ulterior într-o aplicație de inferență. Implementarea este realizată în Python, folosind PyTorch pentru antrenarea rețelelor, Torchvision și **timm** pentru arhitecturi pre-antrenate, iar scikit-learn, Matplotlib și Seaborn pentru evaluare și vizualizare.

Fluxul de lucru este împărțit în mai multe componente principale. Fișierul **my_dataset.py**, regăsit în anexa A se ocupă de încărcarea imaginilor și de aplicarea transformărilor. Fișierul **builder.py** definește modelele, funcțiile de pierdere, modul de calcul al predicțiilor și optimizatorii. Fișierul **utils.py** grupează funcțiile auxiliare necesare pentru rularea experimentelor: crearea directoarelor de rezultate, generarea numelor de fișiere, calculul metricilor suplimentare, rularea unei epoci de antrenare sau validare, salvarea graficelor, construirea matricelor de confuzie, a curbelor ROC și a hărților Grad-CAM. Scriptul **train.py** integrează aceste componente, rulează antrenarea, validează modelul la fiecare epocă, salvează cea mai bună variantă și generează graficele de evaluare.

Separarea funcțiilor utilitare într-un fișier dedicat este importantă deoarece menține **train.py** concentrat pe fluxul principal al experimentului. În același timp, **utils.py** permite reutilizarea aceluiași funcții de evaluare și vizualizare pentru mai multe arhitecturi, surse de date și configurații de antrenare. În experimentele documentate, QWK este folosită ca metrică principală de salvare a celui mai bun model, iar rezultatele generate pentru experimente diferite au aceeași structură și pot fi comparate mai ușor.

Problema este formulată ca o clasificare multiclasă în cinci clase, corespunzătoare stadiilor retinopatiei diabetice. Pentru fiecare imagine, modelul produce un vector de cinci scoruri, iar clasa finală este obținută prin alegerea valorii maxime în cazul funcțiilor de pierdere de tip Cross-Entropy și Focal Loss. Pentru varianta ordinală, ieșirea continuă a rețelei este interpretată prin praguri de decizie optimizate, deoarece clasele au o ordine clinică naturală.

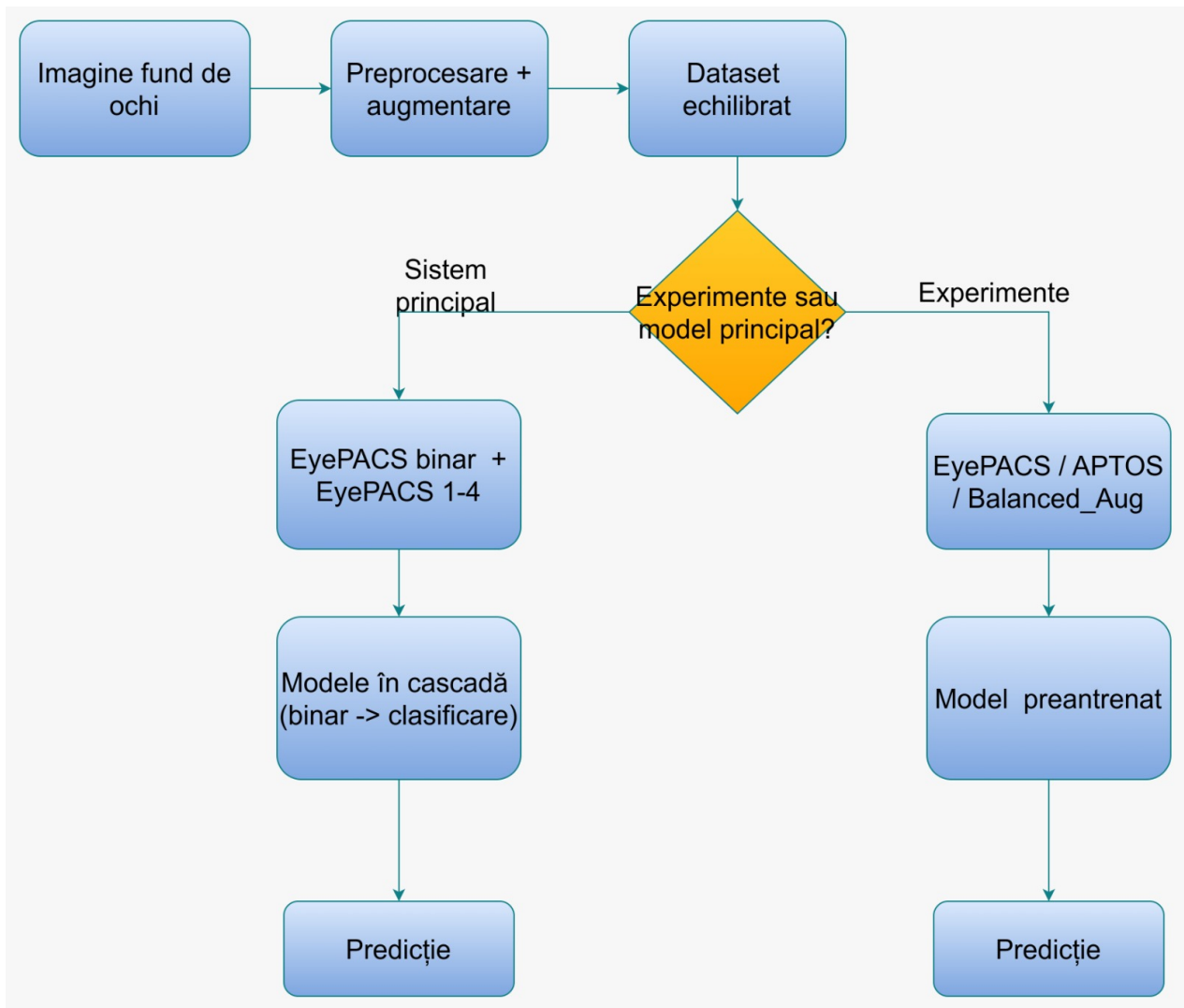


Figura 5.1: Fluxul general al lucrării, de la imaginea de fund de ochi și preprocesare până la ramura de experimente și ramura sistemului principal.

Figura 5.1 sintetizează organizarea întregului sistem. Pornind de la imaginea fundus, datele sunt preprocesate și echilibrate, apoi sunt folosite fie pentru experimente comparative, fie pentru construirea pipeline-ului principal. Ramura experimentală include evaluarea pe EyePACS, APTOS și Diabetic_Balanced_Aug, cu modele preantrenate folosite pentru compararea arhitecturilor și a strategiilor de antrenare. Ramura principală folosește EyePACS_binar pentru decizia bolnav/sănătos și EyePACS_1_4 pentru gradele de boală, astfel încât decizia finală să poată fi obținută prin modele în cascadă.

Pe lângă această formulare multiclasă, soluția principală a lucrării este organizată ca un pipeline în două etape. Prima etapă tratează problema ca detecție binară și decide dacă imaginea aparține clasei 0 sau uneia dintre clasele patologice 1–4. A doua etapă este aplicată doar imaginilor considerate patologice și clasifică severitatea în clasele 1, 2, 3 și 4. Astfel, decizia finală rămâne exprimată pe scala completă 0–4, dar modelul nu mai trebuie să rezolve simultan două probleme diferite: separarea imaginilor sănătoase de cele afectate și diferențierea gradelor de severitate între cazurile bolnave.

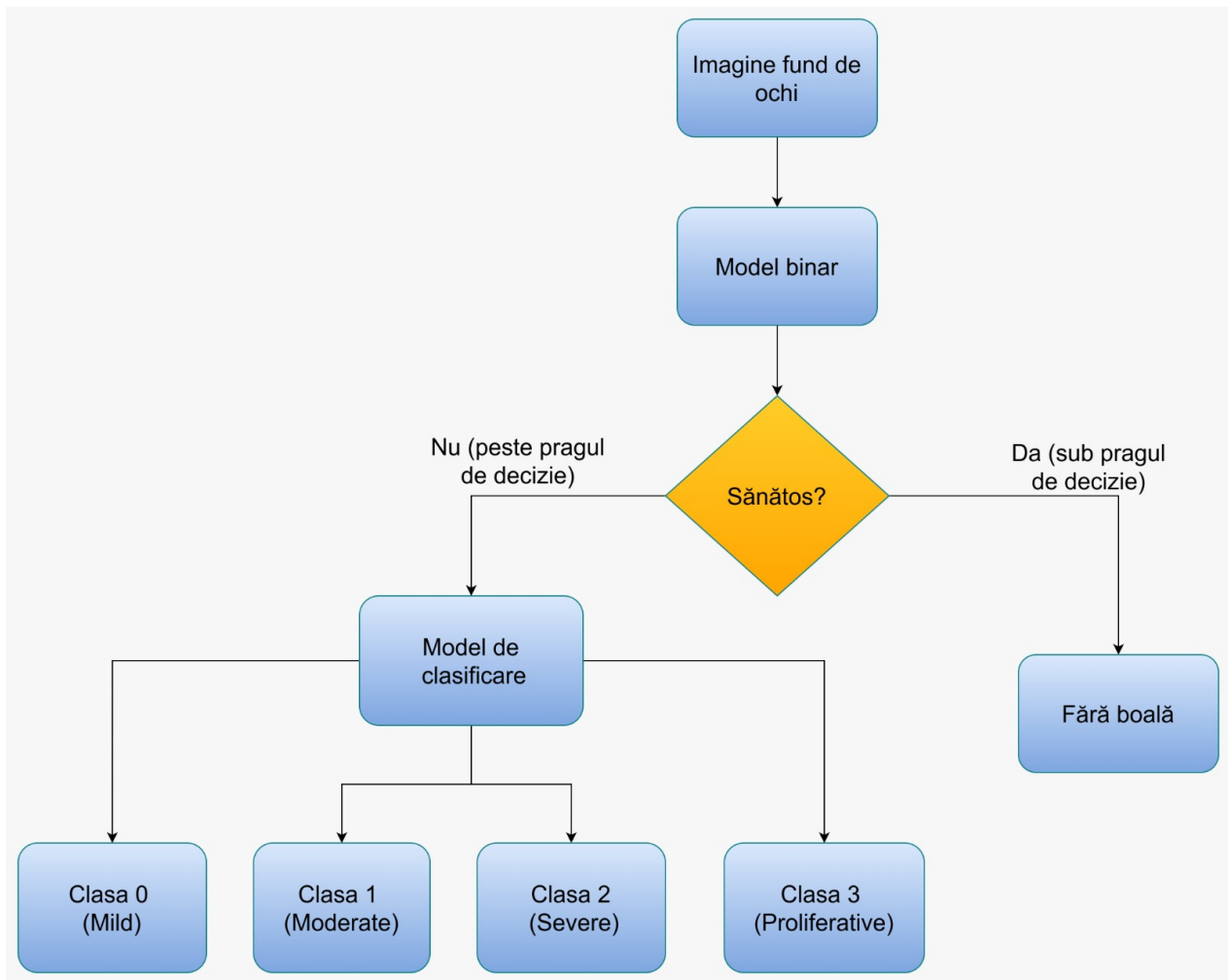


Figura 5.2: Fluxul sistemului principal în două etape: detector binar pentru separarea imaginilor sănătoase de cele patologice, urmat de clasificarea severității pentru cazurile trimise mai departe.

Schema din figura 5.2 detaliază ramura principală a lucrării. Imaginea este evaluată inițial de modelul binar antrenat pe **EyePACS_binar**. Dacă probabilitatea de boală este sub pragul ales pe validare, sistemul returnează direct rezultatul *fără boală*. Dacă imaginea este considerată patologică, aceasta este transmisă către modelul de severitate antrenat pe **EyePACS_1_4**, unde scorul continuu este transformat în clasele 1–4 prin praguri optimizate pe validare. Această separare permite tratarea distinctă a obiectivului de screening și a obiectivului de gradare a bolii.

5.2 Modelele analizate

În experimente au fost analizate arhitecturi convoluționale moderne, alese deoarece permit transfer learning și au fost utilizate frecvent în clasificarea imaginilor medicale [18], [2]. Implementarea permite construirea modelelor **resnet50**, **efficientnet_b3**, **inception_v3** și **incres_v2**. Toate cele patru arhitecturi au fost incluse în implementare, însă în experimentele comparative accentul a fost pus pe ResNet50 și Inception-ResNet-v2, deoarece oferă două repere complementare: o arhitectură reziduală robustă și o arhitectură hibridă cu procesare multi-scală. Pentru sistemul principal în două etape, accentul a fost mutat pe EfficientNet-B3, folosit ca backbone pentru detectorul binar și pentru modelul de clasificare a severității.

Toate modelele sunt adaptate pentru sarcina experimentală curentă. În experimentele inițiale, straturile finale, proiectate inițial pentru clasificarea ImageNet, sunt înlocuite sau configurate astfel încât ieșirea să corespundă celor cinci stadii ale retinopatiei diabetice. În soluția principală în două etape, același principiu este folosit pentru a construi separat un model cu ieșire binară pentru detecție și un model de regresie care produce un scor continuu pentru clasele patologice 1–4. Folosirea ponderilor pre-antrenate este importantă deoarece permite reutilizarea unor trăsături vizuale generale, precum muchii, texturi, forme și tranziții de culoare, care sunt apoi ajustate pentru domeniul medical prin fine-tuning. Această strategie este utilă în special pentru seturile de date medicale, unde numărul de imagini etichetate este mai mic decât în colecțiile generale precum ImageNet, iar variațiile de iluminare, contrast și calitate a achiziției pot influența semnificativ predicția.

5.3 Arhitectura ResNet50

ResNet50 este o arhitectură convoluțională bazată pe conexiuni reziduale [27]. Ideea principală este ca un bloc al rețelei să nu învețe direct o transformare completă, ci o corecție față de intrarea primită. Această abordare facilitează antrenarea rețelelor adânci și reduce riscul degradării performanței odată cu creșterea numărului de straturi. Din acest motiv, ResNet50 este folosită frecvent ca model de referință în probleme de clasificare medicală: are o capacitate suficient de mare pentru a învăța trăsături complexe, dar rămâne mai ușor de antrenat și interpretat decât arhitecturile foarte adânci sau hibride.

În contextul imaginilor de fund de ochi, ResNet50 poate extrage trăsături ierarhice: primele straturi detectează contururi, variații de culoare și texturi simple, iar straturile profunde combină aceste informații în structuri mai complexe, precum vase retiniene, zone de exudate, hemoragii sau regiuni suspecte. În implementarea din `builder.py`, stratul final `fc` este înlocuit cu o secvență formată din `Dropout(0.5)` și un strat liniar cu cinci ieșiri. Dropout-ul are rol de regularizare și reduce tendința de supraînvățare. ResNet50 este astfel folosită atât ca bază de comparație, cât și ca arhitectură robustă pentru experimentele în care se evaluează influența funcțiilor de pierdere, a optimizatorilor și a strategiilor de monitorizare.

5.4 Arhitectura EfficientNet-B3

EfficientNet-B3 aparține familiei EfficientNet, propusă de Tan și Le [29], în care scalarea rețelei nu se face independent doar pe adâncime, lățime sau rezoluție, ci prin *compound scaling*. Această strategie crește simultan cele trei dimensiuni ale modelului folosind un coeficient comun, astfel încât arhitectura să păstreze un echilibru între capacitatea de reprezentare și costul computațional. Varianta B3 este relevantă pentru această lucrare deoarece oferă un compromis între modelele mai mici, care pot pierde detalii fine, și modelele foarte mari, care necesită mai multă memorie și prezintă risc mai ridicat de supraînvățare.

Din punct de vedere structural, EfficientNet utilizează blocuri de tip MBConv, inspirate din convoluțiile separabile pe adâncime și din conexiunile reziduale inversate. Aceste blocuri reduc numărul de parametri în comparație cu o convoluție standard, păstrând totuși capacitatea de a extrage trăsături vizuale relevante. Pentru imaginile de fund de ochi, această eficiență este importantă deoarece leziunile pot fi mici, slab contrastate și distribuite neuniform în imagine. O arhitectură care procesează intrări la rezoluții mai mari poate păstra mai bine informația locală, iar mecanismul de scalare al EfficientNet-B3 susține această nevoie fără a crește excesiv complexitatea modelului.

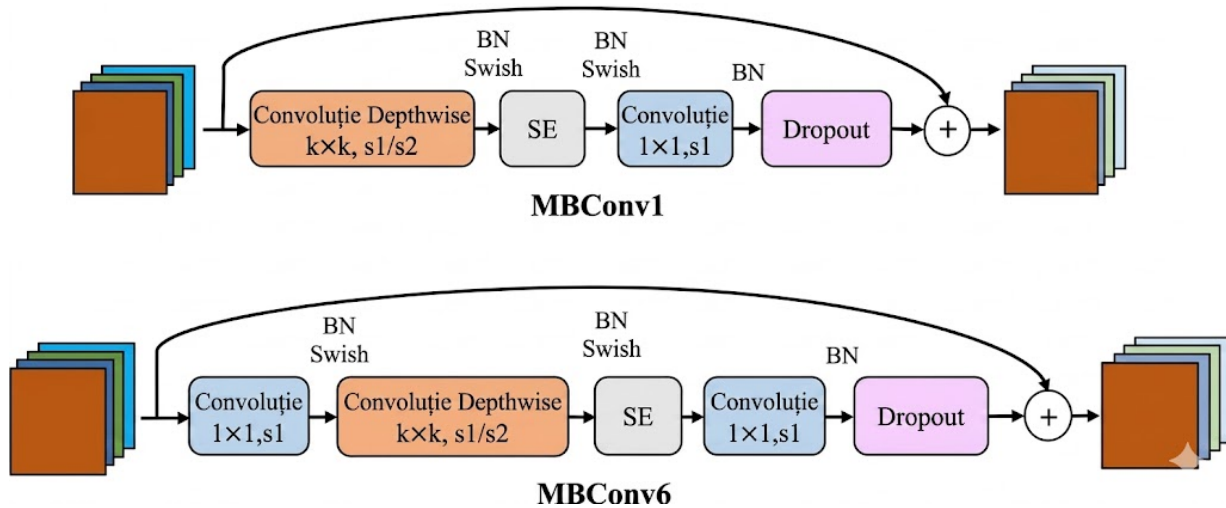


Figura 5.3: Structura blocurilor MBConv în EfficientNet-B3

În implementarea din `builder.py`, modelul este încărcat prin varianta EfficientNet-B3 din Torchvision, cu ponderi pre-antrenate atunci când parametrul `pretrained` este activ. Clasificatorul final este modificat prin înlocuirea stratului `classifier[1]` cu o secvență formată din Dropout (0.5) și un strat liniar cu cinci ieșiri. În cadrul experimentelor, EfficientNet-B3 oferă o alternativă compactă la ResNet50 și Inception-ResNet-V2, fiind utilă pentru a verifica dacă o arhitectură optimizată pentru raportul acuratețe-eficiență poate generaliza bine pe imagini retinene preprocesate și augmentate.

5.5 Arhitectura InceptionV3

InceptionV3 este o arhitectură construită în jurul modulelor Inception, care procesează aceeași intrare prin filtre convoluționale de dimensiuni diferite [28]. Această strategie permite extragerea simultană a trăsăturilor locale și a celor cu context spațial mai larg. Pentru retinopatia diabetică, acest lucru este relevant deoarece semnele patologice pot avea dimensiuni foarte diferite: microanevrismele sunt mici și discrete, în timp ce hemoragiile sau exudatele pot ocupa regiuni mai mari. Arhitectura folosește factorizarea convoluțiilor și module auxiliare pentru a îmbunătăți eficiența și stabilitatea antrenării, fiind una dintre arhitecturile importante folosite în sisteme de detecție automată a retinopatiei diabetice [1].

În cod, InceptionV3 este încapsulat prin clasa `InceptionWrapper`. Această clasă rezolvă particularitatea modelului Torchvision, care în timpul antrenării poate returna și o ieșire auxiliară. Pentru a păstra aceeași interfață cu funcțiile de pierdere, wrapper-ul returnează doar predicția principală. Ultimul strat complet conectat este înlocuit cu un strat liniar cu cinci ieșiri, corespunzătoare claselor 0–4. Prin includerea acestei arhitecturi, metodologia poate evalua dacă procesarea multi-scală aduce beneficii față de o rețea reziduală standard, mai ales în situațiile în care semnele clinice apar la scări foarte diferite în aceeași imagine.

5.6 Arhitectura Inception-ResNet-V2

Inception-ResNet-V2 este o arhitectură hibridă care combină modulele Inception cu conexiuni reziduale [33]. Din familia Inception preia ideea procesării multi-scală, prin ramuri convoluționale paralele, iar din familia ResNet preia conexiunile care facilitează propagarea gradientului. Rezultatul este o rețea profundă, capabilă să învețe reprezentări vizuale bogate

fără a pierde stabilitatea antrenării. În raport cu InceptionV3, arhitectura are o capacitate mai mare și poate beneficia de stabilitatea conexiunilor reziduale atunci când se realizează fine-tuning pe un set de date medical.

Această arhitectură este potrivită pentru imaginile retiniene deoarece decizia de severitate poate depinde atât de leziuni mici, cât și de distribuția globală a modificărilor patologice. Ramurile Inception pot surprinde structuri la scări diferite, iar conexiunile reziduale ajută la păstrarea informației utile de-a lungul rețelei. În implementare, modelul este construit cu biblioteca `timm`, folosind modelul `inception_resnet_v2` și configurând direct numărul de clase la cinci. În comparație cu ResNet50, modelul are o capacitate mai mare, dar în același timp necesită mai multe resurse de calcul. Din acest motiv, experimentele cu Inception-ResNet-V2 sunt utile pentru a observa dacă avantajul reprezentărilor mai bogate justifică un cost computațional mai ridicat.

5.7 Funcții de pierdere utilizate

Funcția de pierdere definește modul în care modelul este penalizat pentru predicții greșite. În această lucrare au fost folosite mai multe variante, pentru a compara atât comportamentul standard în clasificarea multiclasă, cât și strategii mai potrivite pentru clase dezechilibrate sau ordonate clinic.

- **Cross-Entropy** este funcția de pierdere de bază pentru clasificare multiclasă. Aceasta compară distribuția de probabilitate produsă de model cu eticheta reală și oferă un punct de referință stabil pentru celelalte experimente. În implementare, această funcție poate folosi și `label_smoothing`, care distribuie o mică parte din probabilitate către celelalte clase și reduce încrederea excesivă a modelului în predicții.
- **Weighted Cross-Entropy** extinde Cross-Entropy prin introducerea unor ponderi pentru clase. Această variantă este utilă atunci când anumite clase trebuie penalizate mai puternic, de exemplu pentru a reduce tendința modelului de a favoriza clasele mai ușor de învățat.
- **Focal Loss** este implementată prin clasa `FocalLossMultiClass` și pornește de la Cross-Entropy, dar reduce contribuția exemplelor ușoare și acordă mai multă importanță exemplelor dificile [26]. Parametrul `gamma` controlează cât de puternic este accentuată această diferență. În experimentele inițiale a fost analizat intervalul 1.5–2.0, iar valoarea 1.6 a fost selectată pentru experimentele extinse deoarece a oferit un compromis bun între acuratețe, AUC și distribuția erorilor. La fel ca în cazul Cross-Entropy, implementarea permite folosirea parametrului `label_smoothing`.
- **Varianta ordinală bazată pe BCEWithLogitsLoss** exploatează faptul că stadiile retinopatiei diabetice au o ordine naturală. În loc ca eticheta să fie tratată doar ca o clasă independentă, aceasta este transformată într-un vector care descrie nivelurile de severitate depășite.
- **BCEWithLogitsLoss pentru detecția binară** este folosită în prima etapă a soluției principale, unde modelul trebuie să distingă între imagini sănătoase și imagini cu retinopatie diabetică. Această formulare este potrivită deoarece ieșirea modelului este un singur logit, interpretat ulterior ca probabilitate de boală.
- **SmoothL1Loss pentru clasificarea 1–4 prin regresie** este folosită în a doua etapă a sistemului principal. În această formulare, modelul produce un scor continuu pentru

severitatea bolii, iar scorul este transformat ulterior în clasele 1–4 prin praguri optimizate pe validare. SmoothL1Loss este potrivită aici deoarece combină comportamentul stabil al erorii absolute pentru abateri mari cu penalizarea pătratică pentru abateri mici, reducând sensibilitatea la exemple izolate dificil de încadrat.

5.8 Optimizatorul și hiperparametrii de antrenare

Hiperparametrii de antrenare au fost organizați diferit pentru cele două direcții de lucru: experimentele comparative și sistemul principal în două etape. Experimentele au fost folosite pentru a compara arhitecturi, seturi de date și funcții de pierdere, în timp ce soluția principală a fost configurată mai strict, cu două modele **EfficientNet-B3** antrenate pe dataset-urile dedicate `eyepacs_binar` și `eyepacs_1_4`.

Experimente comparative

Pentru experimente, rulările au fost lansate din notebook-ul `experiments.ipynb`, prin apeluri către scriptul de antrenare. Acest cadru a permis modificarea rapidă a arhitecturii, funcției de pierdere, setului de date și metricii monitorizate. Au fost testate toate cele patru arhitecturi descrise anterior, respectiv ResNet50, EfficientNet-B3, InceptionV3 și Inception-ResNet-v2, însă accentul experimental a fost pus pe ResNet50 și Inception-ResNet-v2, deoarece acestea au fost rulate mai extensiv pe combinațiile finale de date.

În aceste rulări au fost comparate **CrossEntropyLoss**, **WeightedCrossEntropy** și **FocalLoss**. Pentru configurațiile finale cu **FocalLoss** s-a folosit și `label_smoothing` 0.05, cu rolul de a reduce supraîncrederea modelului și de a evita predicțiile excesiv de rigide pe clasele dominante. În etapa de explorare au fost testate și valori diferite pentru parametrul γ al Focal Loss, în special în jurul intervalului 1.5–2.0, pentru a observa cât de mult trebuie accentuate exemplele mai greu de clasificat.

Majoritatea configurațiilor comparative recente folosesc **AdamW**, batch size 32 și 8 workeri pentru încărcarea datelor. Pentru rulările mai vechi sau pentru încercările inițiale au existat și variante cu Adam sau cu setări implicite ale scriptului, însă configurațiile păstrate pentru comparațiile finale folosesc în principal **AdamW**. Acesta este potrivit pentru fine-tuning-ul modelelor pre-antrenate deoarece separă regularizarea prin `weight_decay` de actualizarea adaptivă a gradientului, ceea ce ajută la controlul supraînvățării fără a destabiliza complet ponderile deja învățate.

Pentru rulările comparative finale cu ResNet50 și Inception-ResNet-v2 a fost folosit un scheduler de tip **cosine**. Acesta reduce rata de învățare gradual pe parcursul antrenării, fără scăderi bruște, ceea ce este util în fine-tuning deoarece permite modelului să facă actualizări mai mari la început și ajustări mai fine spre final. În acest mod, primele epoci pot adapta rapid capul de clasificare și straturile superioare, iar ultimele epoci stabilizează reprezentările fără a modifica agresiv ponderile pre-antrenate.

Backbone-ul a fost înghețat pentru primele 3 epoci prin `freeze_backbone_epochs`. Această alegere a fost folosită pentru a antrena inițial doar capul nou de clasificare, care pornește de la ponderi inițializate aleator. Dacă întregul model ar fi fost actualizat încă de la început, gradientul produs de un cap încă instabil ar fi putut perturba caracteristicile deja învățate pe ImageNet. După această etapă, backbone-ul este dezghețat și antrenat cu o rată de învățare mai mică decât capul, prin `backbone_lr_mult` 0.1. Astfel, straturile convoluționale pre-antrenate

sunt ajustate prudent la imaginile de fund de ochi, în timp ce capul de clasificare rămâne suficient de flexibil pentru a învăța separarea dintre clase.

În plus, s-a folosit `grad_clip` 1.0 pentru limitarea normei gradientului. Această setare previne actualizările foarte mari ale parametrilor, care pot apărea mai ales când se folosesc dataset-uri diferite, clase dezechilibrate sau funcții de pierdere precum `FocalLoss`. Pentru stabilizarea suplimentară a antrenării a fost activată și media mobilă exponențială a ponderilor, `ema_decay` 0.999. EMA păstrează o versiune netezită a parametrilor modelului, reducând influența oscilațiilor de la o epocă la alta și ajutând la obținerea unui checkpoint mai stabil pe setul de validare.

În funcție de arhitectură și dataset, rulările au fost configurate cu aproximativ 30–50 de epoci. Pentru comparațiile finale s-a folosit `early_stopping_patience` 15 și `early_stopping_min_delta` 0.001. În rulările inițiale au existat și configurații monitorizate după acuratețe, însă pentru experimentele relevante din documentație modelul a fost salvat după QWK pe validare, deoarece această metrică respectă caracterul ordinal al claselor de retinopatie diabetică.

Alegerea metricii de monitorizare este importantă deoarece aceasta decide ce checkpoint este păstrat, nu doar cum este raportată performanța la final. Acuratețea poate fi utilă ca indicator general, dar în clasificarea retinopatiei diabetice nu surprinde severitatea erorilor: o confuzie între clasele 1 și 2 este tratată la fel ca o confuzie între clasele 0 și 4. Din acest motiv, pentru experimentele comparative finale s-a folosit QWK pe setul de validare. QWK măsoară acordul dintre predicții și etichetele reale, corectând acordul care ar putea apărea întâmplător și penalizând mai puternic erorile între clase îndepărtate. Pentru o problemă ordinală precum clasificarea gradelor 0–4, această proprietate este mai apropiată de interpretarea clinică a rezultatului decât o simplă rată de predicții corecte.

Sistemul principal în două etape

Soluția principală nu folosește aceeași configurație generică precum experimentele, ci un notebook local dedicat, în care hiperparametrii sunt fixați explicit pentru fiecare dintre cele două modele. Ambele etape folosesc `EfficientNet-B3`, imagini redimensionate la 300×300 pixeli, batch size 32, 8 workeri și `Dropout` 0.40 în capul modelului. Alegerea acestei arhitecturi pentru sistemul principal a fost motivată de raportul bun dintre performanță și costul de antrenare, dar și de rezoluția nativă de 300 de pixeli, potrivită pentru detaliile din imaginile de fund de ochi.

Prima etapă este detectorul binar sănătos/bolnav. Acesta este antrenat pe `eyepacs.bin` și separă imaginile sănătoase de cele patologice. Funcția de pierdere este `BCEWithLogitsLoss`, potrivită pentru o ieșire binară exprimată prin `logit`. Deoarece dataset-ul binar a fost construit echilibrat, configurația finală nu activează `pos_weight`. Modelul este antrenat timp de cel mult 50 de epoci, cu `early_stopping_patience` 30.

Antrenarea detectorului binar se face în două faze. În primele 10 epoci backbone-ul este înghețat, iar optimizerul `AdamW` actualizează doar capul nou al modelului, cu rată de învățare 10^{-3} și `weight_decay` 10^{-4} . După această etapă, backbone-ul este dezghețat și se aplică fine-tuning diferențiat: 10^{-4} pentru backbone și $3 \cdot 10^{-4}$ pentru capul de clasificare. Scheduler-ul este `ReduceLROnPlateau`, cu `factor` 0.5 și `patience` 2, aplicat pe AUC-ul de validare. Cel mai bun checkpoint este salvat când AUC-ul de validare se îmbunătățește cu cel puțin 10^{-4} .

Pentru detectorul binar, AUC a fost folosit ca metrică principală de monitorizare deoarece evaluează capacitatea modelului de a separa imaginile sănătoase de cele patologice pe întreg intervalul de praguri posibile. În această etapă, ieșirea modelului este un scor continuu înainte de aplicarea pragului final, iar AUC verifică dacă imaginile bolnave tind să primească scoruri mai mari decât cele sănătoase. Această perspectivă este mai robustă decât acuratețea atunci când pragul de decizie poate fi ajustat ulterior sau când distribuția setului de validare nu reflectă perfect distribuția reală de utilizare. Practic, AUC este potrivit pentru prima etapă deoarece obiectivul ei este trierea: modelul trebuie să separe cât mai bine cazurile normale de cele care trebuie trimise către clasificarea de severitate.

Formal, AUC poate fi interpretat ca aria de sub curba ROC:

$$\text{AUC} = \int_0^1 \text{TPR}(u) du,$$

unde u reprezintă rata de fals pozitive, iar TPR este rata de adevărat pozitive obținută pentru diferite praguri de decizie. O interpretare echivalentă, mai intuitivă pentru cazul binar, este:

$$\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-),$$

unde s^+ este scorul atribuit unei imagini patologice, iar s^- este scorul atribuit unei imagini sănătoase. Astfel, AUC măsoară probabilitatea ca modelul să ordoneze corect o imagine bolnavă înaintea uneia sănătoase. Un AUC egal cu 0.5 indică un model fără capacitate reală de separare, similar alegerii aleatoare, iar un AUC apropiat de 1 indică o separare foarte bună între cele două clase.

A doua etapă este modelul de severitate 1–4, antrenat doar pe imaginile patologice din `eyepacs_1.4`. În varianta finală, problema este formulată ca regresie ordinală: modelul produce un scor continuu, iar acesta este transformat în clasele 1–4 prin praguri optimizate pe setul de validare. Funcția de pierdere folosită pentru această variantă este `SmoothL1Loss`. În notebook există și configurația cu `CrossEntropyLoss` și `label_smoothing` 0.10, însă soluția principală folosește varianta regresivă deoarece se aliniază mai bine cu ordinea naturală a gradelor bolii.

Modelul 1–4 este antrenat tot în două faze, dar cu backbone-ul înghețat pentru primele 5 epoci din totalul maxim de 50. În faza inițială se antrenează doar capul cu `AdamW`, rată de învățare 10^{-3} și `weight_decay` 10^{-4} . În faza de fine-tuning se folosesc aceleași rate diferențiate ca în detectorul binar: 10^{-4} pentru backbone și $3 \cdot 10^{-4}$ pentru cap. Scheduler-ul este `ReduceLROnPlateau`, cu `factor` 0.5 și `patience` 2, aplicat pe QWK-ul de validare. Checkpoint-ul final este ales după QWK, cu prag minim de îmbunătățire 10^{-4} și `early_stopping_patience` 30.

Pentru modelul de severitate, QWK este metrica de monitorizare principală deoarece clasele 1–4 reprezintă niveluri ordonate ale aceleiași boli. În acest caz, nu toate erorile au aceeași semnificație: o predicție de clasă 2 pentru o imagine de clasă 3 este mai acceptabilă decât o predicție de clasă 1 pentru o imagine de clasă 4. QWK introduce această idee direct în criteriul de selecție a checkpoint-ului, prin penalizarea pătratică a diferențelor mari dintre clasa reală și clasa prezisă. De aceea, modelul nu este ales doar pentru că obține multe predicții exacte, ci pentru că păstrează cât mai bine ordinea clinică a severității. Această alegere este cu atât mai importantă în varianta regresivă, unde scorul continuu este convertit în clase prin praguri optimizate pe validare.

Formula generală pentru QWK este:

$$\kappa = 1 - \frac{\sum_{i=0}^{K-1} \sum_{j=0}^{K-1} w_{ij} O_{ij}}{\sum_{i=0}^{K-1} \sum_{j=0}^{K-1} w_{ij} E_{ij}},$$

unde K este numărul de clase, O_{ij} este matricea observată a clasificărilor, adică numărul de exemple cu eticheta reală i și predicția j , iar E_{ij} este matricea acordului așteptat întâmplător, calculată pe baza distribuțiilor marginale ale etichetelor reale și predicțiilor. Pentru varianta quadratic weighted, ponderile sunt:

$$w_{ij} = \frac{(i - j)^2}{(K - 1)^2}.$$

Această pondere este 0 atunci când predicția coincide cu eticheta reală și crește pătratic pe măsură ce distanța dintre clase devine mai mare. În cazul modelului 1–4, o confuzie între clase vecine are o penalizare mai mică decât o confuzie între forme ușoare și forme severe. Scorul QWK este apropiat de 1 atunci când acordul dintre predicții și etichete este foarte bun, apropiat de 0 când acordul nu este mai bun decât cel întâmplător și poate deveni negativ dacă modelul este mai slab decât o atribuire aleatoare conform distribuțiilor observate.

Motivația antrenării în două faze este comună celor două modele principale. Separarea antrenării capului de fine-tuning-ul backbone-ului reduce riscul ca un cap inițializat aleator să degradeze rapid reprezentările pre-antrenate. Ratele de învățare diferențiate păstrează stabilitatea caracteristicilor extrase de **EfficientNet-B3**, în timp ce capul nou poate învăța mai rapid distribuția dataset-ului curent. În plus, alegerea metricii monitorizate este specifică fiecărei etape: AUC pentru detectorul binar, unde contează separarea robustă între sănătos și bolnav, și QWK pentru clasificarea severității, unde ordinea claselor este esențială. În ambele cazuri, metrica de monitorizare este folosită atât pentru scheduler, cât și pentru salvarea celui mai bun model, astfel încât optimizarea să urmărească direct comportamentul dorit al fiecărei ramuri din sistem.

5.9 Strategia de evaluare

Evaluarea se realizează în două etape. În timpul antrenării, modelul este monitorizat pe setul de validare, iar cea mai bună variantă este salvată pe disc. La final, modelul salvat este reîncărcat și evaluat pe setul de testare. Pentru experimentele de generalizare între domenii, implementarea permite evaluarea aceluiași model pe mai multe seturi de test în aceeași rulare, prin argumentul `test_sources`. Astfel, un model antrenat pe date combinate poate fi testat separat pe APTOS și pe setul Balanced, fără reluarea antrenării. Această separare este necesară pentru ca testarea să rămână o evaluare cât mai obiectivă a modelului ales.

Metricile urmărite sunt funcția de pierdere, acuratețea, AUC-ul multiclasă, quadratic weighted kappa, raportul de clasificare, matricea de confuzie și curbele ROC. AUC-ul este calculat în regim one-vs-rest și mediat macro, astfel încât fiecare clasă să contribuie egal la scorul final.

Pentru retinopatia diabetică, QWK este relevantă deoarece clasele au o ordine naturală. O eroare între două clase vecine este mai puțin gravă decât o eroare între clase îndepărtate, iar această metrică penalizează mai puternic diferențele mari de severitate. Din acest motiv, experimentele recente folosesc frecvent QWK ca metrică principală de monitorizare. În plus,

scriptul include generarea de hărți Grad-CAM, utile pentru a observa regiunile imaginii care influențează predicția modelului.



Figura 5.4: Exemplu de hărți Grad-CAM pentru un model ResNet50 antrenat cu Focal Loss și Adam

5.10 Implementarea procesului de antrenare

Scriptul `train.py` coordonează întregul proces experimental. După citirea argumentelor, acesta selectează dispozitivul de calcul disponibil, creează directoarele pentru rezultate și încarcă datele prin clasele de dataset definite în `my_dataset.py`. Implementarea permite folosirea surselor `balanced`, `balanced_aug`, `aptos`, precum și a combinațiilor `combined` și `balanced_aug_aptos`. În plus, scriptul poate primi mai multe directoare de dataset pentru antrenare și le poate trata ca o sursă comună, concatenată. Astfel, un experiment poate fi rulat pe imagini provenite din mai multe distribuții, fără a crea manual un nou director intermediar. În mod similar, testarea poate fi făcută pe mai multe seturi, fiecare păstrat ca sursă separată de evaluare. Fiecare dataset este transmis către un `DataLoader`, cu `shuffle=True` pentru antrenare și `shuffle=False` pentru validare și testare.

Pentru fiecare epocă, modelul trece printr-o fază de antrenare și una de validare. În faza de antrenare se calculează gradientul și se actualizează parametrii, iar în faza de validare calculele sunt realizate fără gradient. Dacă metrica de selecție se îmbunătățește, ponderile sunt salvate într-un fișier `.pth`. La final, scriptul generează graficele de istoric, matricea de confuzie, curbele ROC și, atunci când este posibil, imaginile Grad-CAM. În cazul evaluării pe mai multe surse de test, fiecare set primește fișiere separate, marcate în nume prin sufixe precum `test_aptos`, `test_balanced` sau `test_balanced_aug`.

Un avantaj important al scriptului este flexibilitatea oferită prin argumentele din linia de comandă. Fără modificarea codului sursă, se poate selecta arhitectura, funcția de pierdere, optimizatorul, numărul de epoci, dimensiunea batch-ului, rata de învățare, dimensiunea imaginilor, ponderile claselor, valoarea `gamma` pentru Focal Loss și metrica folosită pentru salvarea celui mai bun model. În plus, scriptul permite activarea sau dezactivarea unor tehnici precum `amp`, `grad_clip`, `ema_decay`, `model_dropout`, `freeze_backbone_epochs`, `backbone_lr_mult` și `early_stopping_patience`. Această organizare a fost utilă în experimentele exploratorii, deoarece aceeași implementare putea fi rulată rapid în configurații diferite.

Flexibilitatea este prezentă și la nivelul dataset-urilor. Argumentele importante din `train.py` sunt:

- `--train_dataset_roots` primește unul sau mai multe directoare folosite pentru antrenare și validare. Dacă sunt transmise mai multe căi, seturile sunt concatenate logic în timpul încărcării datelor.

- `--train_dataset_names` permite atribuirea unor nume clare surselor de antrenare, astfel încât experimentul și fișierele rezultate să poată fi identificate ușor.
- `--test_dataset_roots` primește unul sau mai multe directoare folosite pentru testarea finală. Rezultatele sunt raportate separat pentru fiecare sursă de test.
- `--test_dataset_names` definește numele surselor de test, folosite apoi în sufixele fișierelor, de exemplu `test_apto`s sau `test_balanced`.
- `--train_augment` controlează augmentările online, prin valorile `basic` și `none`. Această opțiune este utilă atunci când dataset-ul conține deja augmentări sau imagini preprocesate offline.
- `--model`, `--loss_name`, `--optimizer` selectează arhitectura, funcția de pierdere și optimizatorul, permițând compararea rapidă a configurațiilor experimentale.
- `--monitor_metric` stabilește criteriul după care este salvat cel mai bun model; în experimentele documentate a fost folosit QWK.
- `--scheduler`, `--amp`, `--grad_clip` și `--ema_decay` activează tehnici suplimentare pentru stabilizarea antrenării și evaluării.

Astfel, același model poate fi antrenat pe date combinate și testat separat pe APTOS, Balanced sau Balanced Aug, iar graficele rezultate sunt salvate cu nume care reflectă sursa de antrenare și sursa de testare. Această organizare a fost utilă pentru experimentele de generalizare între dataset-uri, deoarece a permis compararea comportamentului modelului pe distribuții diferite în aceeași rulare experimentală.

Pentru seturile care conțin deja imagini augmentate sau preprocesate, scriptul permite dezactivarea augmentărilor online prin `train_augment=none`. Această opțiune evită aplicarea dublă a augmentărilor în experimentele recente, păstrând în același timp comportamentul vechi prin valoarea implicită `train_augment=basic`.

Capitolul 6

Rezultate experimentale

Înainte de prezentarea experimentelor principale, capitolul pornește de la rezultatele preliminare obținute pe setul `Diabetic_Balanced_Data`. Această ordine este utilă deoarece comparațiile între funcțiile de pierdere și arhitecturi au fost realizate inițial pe un set public care părea potrivit pentru experimente controlate, dar care a evidențiat ulterior o problemă importantă de *data leakage*. Astfel, primele secțiuni au rolul de a explica de ce scorurile foarte ridicate trebuie interpretate cu prudență și de ce a fost necesară reorganizarea datelor înainte de experimentele finale.

După această analiză de context, capitolul continuă cu configurația experimentală folosită pentru experimentele proprii: comparația dintre APTOS, EyePACS și `Balanced_Aug` în variantele baseline, augmentate și augmentate + preprocesate. În acest fel, rezultatele principale sunt prezentate după ce au fost clarificate limitele dataset-urilor inițiale și ale protocoalelor afectate de scurgeri de informație.

6.1 Compararea funcțiilor de pierdere pe `Diabetic_Balanced_Data`

Înainte de interpretarea comparațiilor din această secțiune, trebuie menționat explicit că varianta publică a setului `Diabetic_Balanced_Data` conține *data leakage*, deoarece augmentări ale aceleiași imagini pot ajunge în subseturi diferite. Din acest motiv, valorile absolute obținute pe acest set trebuie privite cu prudență și nu trebuie confundate cu o estimare finală a generalizării. Totuși, setul rămâne util pentru comparații experimentale între funcții de pierdere și arhitecturi, deoarece toate configurațiile sunt rulate în aceleași condiții, pe aceeași distribuție și cu același protocol de evaluare.

Figura 6.1 sintetizează istoricul funcției de pierdere pentru patru configurații ResNet50: *focal loss* cu Adam, *cross-entropy* cu Adam, *weighted cross-entropy* cu AdamW și *focal loss* cu AdamW. Scopul graficului nu este alegerea unui model final pe baza acestui dataset, ci observarea comportamentului relativ al funcțiilor de pierdere în timpul antrenării.

Se observă că toate configurațiile învață rapid pe setul de antrenare, însă diferența dintre curba de antrenare și cea de validare rămâne importantă pentru interpretare. *Cross-entropy* simplu este o alegere stabilă ca punct de plecare, dar poate favoriza exemplele ușor de clasificat. Varianta ponderată nu aduce un avantaj clar în acest context, deoarece setul este deja echilibrat artificial, iar ponderarea suplimentară nu rezolvă principala problemă a datelor. În schimb, *focal loss* este mai potrivită pentru continuarea experimentelor deoarece pune accent pe exemplele dificile și pe confuziile care apar între clasele apropiate.

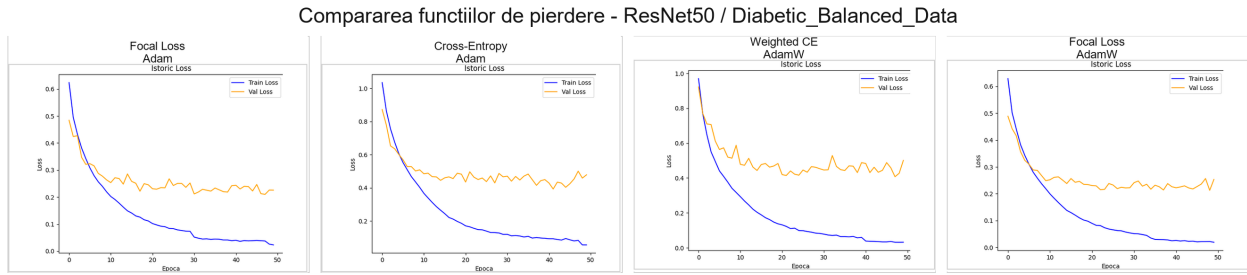


Figura 6.1: Comparația istoricului loss-ului pentru ResNet50 pe Diabetic_Balanced_Data.

6.2 Compararea arhitecturilor pe Diabetic_Balanced_Data

Pentru comparația arhitecturilor au fost analizate matricile de confuzie absolute pentru ResNet50, InceptionV3, InceptionResNet-v2 și EfficientNet-B3, în configurații rulate pe același set Diabetic_Balanced_Data. Figura 6.2 arată nu doar câte imagini sunt clasificate corect, ci și direcția erorilor dintre clase.

Rezultatele sunt apropiate la nivel global, majoritatea configurațiilor obținând valori în jurul intervalului 0,88–0,90 pentru acuratețe. De aceea, interpretarea relevantă nu este doar care model are scorul maxim, ci unde apar confuziile. Clasele 3 și 4 sunt recunoscute bine de toate arhitecturile, ceea ce indică faptul că formele severe au trăsături vizuale mai ușor de separat. În schimb, clasele 0, 1 și 2 concentrează cele mai importante erori, deoarece diferențele dintre absența bolii, formele ușoare și formele moderate sunt mai subtile.

EfficientNet-B3 are comportamentul cel mai echilibrat în aceste teste, cu aproximativ 0,90 acuratețe, dar diferența față de celelalte arhitecturi nu este suficientă pentru a elimina complet ResNet50 din analiză. ResNet50 rămâne un baseline robust, mai simplu de antrenat și util pentru comparațiile dintre preprocesări, augmentări și funcții de pierdere. InceptionV3 și InceptionResNet-v2 confirmă aceeași tendință generală: arhitectura influențează distribuția erorilor, însă calitatea datelor și protocolul de împărțire rămân decisive.

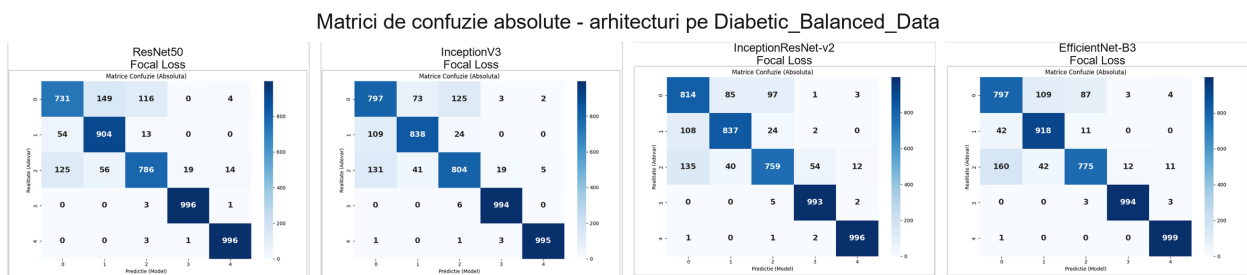


Figura 6.2: Matrici de confuzie pentru arhitecturile comparate pe Diabetic_Balanced_Data.

6.3 Analiza data leakage în APTOS și Diabetic_Balanced_Data

O etapă separată a analizei a fost dedicată verificării riscului de *data leakage*. Această verificare a devenit necesară după examinarea setului Diabetic_Balanced_Data, unde imaginile originale și augmentările derivate din aceeași imagine puteau ajunge în subseturi diferite. Într-un astfel de scenariu, modelul nu mai este evaluat pe imagini complet independente, ci poate întâlni în validare sau testare variante foarte apropiate de exemplele văzute la antrenare. Rezultatul este o estimare artificial optimistă a performanței.

Importanța verificării este cu atât mai mare cu cât setul `Diabetic_Balanced_Data` este disponibil public [32] și apare inclusiv în articole de tip review despre dataset-uri fundus utilizate pentru retinopatie diabetică [34]. Prezența într-o sinteză bibliografică nu garantează însă că organizarea locală a fișierelor este potrivită pentru un protocol experimental fără scurgeri de informație. Din acest motiv, ultimele experimente din notebook-ul `experiments.ipynb`, secțiunea *Experimente APTOS data-leakage + Balanced_Data fără data-leakage*, au fost folosite ca experimente de control.

Primul a fost realizat pe APTOS, deoarece pentru acest set se putea construi direct o comparație între două fluxuri. În varianta incorectă, augmentarea a fost aplicată înainte de împărțirea în train, validare și test, ceea ce permite ca transformări ale aceleiași imagini să apară în subseturi diferite. În varianta corectă, împărțirea a fost realizată înainte de augmentare, iar augmentările au rămas în subsetul imaginii originale. Diferența dintre rezultate este puternică: fluxul *augment* -> *split* obține o acuratețe de 96,20%, AUC de 0,9982 și QWK de 0,9813, în timp ce fluxul *split* -> *augment* coboară la o acuratețe de 82,02% și AUC de 0,9170. Această diferență arată că performanța foarte ridicată nu provenea doar din capacitatea modelului, ci și din contaminarea dintre subseturi.

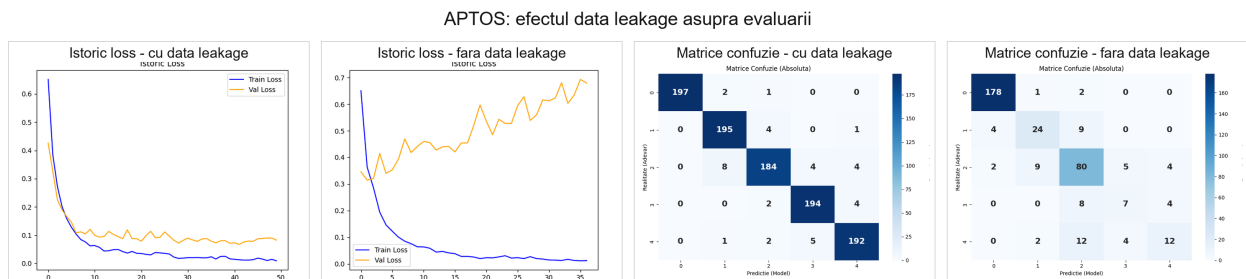


Figura 6.3: Comparație APTOS între fluxul cu *data leakage* și fluxul corect fără *data leakage*: istoricul funcției de pierdere și matricea de confuzie absolută pentru fiecare variantă.

Al doilea control a urmărit direct setul `Diabetic_Balanced_Data`. În varianta distribuită public, rezultatele sunt ridicate: acuratețe de 88,77%, AUC de 0,9825 și QWK de 0,9297. După reorganizarea setului printr-un split grupat, în care imaginea originală și toate augmentările asociate rămân în același subset, performanța scade la o acuratețe de 60,47%, AUC de 0,8878 și QWK de 0,7641. Scăderea nu trebuie interpretată ca o degradare a metodei, ci ca o corectare a protocolului de evaluare: noul rezultat reflectă mai realist capacitatea modelului de a generaliza pe imagini independente.

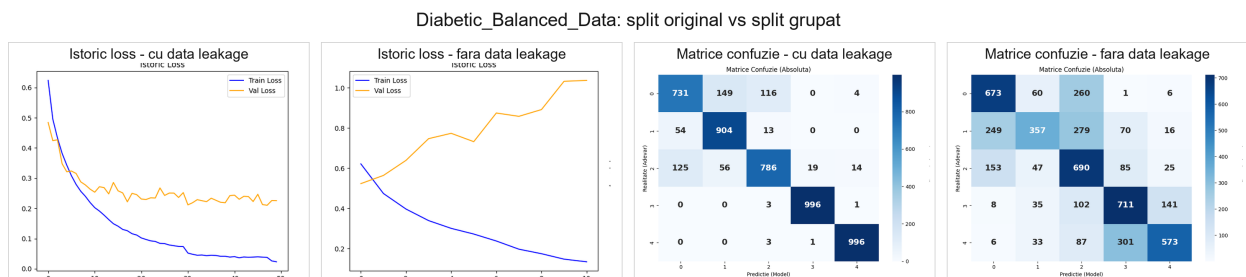


Figura 6.4: Comparație pentru `Diabetic_Balanced_Data` între varianta publică afectată de *data leakage* și varianta rearanjată prin split grupat: istoricul funcției de pierdere și matricea de confuzie absolută pentru fiecare variantă.

După eliminarea scurgerii de informație, matricele de confuzie devin și ele mai realiste. În varianta cu *data leakage*, diagonala principală este mult mai puternică deoarece modelul poate

recunoaște imagini foarte asemănătoare cu cele văzute în antrenare. În varianta cu split grupat, confuziile cresc, însă acestea reflectă mai bine dificultatea reală a problemei: modelul trebuie să generalizeze pe imagini independente, nu pe transformări apropiate ale acelorași funduri de ochi.

Cele mai multe erori apar între clase care sunt apropiate clinic sau vizual.

În `Diabetic_Balanced_Data` reorganizat, clasa 1 este confundată frecvent cu clasele 0 și 2, deoarece semnele de retinopatie ușoară sunt subtile și pot fi apropiate fie de imagini normale, fie de forme moderate. Clasa 2 este confundată atât cu 0, cât și cu 3, ceea ce indică dificultatea delimitării stadiului moderat atunci când leziunile sunt parțiale, mici sau afectate de calitatea imaginii. Pentru clasele severe, se observă confuzii între 3 și 4, deoarece ambele pot conține leziuni avansate, iar diferența dintre retinopatie severă și proliferativă nu este întotdeauna evidentă doar din imaginea globală. Aceste confuzii sunt de așteptat într-un protocol corect și sunt mai utile pentru analiză decât o matrice aproape perfectă obținută prin contaminarea subseturilor.

Aceste experimente justifică folosirea variantei reorganizate cu split grupat și a scriptului `Scripts/DIabetic_Balanced_Data/grouped_split_diabetic_balanced_data.py`. Ele arată că, în clasificarea retinopatiei diabetice, protocolul de împărțire a datelor este la fel de important ca arhitectura sau funcția de pierdere. În plus, ele explică de ce rezultatele obținute pe seturi care conțin augmentări preexistente trebuie interpretate cu prudență: o acuratețe mare poate semnală un model bun, dar poate semnală și faptul că separarea train–validare–test nu a fost suficient de strictă sau dezechilibră între clase.

6.4 Configurația experimentală

Capitolul de rezultate este actualizat pe baza experimentelor rulate în notebook-ul `experiments.ipynb`. Spre deosebire de varianta inițială, care compara în principal arhitecturi și configurații izolate, analiza de față urmărește efectul etapelor de lucru asupra seturilor de date: varianta de bază, varianta cu augmentări și varianta în care augmentările sunt combinate cu preprocesări de tip Ben Graham. Pentru a păstra comparația coerentă, rezultatele sunt prezentate sub forma unei matrici experimentale cu trei coloane și trei linii.

Coloanele corespund surselor de date analizate:

- **APTOS** – setul APTOS 2019 reorganizat local și extins cu variante augmentate și preprocesate;
- **EyePACS** – setul EyePACS procesat local, folosit pentru a observa comportamentul pe o sursă mare și variabilă;
- **Balanced_Aug** – setul derivat din `Diabetic_Balanced_Aug`, utilizat pentru a analiza efectul echilibrării și al augmentărilor.

Liniile urmăresc gradul de intervenție asupra datelor:

- **Baseline** – imaginile reorganizate și împărțite în partiții de antrenare, validare și testare;
- **Augmentări** – imaginile extinse prin transformări geometrice și fotometrice, cu scopul de a crește diversitatea vizuală;
- **Augmentări + preprocesări** – imaginile augmentate și apoi normalizate vizual prin preprocesări de tip Ben Graham, pentru reducerea variațiilor de iluminare și contrast.

Pentru InceptionResNet-v2, acolo unde notebook-ul nu conține exportul baseline cu exact aceeași denumire ca în cazul ResNet50, a fost folosită varianta curățată disponibilă pentru aceeași sursă de date. Astfel se păstrează comparația pe cele trei niveluri de lucru, fără a introduce rezultate nesalvate în experimente.

Experimentele incluse în această sinteză folosesc funcția *focal loss* cu $\gamma = 1.5$, iar rulările ResNet50 au fost actualizate cu optimizatorul AdamW. Pentru istoricul funcției de pierdere sunt prezentate separat două arhitecturi, ResNet50 și InceptionResNet-v2, iar scorurile de acuratețe și QWK sunt păstrate pentru ResNet50, ca model de referință în analiza pe cele nouă variante. Pentru scorurile precision/recall/F1 pe clase, matricea de confuzie și curbele ROC multiclasă este păstrată sinteza pe ResNet50, folosită ca variantă principală în comparația finală. Această organizare face vizibilă contribuția muncii depuse asupra seturilor de date: fiecare rând adaugă o etapă de prelucrare, iar fiecare coloană arată dacă aceeași strategie se comportă similar sau diferit pe o altă sursă de imagini.

6.5 Istoricul funcției de pierdere

Figurile 6.5 și 6.6 prezintă evoluția funcției de pierdere pentru cele nouă experimente, separat pentru ResNet50 și InceptionResNet-v2. Cele două grafice nu reprezintă însă exact același moment al dezvoltării codului. Pentru ResNet50 sunt folosite rulările actualizate, după introducerea ultimelor setări de fine-tuning, cu AdamW, scheduler, înghețarea inițială a backbone-ului și rată de învățare mai mică pentru straturile pre-antrenate. Pentru InceptionResNet-v2 sunt păstrate rezultatele obținute cu varianta de cod de dinaintea acestor ajustări finale de fine-tuning. Din acest motiv, comparația este utilă pentru a observa comportamentul celor două arhitecturi, dar interpretarea curbelor trebuie să țină cont de diferența dintre regimurile de antrenare.

Curbele evidențiază o diferență constantă între pierderea pe antrenare și pierderea pe validare: în multe cazuri, *train loss* scade rapid, în timp ce *val loss* rămâne mai instabil sau chiar crește. Acest comportament indică faptul că problema nu este doar una de capacitate a modelului, ci și una de generalizare între imaginile de antrenare și cele de validare. În clasificarea retinopatiei diabetice, această diferență este de așteptat, deoarece imaginile pot proveni din surse diferite, cu iluminare, rezoluție, câmp vizual și calitate variabile, iar clasele vecine sunt greu de separat chiar și vizual.

Compararea pe rânduri arată că augmentările cresc diversitatea setului de antrenare, dar nu produc automat o scădere uniformă a pierderii pe validare. În cazul imaginilor cu augmentări și preprocesări, curbele confirmă că normalizarea vizuală modifică distribuția intrărilor și poate face antrenarea mai controlabilă, însă efectul depinde de dataset. Această observație este importantă deoarece justifică evaluarea separată pentru APTOS, EyePACS și Balanced_Aug, nu doar raportarea unui singur scor global.

Graficul din figura 6.5 urmărește evoluția pierderii pentru ResNet50 în configurațiile actualizate cu fine-tuning. În general, pierderea pe antrenare coboară vizibil în primele epoci, semn că modelul reușește să adapteze rapid capul de clasificare și ultimele straturi la distribuția fiecărui dataset. Scăderea nu este însă perfect liniară, deoarece *focal loss* acordă o importanță mai mare exemplelor dificile, iar acestea pot produce oscilații de la o epocă la alta. Pe validare, curbele sunt mai puțin netede și se pot plafona mai repede, ceea ce arată că modelul învață relativ ușor datele de antrenare, dar generalizarea depinde mult de sursa imaginilor și de preprocesările aplicate.

Pentru ResNet50, ultimele setări de fine-tuning ajută la controlul acestei dinamici. Înghețarea inițială a backbone-ului permite capului nou să se stabilizeze înainte ca întregul model să fie actualizat, iar rata de învățare mai mică pentru backbone reduce riscul de a modifica agresiv reprezentările pre-antrenate. Din acest motiv, chiar dacă apare o distanță între *train loss* și *val loss*, curbele ResNet50 sunt mai ușor de interpretat: scăderea pierderii pe antrenare arată învățarea efectivă, iar stagnarea sau creșterea ușoară pe validare semnalează limita de generalizare pentru combinația respectivă de dataset și preprocesare.

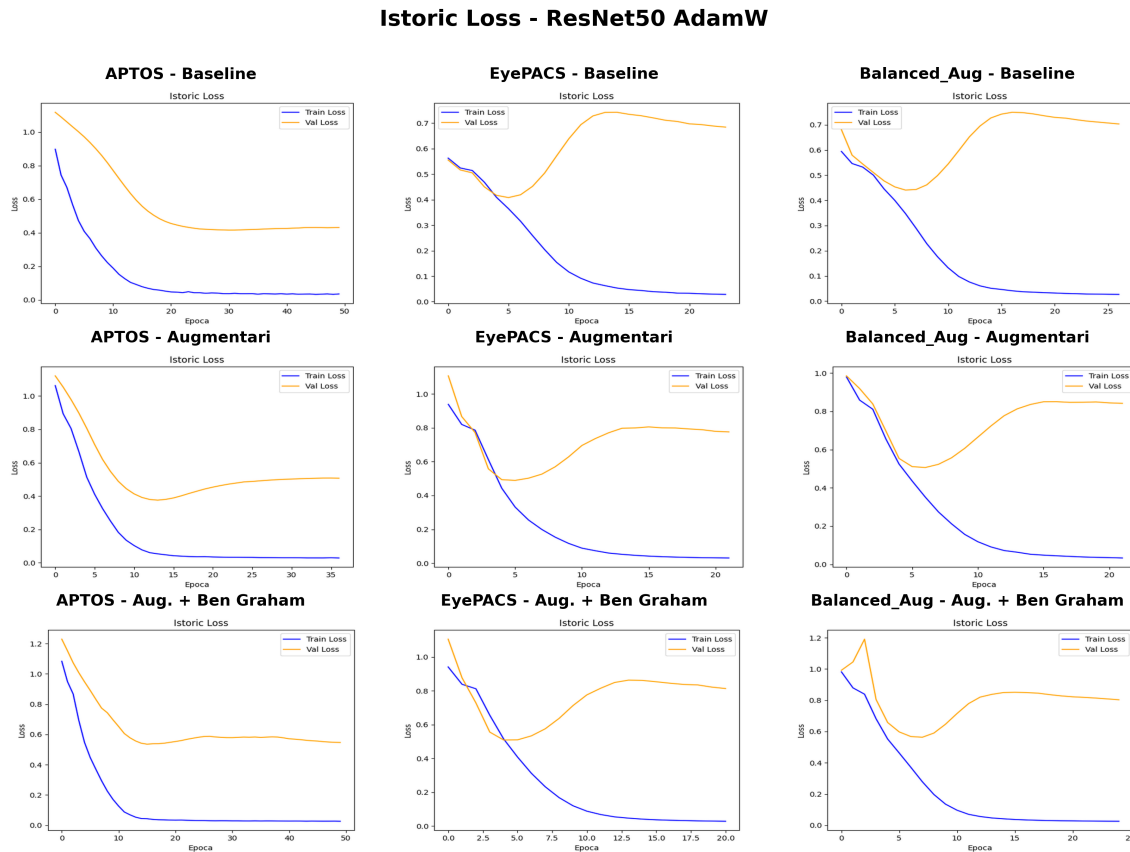


Figura 6.5: Istoricul loss-ului pentru ResNet50 cu AdamW pe APTOS, EyePACS și Balanced_Aug.

Pentru InceptionResNet-v2, istoricul loss-ului arată aceeași tensiune între învățarea pe train și stabilitatea pe validare, dar rezultatele trebuie citite ca rezultate anterioare etapei finale de fine-tuning. În această variantă, modelul nu beneficiază de aceleași ajustări aplicate ulterior rulărilor ResNet50, iar arhitectura este mai complexă și mai sensibilă la alegerea ratei de învățare. De aceea, în unele experimente pierderea pe validare crește în loc să scadă, chiar dacă la un moment dat se stabilizează.

Creșterea *val loss*-ului pentru InceptionResNet-v2 poate fi explicată prin mai mulți factori. În primul rând, modelul are o capacitate mare și poate începe să se adapteze prea specific la imaginile de antrenare, în timp ce validarea rămâne mai dificilă sau mai diferită vizual. În al doilea rând, folosirea *focal loss* face ca exemplele grele să aibă o contribuție mai mare la pierdere; dacă setul de validare conține cazuri ambigue sau clase greu separabile, loss-ul poate crește chiar și atunci când modelul începe să învețe anumite tipare utile. În al treilea rând, lipsa fine-tuning-ului în două faze poate face actualizarea backbone-ului mai instabilă la început, deoarece capul de clasificare nu este încă suficient de bine adaptat la problema curentă.

Faptul că loss-ul se stabilizează după o anumită perioadă arată că antrenarea nu este complet divergentă, ci ajunge într-un regim relativ constant. Totuși, stabilizarea la o valoare mai ridicată decât cea inițial dorită indică faptul că modelul nu reușește să transforme această stabilitate într-o generalizare mai bună pe validare. Din acest motiv, InceptionResNet-v2 rămâne util ca experiment comparativ, dar concluziile principale sunt formulate pe baza rulărilor ResNet50 actualizate, unde fine-tuning-ul final oferă un protocol de antrenare mai controlat.

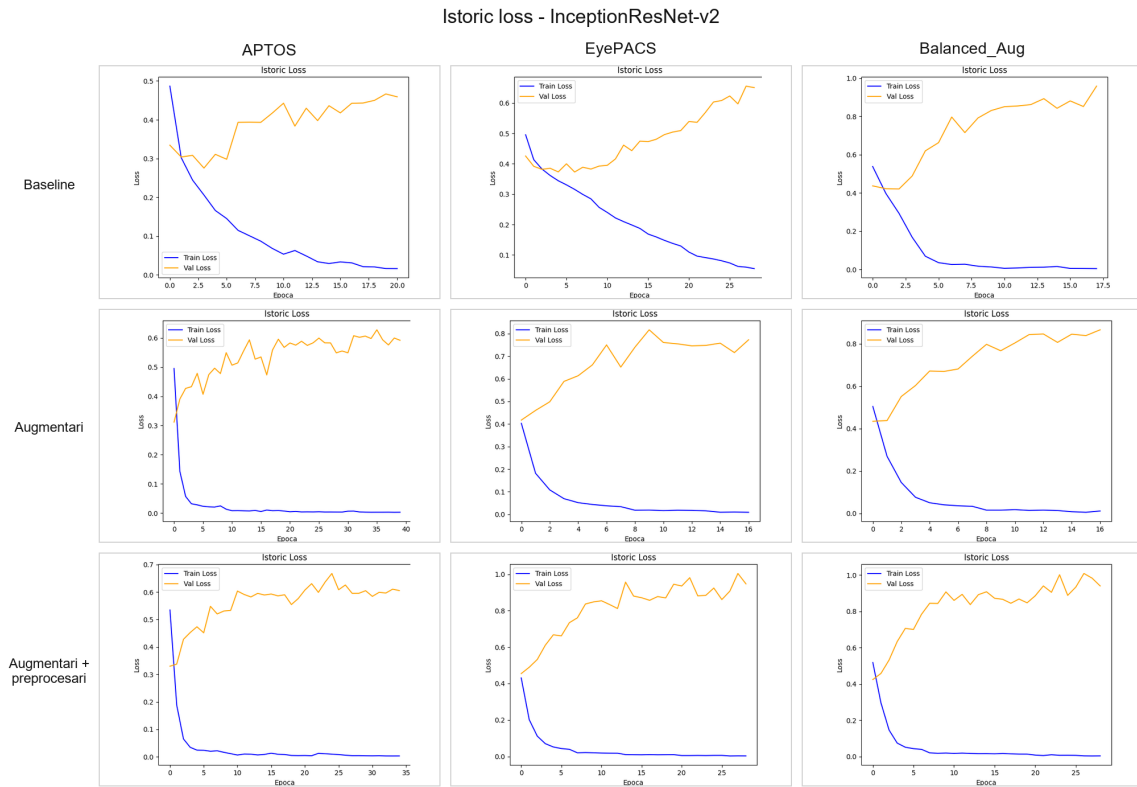


Figura 6.6: Istoricul loss-ului pentru InceptionResNet-v2 pe APTOS, EyePACS și Balanced_Aug.

6.6 Istoricul acurateții

Figura 6.7 prezintă evoluția acurateții pentru rulările ResNet50 actualizate. În gradarea retinopatiei diabetice, acest scor global trebuie interpretat cu prudență: modelul poate obține o acuratețe ridicată prin favorizarea claselor mai numeroase, fără să recunoască suficient clasele intermediare sau cazurile severe. Acuratețea rămâne utilă ca indicator rapid al progresului antrenării, însă concluziile sunt formulate în principal pe baza QWK, F1 pe clase și matrici de confuzie.

În experimente, acuratețea de antrenare crește rapid în majoritatea cazurilor, ceea ce arată că modelul reușește să învețe exemplele disponibile. Diferența față de acuratețea de validare este însă esențială pentru interpretare: un scor ridicat pe antrenare nu este suficient dacă validarea rămâne instabilă sau plafonată.

Rezultatele finale confirmă această limitare: pe APTOS se obțin acurateți de 0.82, 0.79 și 0.81 pentru baseline, augmentări și augmentări cu Ben Graham, în timp ce pe EyePACS valorile sunt 0.77, 0.74 și 0.73. Pentru Balanced_Aug, acuratețea scade de la 0.75 la 0.72 și 0.69, deși macro-F1 crește de la 0.42 la 0.49, respectiv 0.51. Prin urmare, augmentările și preprocesarea

nu trebuie evaluate doar prin acuratețe, ci prin distribuția erorilor pe clase și prin stabilitatea metricilor ordinale.

Istoric Acuratete - ResNet50 AdamW

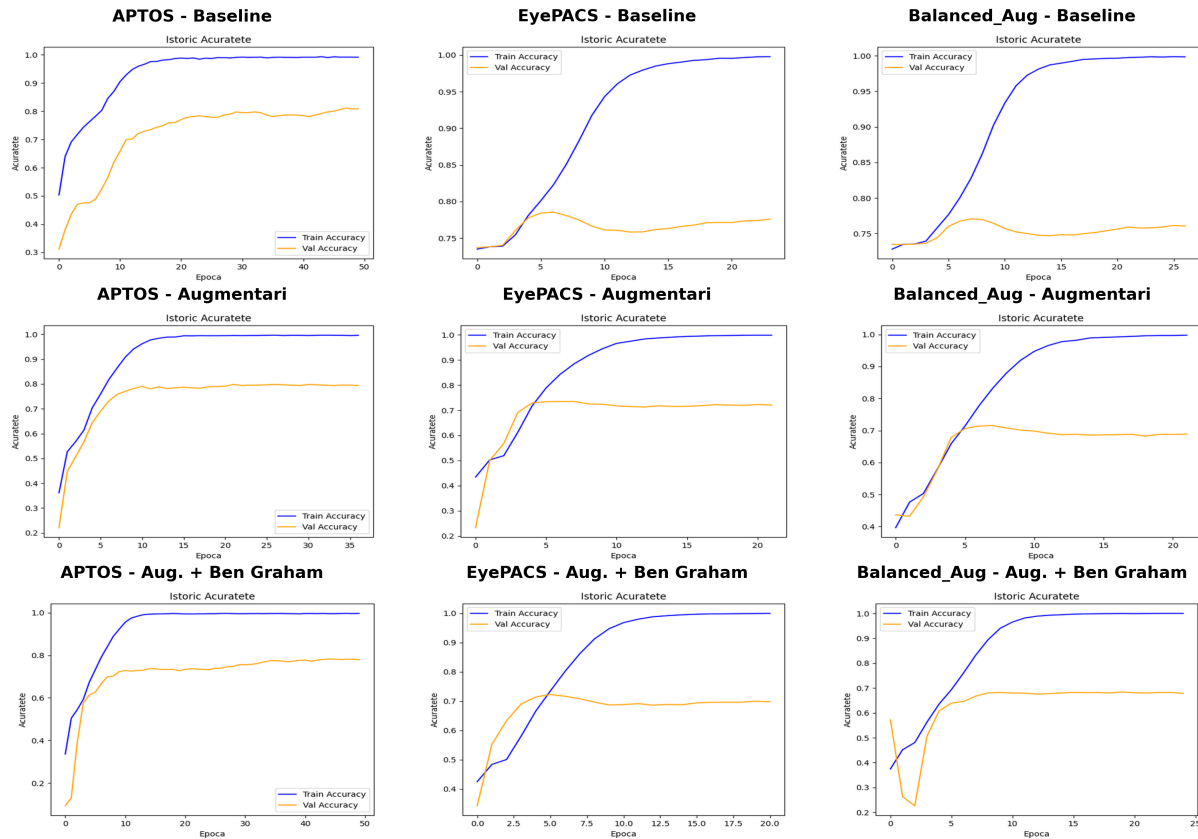


Figura 6.7: Istoricul acurateții pentru ResNet50 cu AdamW pe cele nouă variante de experimente.

6.7 Istoricul scorului QWK

Pentru gradarea retinopatiei diabetice, acuratețea nu surprinde complet severitatea erorilor. De aceea, figura 6.8 urmărește scorul QWK, o metrică mai potrivită pentru clase ordinale. O confuzie între clase vecine, de exemplu 1 și 2, este mai puțin severă decât o confuzie între clasa 0 și clasa 4, iar QWK reflectă această diferență.

Istoricul QWK arată cât de stabil devine modelul în raport cu ordinea clinică a claselor. În experimentele în care QWK crește odată cu antrenarea, modelul nu doar clasifică mai multe imagini corect, ci învață o structură mai apropiată de severitatea bolii. Această metrică este relevantă pentru lucrare deoarece sistemul final nu urmărește doar o etichetă numerică, ci o estimare a stadiului retinopatiei diabetice.

QWK este urmărit deoarece clasele au ordine clinică. Pentru ResNet50, acest grafic arată dacă modelul învață nu doar etichete corecte, ci și o separare coerentă între gradele de severitate.

Istoric QWK - ResNet50 AdamW

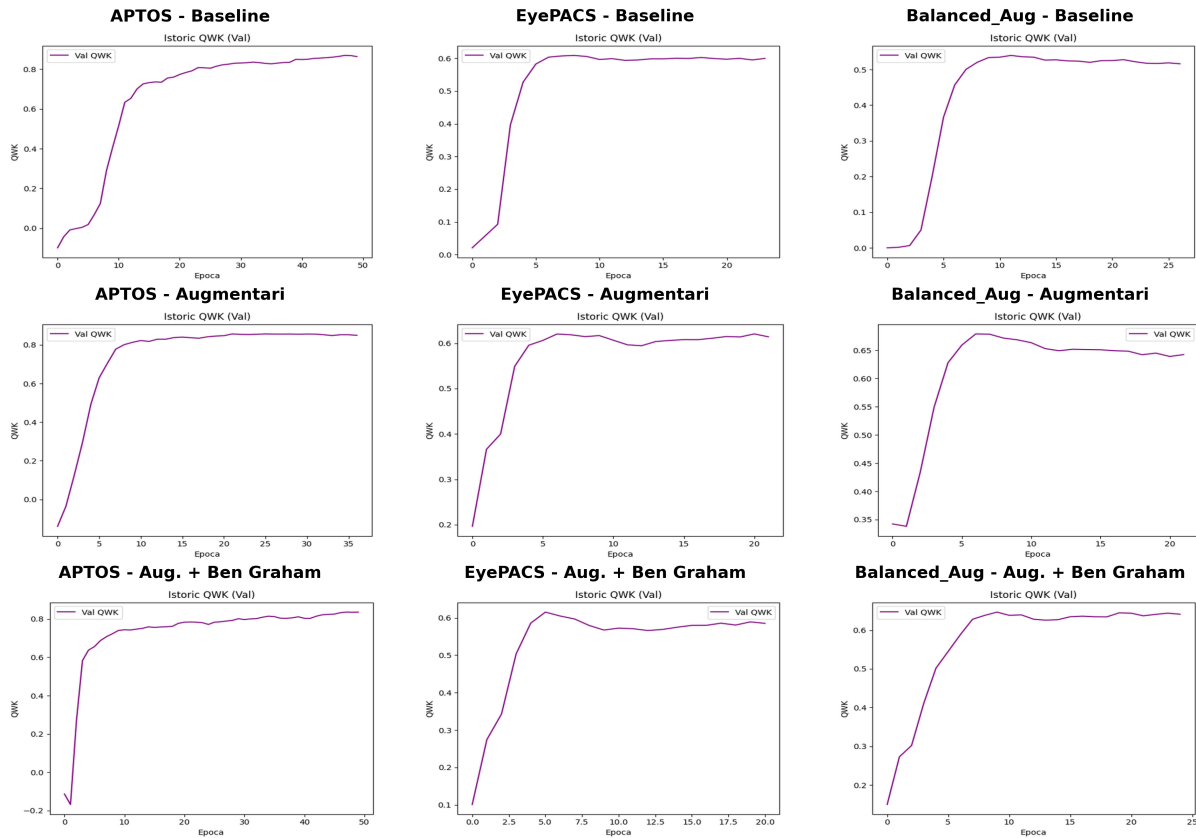


Figura 6.8: Istoricul scorului QWK pentru ResNet50 cu AdamW pe cele nouă variante de experimente.

Din figura 6.8 se observă că evoluția QWK nu este identică pentru toate combinațiile de set de date și strategie de antrenare. Pe APTOS, curbele ajung în zona 0.83–0.87, semn că modelul păstrează mai bine ordinea severității. Pe EyePACS, QWK se stabilizează în jurul intervalului 0.59–0.62, iar pe Balanced_Aug valorile sunt mai sensibile la etapa de pregătire a datelor, cu un comportament mai bun pentru variantele augmentate decât pentru baseline. Diferența față de acuratețe arată că o scădere a scorului global nu înseamnă neapărat o degradare ordinală uniformă.

Un aspect important este că QWK penalizează mai puternic confuziile între clase îndepărtate. De aceea, chiar dacă acuratețea poate părea apropiată între două experimente, o curbă QWK mai bună indică faptul că modelul greșește mai rar severitatea într-un mod clinic grav. Pentru această lucrare, graficul este util deoarece arată nu doar dacă ResNet50 ajunge la performanțe bune, ci și dacă învățarea este stabilă în raport cu ordinea naturală a claselor de retinopatie diabetică.

6.8 Precision, recall și F1 pe clase

Figura 6.9 prezintă valorile precision, recall și F1 pentru fiecare clasă. Această analiză este mai informativă decât acuratețea globală deoarece arată unde modelul recunoaște bine imaginile și unde apar dificultăți. În noile rulări, APTOS are cele mai bune scoruri pe clasele intermediare: F1 pentru clasa 1 este 0.56, 0.51 și 0.62, iar pentru clasa 2 este 0.73, 0.73 și 0.76, în ordinea baseline, augmentări și augmentări cu Ben Graham. Pentru EyePACS și Balanced_Aug, clasa

1 rămâne cea mai dificilă, dar varianta `Balanced_Aug` cu Ben Graham ridică F1-ul clasei 1 la 0.25, față de 0.07 în baseline.

Prin raportarea pe clase se observă mai clar efectul augmentărilor. Dacă augmentările îmbunătățesc recall-ul pentru clasele rare, modelul devine mai util în scenarii în care ratarea cazurilor patologice este costisitoare. Dacă precision-ul scade, apare riscul de alarme false. Din acest motiv, scorurile pe clase trebuie analizate împreună cu matricele de confuzie, nu izolat. În ansamblu, graficele F1 arată că principala dificultate nu este separarea extremelor, ci delimitarea fină a stadiilor apropiate. Clasele 1 și 2 sunt cele mai importante de urmărit, deoarece diferențele vizuale dintre absența bolii, formele ușoare și formele moderate pot fi subtile.

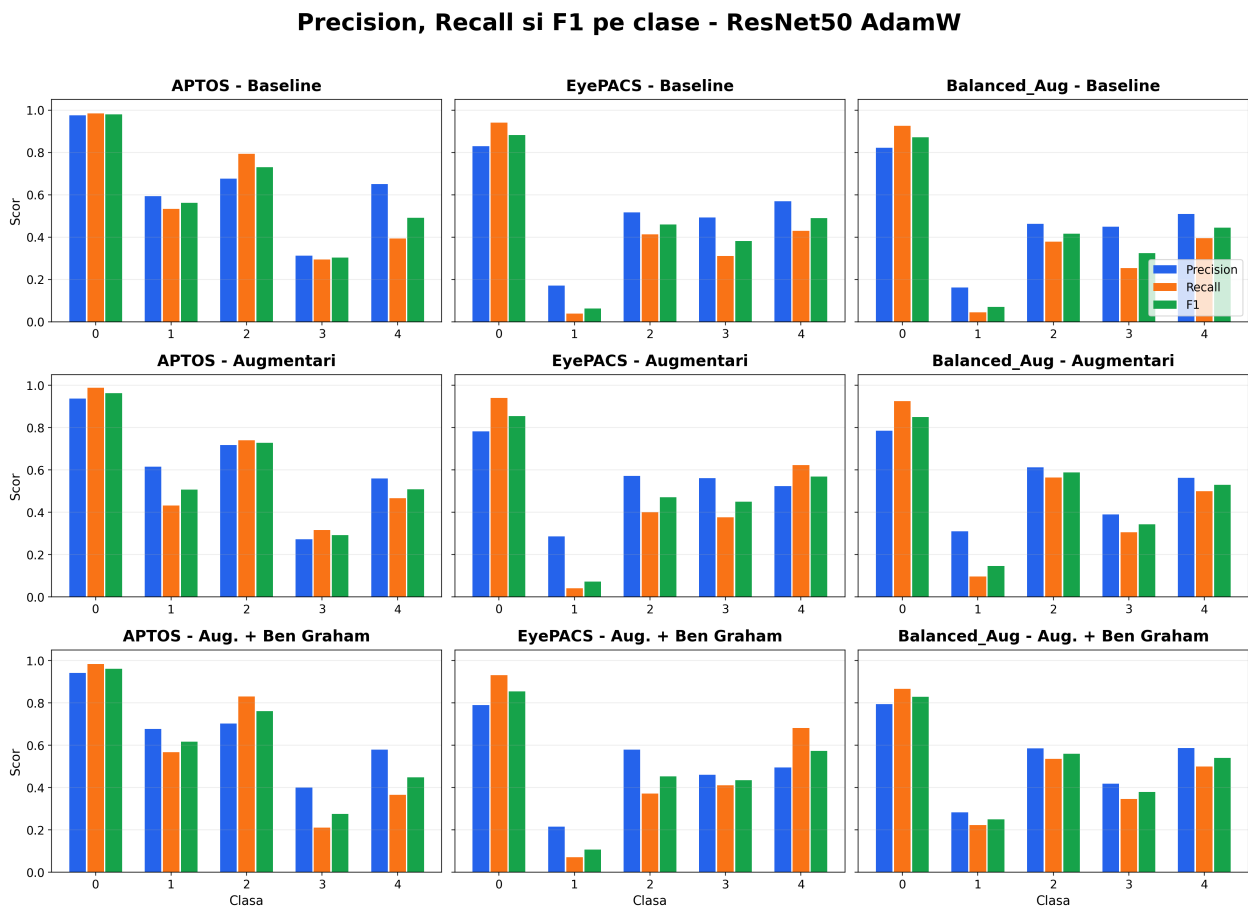


Figura 6.9: Precision, recall și F1 pe clase pentru cele nouă experimente ResNet50 cu AdamW.

6.9 Matricele de confuzie

Matricele de confuzie din figura 6.10 arată explicit tipurile de erori produse de model. Pentru retinopatia diabetică, nu toate erorile au aceeași semnificație: confuziile între clase apropiate pot reflecta ambiguitatea naturală a imaginilor, în timp ce confuziile între clasa 0 și clasele avansate indică o problemă mai gravă de detecție.

Aceste rezultate confirmă importanța curățării seturilor de date și a controlului asupra augmentărilor. Atunci când imaginile augmentate sunt distribuite incorect între partiții sau când preprocesările nu sunt aplicate consecvent, matricea de confuzie poate sugera performanțe

artificiale sau erori greu de interpretat. De aceea, în această lucrare, pregătirea dataseturilor a fost tratată ca o etapă experimentală centrală, nu doar ca un pas auxiliar înainte de antrenare.

Pentru claritate, figura păstrează doar matricile absolute, nu și variantele normalizate. Valorile absolute fac mai ușor de observat unde se concentrează efectiv erorile. Se remarcă un tipar comun: o parte din imaginile din clasele 1 și 2 sunt prezise ca 0, ceea ce arată că modelul are dificultăți în delimitarea cazurilor incipiente față de absența bolii. Totuși, clasele marginale nu sunt amestecate puternic între ele: imaginile sănătoase nu sunt confundate masiv cu stadiile severe, iar clasa 4 nu este trimisă frecvent direct către clasa 0. Această observație este importantă deoarece susține trecerea la un sistem în două faze: mai întâi o decizie binară sănătos/patologic, apoi o clasificare mai fină doar pentru imaginile patologice. Matricele de confuzie indică tipul erorilor, nu doar numărul lor. În această figură sunt păstrate doar valorile absolute, pentru a evidenția direct confuziile dintre clasele reale și cele prezise. În rulările actualizate, APTOS rămâne cel mai echilibrat: în varianta cu Ben Graham, clasa 2 are 83 predicții corecte din 100, iar clasa 1 ajunge la 21 din 37. Pe EyePACS și Balanced_Aug se observă în continuare absorbția claselor 1 și 2 în clasa 0, ceea ce explică diferența dintre acuratețea globală și scorul F1.

Matrice de confuzie absoluta - ResNet50 AdamW

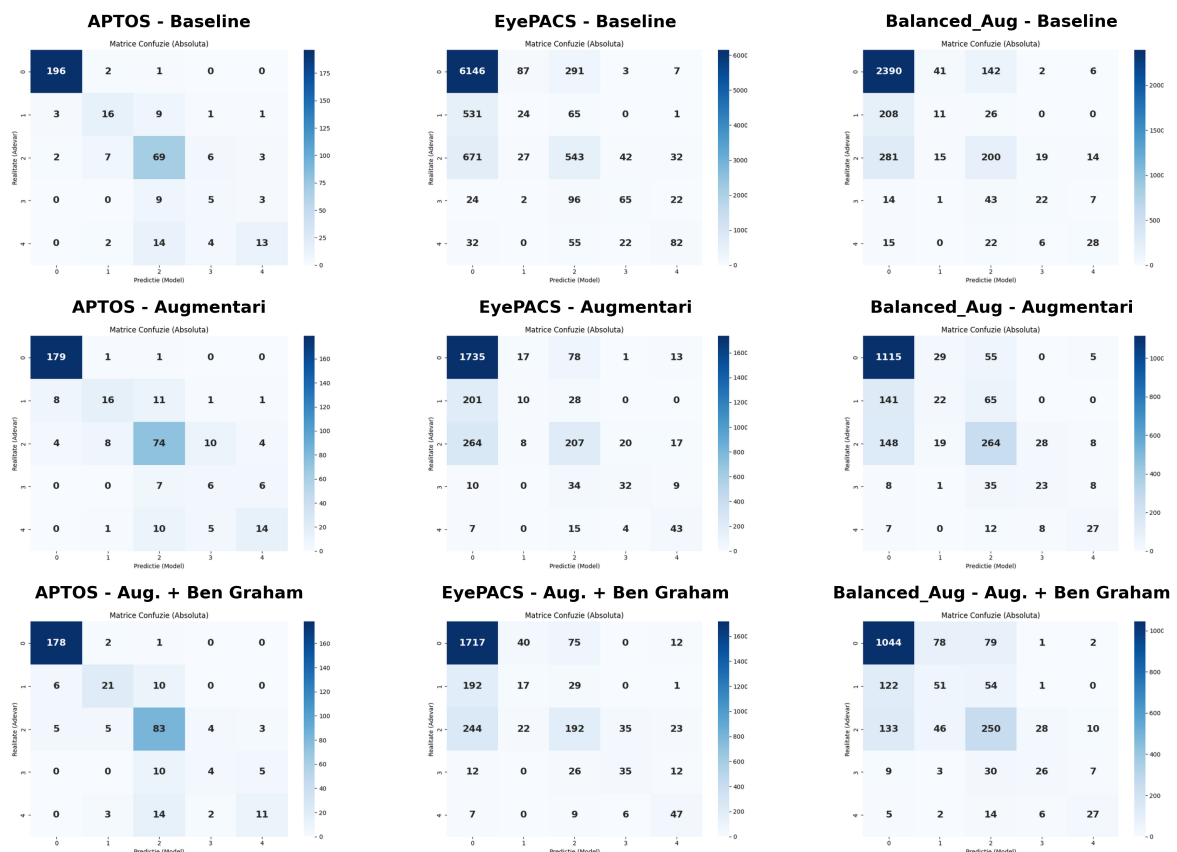


Figura 6.10: Matricele de confuzie absolute pentru ResNet50 cu AdamW pe cele trei dataseturi și cele trei configurații.

6.10 Curbele ROC multiclasă

Figura 6.11 prezintă curbele ROC multiclasă. Acestea sunt utile pentru a observa capacitatea modelului de a separa fiecare clasă în raport cu celelalte, independent de un prag fix de decizie.

În context multclasă, curbele ROC completează analiza pe clase și arată dacă modelul produce scoruri probabilistice coerente, chiar și atunci când predicția finală este greșită.

Compararea pe coloane indică diferențe între domeniile vizuale ale seturilor de date. APTOS, EyePACS și Balanced_Aug nu au aceeași distribuție a imaginilor, iar curbele ROC reflectă această variație. Compararea pe rânduri arată efectul etapelor de lucru asupra datelor: augmentările și preprocesările pot schimba separabilitatea claselor, dar aceste lucruri trebuie verificate pentru fiecare dataset.

Curbele ROC multclasă arată capacitatea modelului de a separa fiecare clasă față de celelalte. Ele completează matricele de confuzie deoarece analizează scorurile probabilistice, nu doar predicția finală.

Curbe ROC multclasă - ResNet50 AdamW

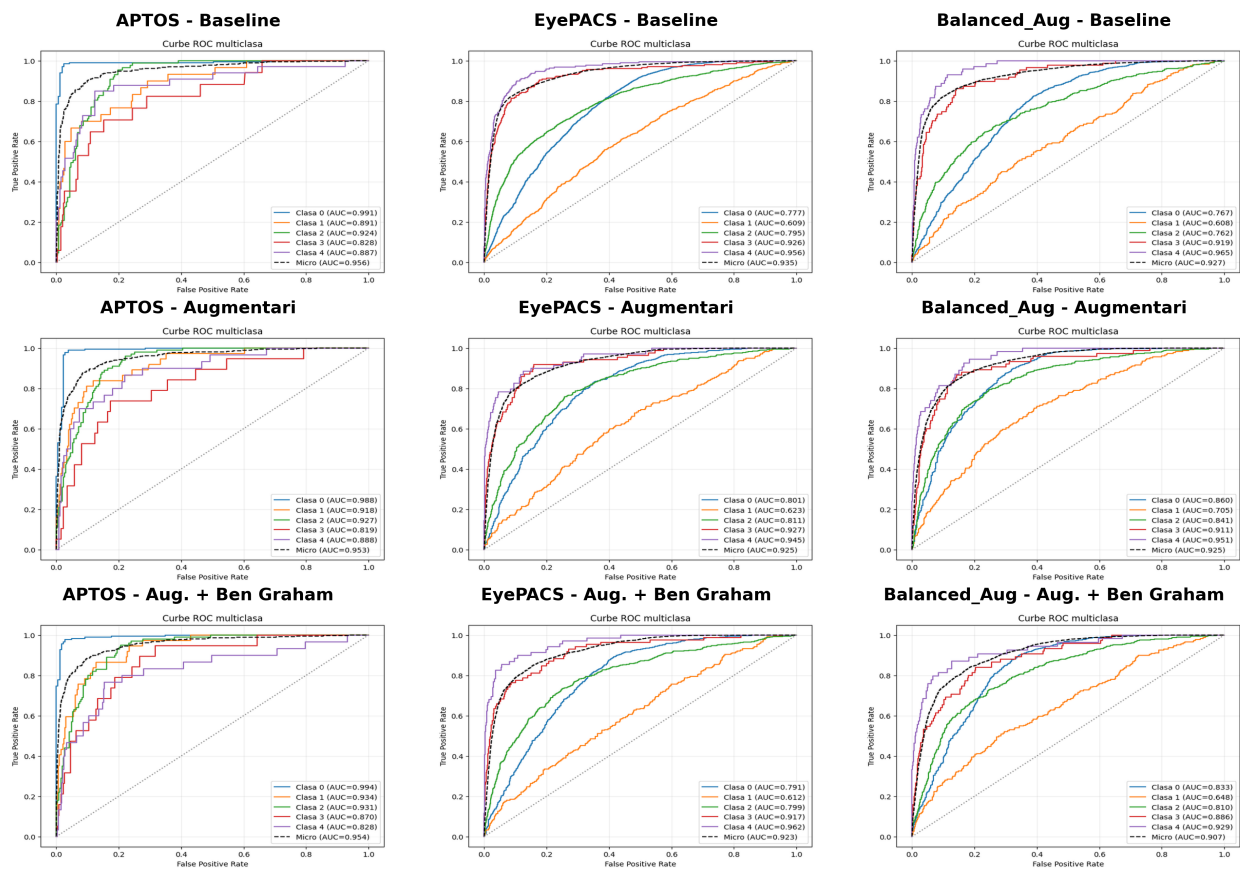


Figura 6.11: Curbele ROC multclasă pentru cele nouă experimente ResNet50 cu AdamW din `experiments.ipynb`.

Din figura 6.11 se observă că performanța nu trebuie interpretată doar prin poziția generală a curbelor, ci și prin diferențele dintre clase. Curbele apropiate de colțul stânga-sus indică o separare bună între clasa analizată și restul claselor, ceea ce sugerează că modelul atribuie scoruri probabilistice mai sigure. În general, clasele extreme sunt mai ușor de separat: imaginile fără retinopatie au aspect diferit față de imaginile patologice avansate, iar clasele severe conțin leziuni mai vizibile. În schimb, clasele intermediare au curbe mai puțin ideale, deoarece diferențele vizuale dintre formele ușoare și moderate sunt subtile și pot depinde mult de iluminare, calitatea imaginii și preprocesare.

Comparativ cu matricele de confuzie, curbele ROC oferă o perspectivă mai fină asupra încrederii modelului. O matrice de confuzie arată eticheta finală aleasă, în timp ce ROC arată dacă modelul a acordat totuși scoruri rezonabile clasei corecte înainte de aplicarea pragului sau a deciziei finale. Din acest motiv, o curbă ROC bună poate indica faptul că modelul are informație utilă în scorurile sale, chiar dacă unele predicții finale rămân greșite. Această observație susține și abordarea în două etape: prima etapă poate fi optimizată pentru separarea robustă între sănătos și patologic, iar a doua etapă poate trata separat diferențele mai dificile dintre clasele 1–4.

6.11 Integrarea cu sistemul final în două etape

Varianta finală a experimentului este implementată în notebook-ul `Notebooks/Main/Local/diabetic-retinopathy-classifier-FIXED-3ramuri.ipynb`, iar rezultatele exportate sunt grupate în directorul `Notebooks/Main/Local/Rezultate_main`. Spre deosebire de experimentele comparative anterioare, această rulare urmărește direct configurația folosită în sistemul final: un model `EfficientNet-B3` pentru decizia binară sănătos–patologic și un al doilea model `EfficientNet-B3` pentru gradarea severității pe clasele 1–4. Separarea celor două sarcini reduce dificultatea problemei multiclase, deoarece clasa sănătoasă nu mai concurează direct cu stadiile patologice în același clasificator.

Prima etapă tratează problema ca o clasificare binară între imaginile *sănătoase* și imaginile cu retinopatie diabetică. Criteriul de selecție este AUC pe validare, deoarece în această etapă este important ca modelul să ordoneze cât mai bine imaginile sănătoase față de cele patologice, înainte de alegerea pragului final. În ultima rulare, cel mai bun AUC pe validare este 0.8634, iar pe setul de test modelul obține un AUC de 0.8567. Pragul operațional este ales pe setul de validare prin F_2 , ceea ce păstrează accentul pe sensibilitatea față de clasa patologică. Pe setul de test, raportul actualizat indică o acuratețe de aproximativ 0.771, precision de 0.764 și recall de 0.784 pentru clasa *Bolnav*. Față de rularea anterioară, modelul binar devine mai echilibrat: reduce numărul de false pozitive, dar menține suficient de multe cazuri patologice trimise către etapa de gradare.

A doua etapă este aplicată imaginilor trimise mai departe de modelul binar și clasifică severitatea în stadiile 1–4. Pentru această etapă a fost păstrată formularea de tip *regression*, deoarece scorul continuu este transformat în clase prin praguri optimizate pe validare, iar metrica principală devine Quadratic Weighted Kappa. Pragurile obținute pe validare sunt aproximativ 0.580, 1.393 și 2.158. Modelul atinge un Val QWK de 0.6575, iar pe test obține accuracy 0.6170 și QWK 0.6576. Scorurile macro pentru etapa de severitate sunt precision 0.577, recall 0.597 și F1 0.572, ceea ce confirmă că problema rămâne dificilă mai ales pentru stadiile intermediare.

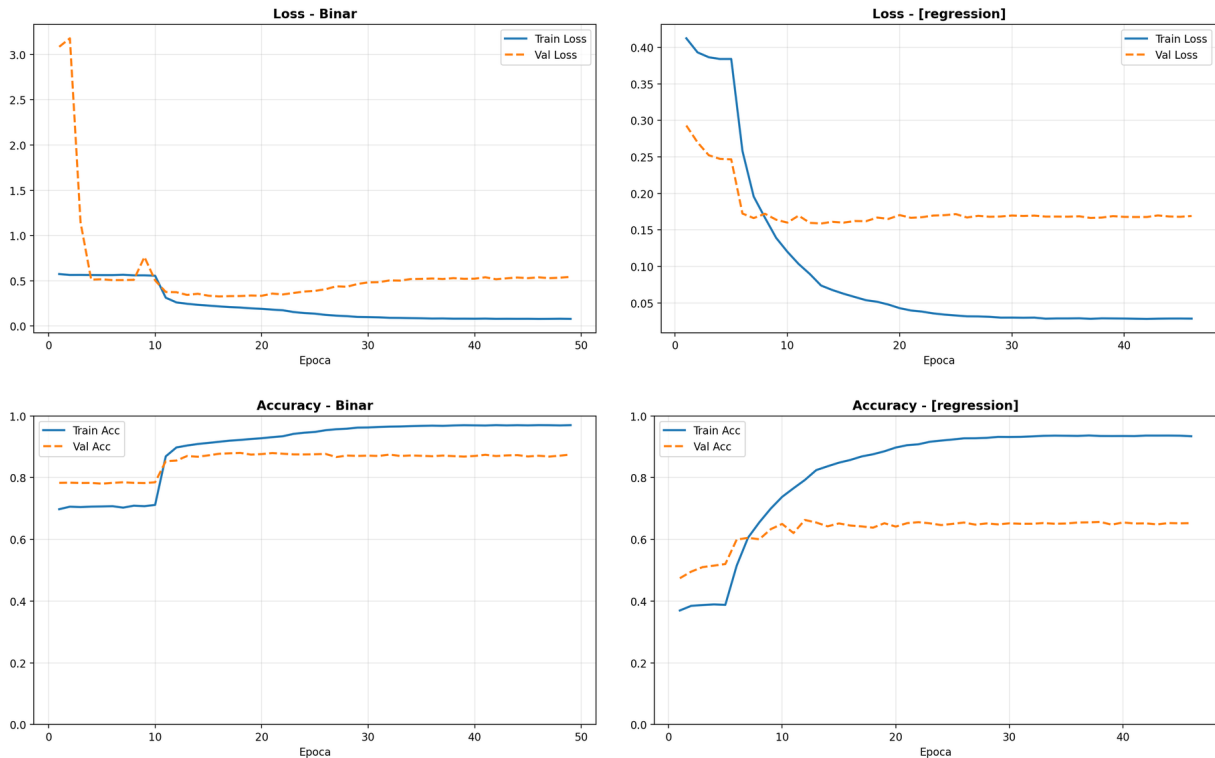


Figura 6.12: Curbele de pierdere și acuratețe pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.

Figura 6.12 reunește evoluția pierderii și a acurateții pentru cele două componente ale sistemului final. Pentru detectorul binar, pierderea pe validare este cea care scade brusc la început, după care se stabilizează și fluctuează în jurul unor valori mai mici. Pierderea pe antrenare are o evoluție mai graduală, ceea ce indică faptul că modelul continuă să ajusteze separarea dintre imaginile sănătoase și cele patologice. Acest comportament este frecvent în setările de screening, unde imaginile din validare pot avea calitate, iluminare și leziuni diferite față de cele din antrenare. De aceea, selecția modelului nu se bazează doar pe valoarea pierderii, ci pe capacitatea scorurilor de a separa cele două clase.

În partea de acuratețe, detectorul binar ajunge relativ repede într-o zonă stabilă pe validare. Acuratețea rămâne însă o metrică secundară pentru această componentă, deoarece pragul final este ales pentru a favoriza sensibilitatea față de cazurile patologice. Astfel, o acuratețe de test de aproximativ 0.771 trebuie interpretată împreună cu recall-ul de 0.784 pentru clasa *Bolnav*: modelul acceptă în continuare o parte dintre falsele pozitive, dar separarea este mai echilibrată decât în rularea precedentă.

Pentru modelul de severitate, pierderea pe validare scade mai brusc decât pierderea pe antrenare și se stabilizează relativ repede, în jurul epocilor 15–20. Pierderea pe antrenare continuă să scadă mai mult timp și se stabilizează mai târziu, aproximativ în jurul epocii 30. Acest lucru arată că modelul continuă să ajusteze reprezentările pe setul de antrenare chiar și după ce performanța pe validare a ajuns într-o zonă stabilă. Acuratețea de validare se stabilizează în zona 0.64–0.66, ceea ce indică faptul că modelul învață reprezentări utile, dar rămâne limitat de suprapunerea vizuală dintre stadiile patologice. Diferența dintre stadiile 1 și 2 sau dintre stadiile 3 și 4 poate fi subtilă, iar imaginile pot conține semne locale care nu domină întreaga fotografie de fund de ochi.

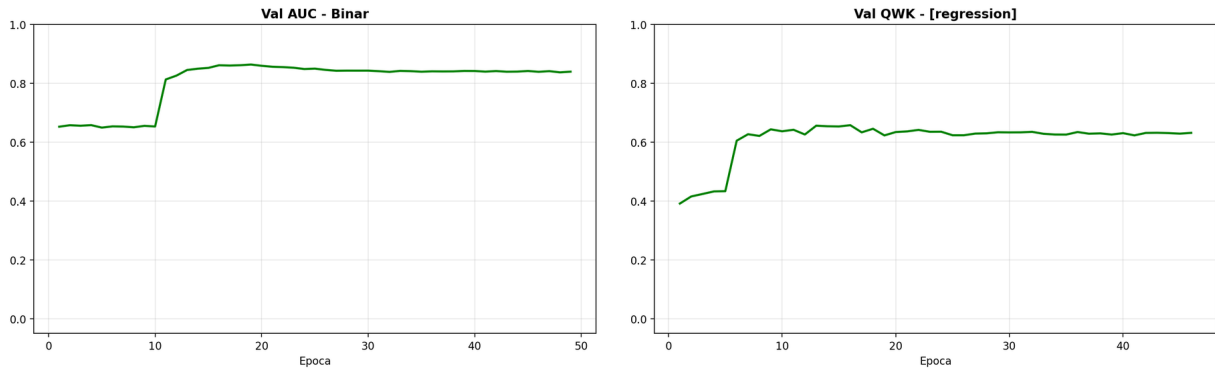


Figura 6.13: Evoluția AUC pentru detectorul binar și a QWK pentru modelul de severitate 1–4 prin regresie.

Figura 6.13 pune alături metricile principale de monitorizare pentru cele două etape. Curba AUC este cea mai relevantă pentru detectorul binar, deoarece nu depinde de un prag fix. Valoarea maximă de 0.8634 pe validare și valoarea de 0.8567 pe test arată că modelul păstrează o capacitate bună de separare și în afara setului folosit pentru alegerea checkpoint-ului. Diferența mică dintre validare și test sugerează că performanța nu este rezultatul unei ajustări accidentale pe validare, deși valorile obținute arată și limitele problemei: există în continuare imagini sănătoase cu aspect suspect și imagini patologice cu semne discrete, greu de separat automat.

Pentru etapa de severitate, QWK este mai informativă decât acuratețea simplă, deoarece ține cont de ordinea clinică a claselor. Pe test, modelul obține accuracy 0.6170 și QWK 0.6576, ceea ce arată că o parte dintre erori rămân între clase apropiate și nu reprezintă confuzii complet arbitrare. Această interpretare este importantă în formularea prin regresie, unde scorul continuu este transformat în clase prin praguri optimizate pe validare.

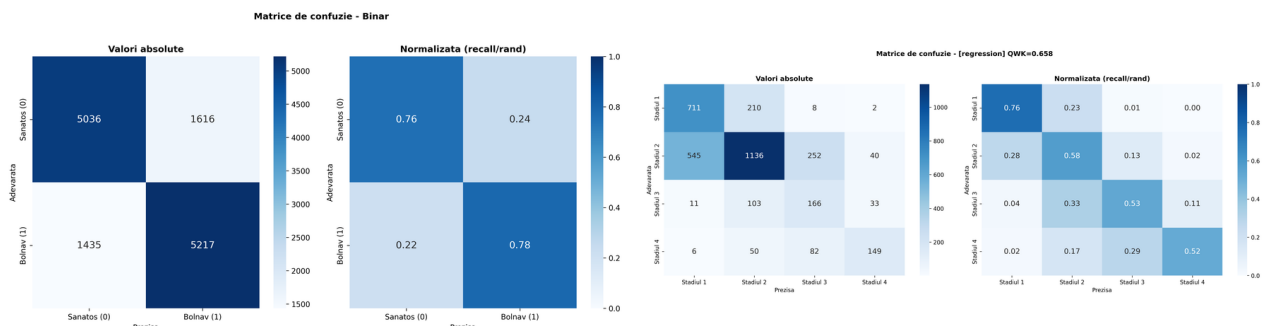


Figura 6.14: Matrici de confuzie absolute pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.

Matricile de confuzie din figura 6.14 evidențiază explicit comportamentul actualizat al detectorului binar. Modelul identifică 5217 imagini patologice din 6652, ceea ce corespunde unui recall de aproximativ 0.784 pentru clasa *Bolnav*. În același timp, apar 1616 false pozitive, adică imagini sănătoase trimise mai departe către modelul de severitate. Comparativ cu varianta anterioară, matricea indică un compromis mai bun între sensibilitate și specificitate: numărul de imagini sănătoase direcționate greșit către etapa a doua scade, iar detectorul păstrează o

capacitate bună de a reține cazurile patologice. Falsele pozitive nu pot fi corectate automat de etapa a doua, deoarece modelul de severitate primește doar clase patologice și va atribui inevitabil o clasă din intervalul 1–4.

Pentru etapa 1–4, cele mai multe predicții corecte apar pentru stadiile 1 și 2, unde există și cel mai mare suport în test. Confuzia dintre stadiile 1 și 2 rămâne principala sursă de eroare, iar stadiile 3 și 4 sunt afectate de numărul mai redus de exemple și de proximitatea vizuală a leziunilor. În contextul sistemului final, etapa de severitate trebuie interpretată împreună cu decizia binară: ea poate diferenția gradele de boală pentru imaginile suspecte, dar nu mai are o clasă 0 prin care să respingă un caz sănătos trimis greșit mai departe.

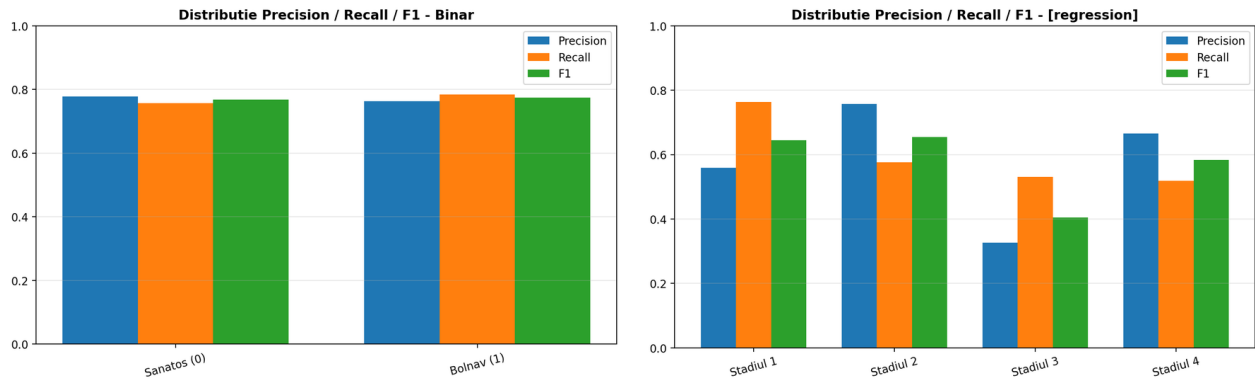


Figura 6.15: Distribuția indicatorilor precision, recall și F1 pentru detectorul binar și pentru modelul de severitate 1–4 prin regresie.

Distribuția precision–recall–F1 din figura 6.15 completează interpretarea matricilor de confuzie. Pentru detectorul binar, scorurile celor două clase sunt apropiate, cu valori F1 în jurul lui 0.77. Clasa patologică păstrează un recall ușor mai mare decât precision-ul, ceea ce este potrivit pentru rolul de prim filtru al pipeline-ului. În ansamblu, modelul binar nu este optimizat pentru o decizie izolată de diagnostic, ci pentru separarea inițială a imaginilor fără boală de cele care trebuie analizate mai detaliat de clasificatorul 1–4.

Pentru etapa de severitate, indicatorii arată un comportament neuniform între clase. Clasa *Stadiul 2* are precision ridicat, dar recall mai redus, deoarece o parte dintre imaginile reale de stadiu 2 sunt absorbite de stadiul 1. Clasa *Stadiul 3* rămâne cea mai dificilă, cu F1 de aproximativ 0.404, în timp ce stadiile extreme au un comportament mai stabil. Această distribuție confirmă că dificultatea principală a etapei a doua nu este detectarea bolii, ci delimitarea exactă a severității.

Integrarea finală se bazează, așadar, pe o decizie secvențială. Imaginea este preprocesată și trimisă mai întâi către modelul binar. Dacă probabilitatea de boală este sub pragul stabilit pe validare, sistemul returnează clasa 0. Dacă probabilitatea depășește pragul, imaginea este transmisă către modelul de severitate, iar predicția este mapată în scala 1–4. Această arhitectură permite optimizarea separată a sensibilității pentru screening și a respectării ordinii clinice pentru stadiile patologice.

6.12 Analiza calitativă a exemplelor de inferență

Înainte de interpretarea exemplelor, este important de precizat cum este calculată valoarea afișată în aplicația de inferență ca nivel de încredere, deoarece semnificația ei depinde de modelul

selectat. Pentru modelele experimentale multclasă, rețeaua produce cinci scoruri, câte unul pentru fiecare clasă 0–4. Aceste scoruri sunt transformate în probabilități prin funcția *softmax*, iar clasa finală este clasa cu probabilitatea maximă:

$$p_i = \frac{e^{z_i}}{\sum_{j=0}^4 e^{z_j}}, \quad \hat{y} = \arg \max_i p_i.$$

În acest caz, valoarea afișată în interfață este probabilitatea softmax a clasei prezise, iar a doua opțiune este următoarea clasă cu probabilitate mare.

Pentru pipeline-ul principal, valoarea afișată este calculată diferit, deoarece decizia este secvențială. Prima etapă este detectorul binar, care produce un scor transformat prin sigmoid în probabilitatea de boală:

$$p_{\text{boală}} = \sigma(z), \quad p_{\text{sănătos}} = 1 - p_{\text{boală}}.$$

Dacă detectorul decide clasa 0, aplicația afișează ca nivel de încredere valoarea $p_{\text{sănătos}}$. Dacă imaginea este trimisă către etapa a doua, modelul de severitate folosit în pipeline-ul principal produce un scor de regresie pentru clasele 1–4. Clasa este obținută prin compararea scorului cu pragurile salvate în `rounder_coef.npy`, iar nivelul de încredere afișat este o estimare derivată din apropierea scorului față de cele patru grade:

$$q_k = \frac{e^{-(k-s)^2}}{\sum_{r=0}^3 e^{-(r-s)^2}}, \quad k \in \{0, 1, 2, 3\},$$

unde s este scorul de regresie, iar q_k este apoi asociat claselor patologice 1–4. Prin urmare, pentru pipeline-ul principal, procentul afișat la etapa de severitate nu este o probabilitate softmax obținută direct din rețea, ci o estimare construită din scorul de regresie. În cardul *Metadata model* din aplicație, lista de probabilități afișată pentru clasele 1–4 este ponderată și cu probabilitatea de boală a detectorului binar, pentru a reflecta întregul traseu al pipeline-ului.

Pentru a completa evaluarea cantitativă, au fost analizate și câteva exemple calitative generate prin aplicația de inferență. Aceste exemple sunt utile deoarece arată nu doar clasa finală, ci și distribuția scorurilor între clasele vecine, respectiv situațiile în care modelul este sigur sau ambiguu. În continuare sunt prezentate patru cazuri reprezentative: trei rulări ale pipeline-ului principal și o rulare a unui model experimental multclasă.

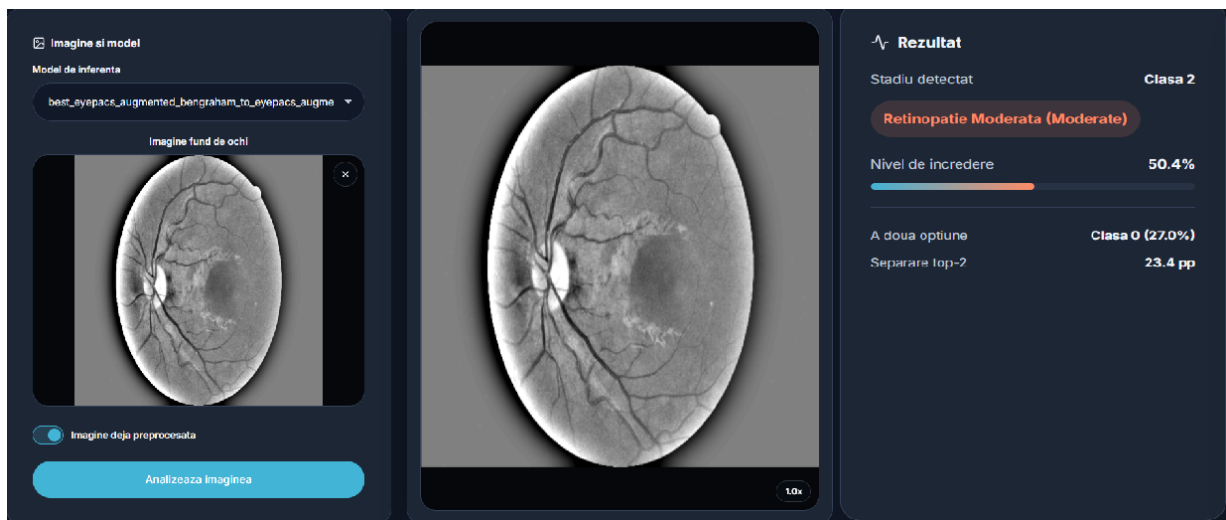


Figura 6.16: Model experimental: imagine de clasa 2, cu al doilea scor ca mărime la clasa 0.

Exemplul din figura 6.16 arată o limită importantă a clasificatoarelor multiclase antrenate direct pe toate clasele: deși imaginea aparține clasei 2, următorul scor ca mărime este atribuit clasei 0. Predicția principală rămâne astfel corectă, dar distribuția scorurilor arată că modelul păstrează o incertitudine importantă între un caz moderat și absența bolii. Această apropiere este relevantă deoarece o mică modificare a scorurilor sau o imagine cu o calitate mai slabă ar putea împinge decizia către clasa 0, generând un fals negativ. Exemplul evidențiază de ce un clasificator multiclase direct poate confunda uneori trăsăturile moderate cu aspectul normal al fundului de ochi și de ce este utilă separarea explicită dintre detecția bolii și gradarea severității.

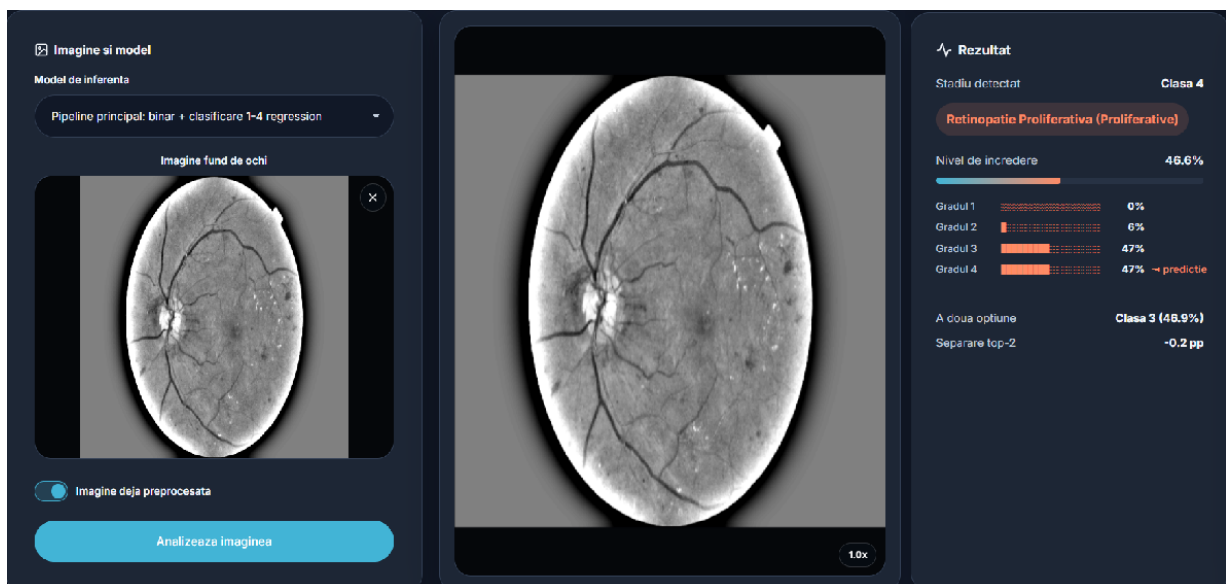


Figura 6.17: Pipeline principal: caz sever, cu scoruri apropiate între stadiile 3 și 4.

Figura 6.17 arată o limită a sistemului final: în cazurile severe, scorurile pentru stadiile 3 și 4 pot fi foarte apropiate. Această ambiguitate este așteptată, deoarece trecerea dintre retinopatia severă și cea proliferativă depinde de semne vizuale fine, de calitatea imaginii și de prezența unor leziuni care pot fi localizate doar într-o regiune redusă a fundului de ochi. Totuși, confuzia rămâne între clase vecine și nu schimbă concluzia principală că imaginea este

patologică și necesită evaluare medicală. Modelul semnalează corect severitatea ridicată, chiar dacă granularitatea exactă între stadiile 3 și 4 rămâne dificilă.

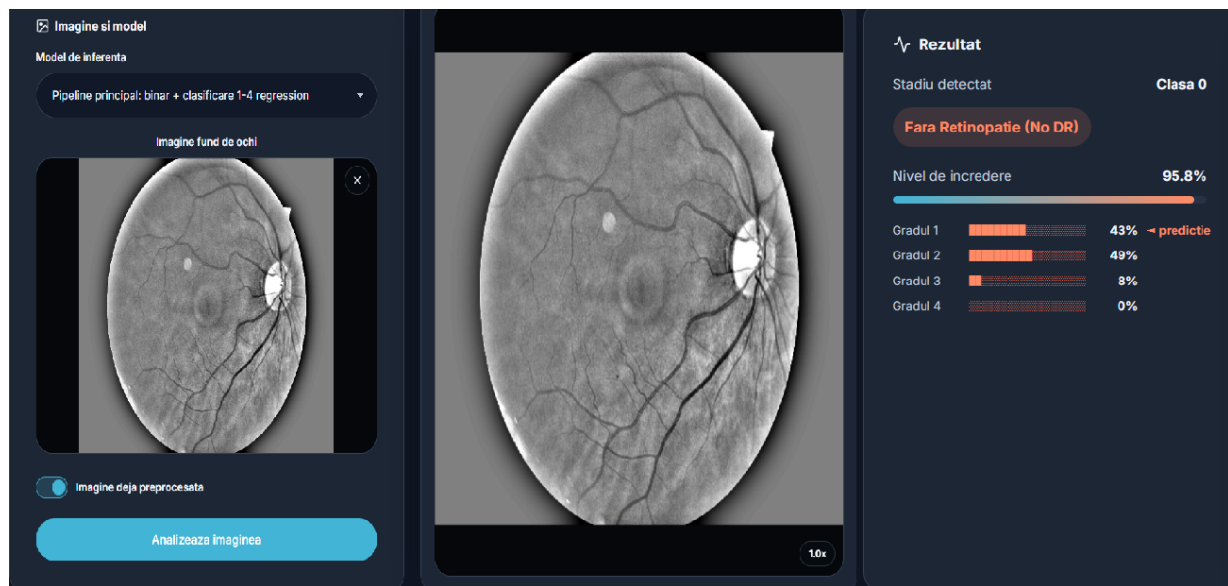


Figura 6.18: Pipeline principal: decizie binară corectă pentru clasa 0.

Exemplul din figura 6.18 evidențiază avantajul sistemului principal. Pentru imaginea normală, detectorul binar returnează clasa 0 cu încredere ridicată, iar distribuția estimată pentru gradele 1–4 rămâne doar o informație secundară a clasificatorului de severitate. Decizia finală nu este lăsată să fie dominată de o confuzie între grade patologice, deoarece prima etapă filtrează cazul ca nepatologic. Acesta este exact rolul sistemului în două etape: separarea robustă a imaginilor sănătoase înainte de problema mai dificilă a gradării.

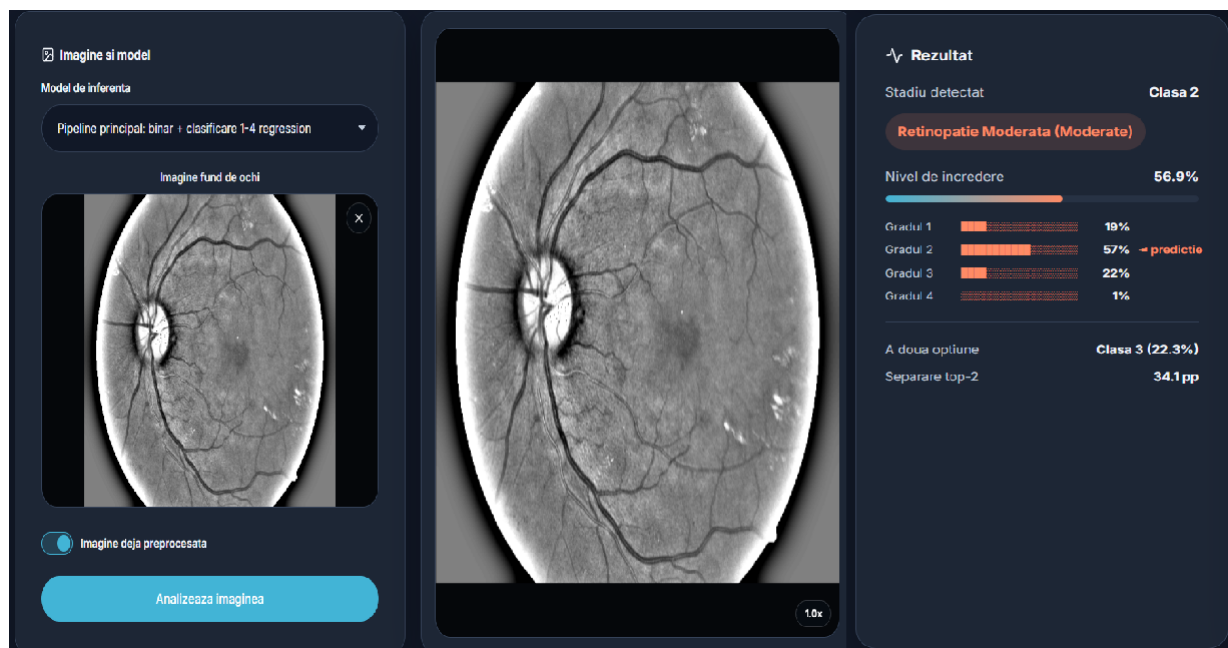


Figura 6.19: Pipeline principal: exemplu patologic încadrat în stadiul 2.

În figura 6.19, modelul principal indică un maxim clar pe gradul 2, iar următoarea opțiune este gradul 3, adică o clasă vecină. Acest comportament este util clinic, deoarece sistemul

identifică imaginea ca patologică și păstrează incertitudinea în interiorul stadiilor apropiate. O astfel de eroare potențială este mai acceptabilă decât o confuzie între sănătos și boală, deoarece păstrează ordinea clinică a severității. Din acest motiv, sistemul în două etape este preferabil unui singur clasificator cu cinci clase: prima etapă optimizează detecția bolii și reduce fals-negativele, iar a doua etapă se concentrează pe gradarea cazurilor deja suspecte, unde erorile pot fi interpretate în termeni de severitate ordinală.

Capitolul 7

Aplicația de inferență

7.1 Scopul aplicației dezvoltate

Pentru a demonstra utilizarea practică a modelelor antrenate, a fost dezvoltată o aplicație web de inferență. Scopul aplicației este de a permite încărcarea unei imagini de fund de ochi, selectarea unui model salvat sau a pipeline-ului principal și obținerea unei predicții privind stadiul retinopatiei diabetice. Aplicația are rol experimental și nu înlocuiește diagnosticul medical, deoarece sistemele automate pentru retinopatia diabetică necesită validare clinică riguroasă înainte de utilizare în screening sau decizie medicală [1], [21].

Rezultatul afișat include clasa prezisă, denumirea stadiului, nivelul de încredere, a doua opțiune atunci când aceasta poate fi calculată, separarea dintre primele două clase, timpul de execuție și recomandări orientative pentru medic și pacient. Interfața include și metadate tehnice păstrate la nivelul informațiilor utile pentru interpretare: fișierul folosit, arhitectura, dimensiunea de intrare, dimensiunea imaginii originale, dimensiunea imaginii folosite efectiv la inferență, metoda de preprocesare și, pentru pipeline-ul principal, detectorul binar, modelul de severitate și ramura utilizată.

7.2 Arhitectura aplicației

Aplicația este organizată în două componente principale: un backend Flask și un frontend React, două tehnologii potrivite pentru separarea logicii de inferență de interfața utilizatorului [35], [36]. Backend-ul este responsabil pentru încărcarea modelului, verificarea și preprocesarea imaginii, precum și calcularea predicției. Frontend-ul este responsabil pentru interfața de utilizare, selecția modelului, încărcarea imaginii, vizualizarea imaginii preprocesate și afișarea rezultatului.

Cele două componente comunică prin cereri HTTP. Backend-ul expune endpoint-uri sub forma `/api/models` și `/api/predict`, iar frontend-ul trimite cereri către acestea folosind biblioteca Axios [37]. Pentru comunicarea între porturi diferite, backend-ul folosește `flask_cors`, care permite configurarea politicilor CORS pentru aplicații Flask [38]. Endpoint-ul `/api/models` returnează atât lista modelelor, cât și grupurile folosite în selectorul din interfață, iar endpoint-ul `/api/predict` primește imaginea, identificatorul modelului și opțiunea prin care utilizatorul precizează dacă imaginea este deja preprocesată.

7.3 Componenta backend

Componenta backend este implementată în fișierul `Inference_site/app.py`. Aceasta definește aplicația Flask, stabilește directorul în care se află modelele salvate și selectează dispozitivul de calcul: CUDA, dacă este disponibil, sau CPU în caz contrar. Pentru reconstruirea arhitecturii modelului, backend-ul importă funcția `get_model` din `builder.py`.

Endpoint-ul `/api/models` returnează un catalog de modele, nu doar o listă simplă de fișiere. Catalogul este împărțit în două grupuri: *Modele principale*, care conține pipeline-ul final `binar + clasificare 1-4 regression`, și *Experimente*, care conține modelele salvate în directorul `Notebooks/Rezultate/New/modele`. Pentru soluția principală sunt folosite ponderile detectorului binar din `Notebooks/Main/Local/Rezultate_main/binar/efficientnet_b3_binar_best.pth` și ponderile modelului de severitate din `Notebooks/Main/Local/Rezultate_main/clasificare_regression/model_best.pth`, împreună cu pragurile de rotunjire salvate în `rounder_coef.npy`. Dacă unul dintre aceste fișiere lipsește, opțiunea principală este păstrată în catalog, dar este marcată ca indisponibilă.

Endpoint-ul `/api/predict` primește imaginea, identificatorul modelului selectat și câmpul `image_preprocessed`. Dacă utilizatorul marchează imaginea ca fiind deja preprocesată, backend-ul o folosește direct pentru inferență și reține acest lucru în răspunsul JSON. În caz contrar, imaginea este convertită în RGB și trecută printr-o preprocesare de tip Ben Graham cu mască de retină, la rezoluția 512×512 . Imaginea finală este apoi redimensionată la dimensiunea așteptată de model: 300×300 pentru pipeline-ul principal EfficientNet-B3, respectiv 224×224 sau 299×299 pentru modelele experimentale, în funcție de arhitectură. Tensorul rezultat este normalizat cu valorile ImageNet, iar predicția este calculată cu `torch.no_grad()`, pentru a dezactiva calculul gradientului în timpul inferenței [39].

Pentru opțiunea principală, backend-ul rulează o decizie secvențială. Prima etapă încarcă detectorul binar **EfficientNet-B3** și calculează probabilitatea ca imaginea să fie patologică. Dacă această probabilitate nu depășește pragul operațional al aplicației, sistemul returnează clasa 0. Dacă imaginea este considerată patologică, rezultatul final este dat de modelul de regresie pentru clasele 1-4: scorul continuu este mapat prin pragurile învățate pe validare, iar clasa rezultată este translatată în scala finală 1-4. În implementarea curentă, etapa a doua este evaluată și pentru a putea furniza metadate despre scorul de severitate, însă predicția finală folosește ramura 1-4 doar atunci când detectorul binar trece pragul. Răspunsul JSON include detectorul folosit, modelul de etapă a doua, probabilitatea de boală, scorul de regresie și detaliile de preprocesare.

7.4 Componenta frontend

Componenta frontend este implementată în `Inference_site/App.jsx`. La pornire, aceasta solicită catalogul de modele disponibile și selectează automat prima opțiune validă. Selectorul este organizat pe grupuri, astfel încât utilizatorul poate alege între pipeline-ul principal și modelele experimentale din directorul de rezultate. În versiunea actuală, opțiunea principală este afișată explicit ca **Pipeline principal: binar + clasificare 1-4 regression**, iar modelele experimentale sunt listate separat, cu numele fișierelor `.pth` generate în etapa de antrenare. Dacă pipeline-ul principal nu poate fi încărcat din cauza unui fișier lipsă, opțiunea apare ca indisponibilă în selector.

După alegerea modelului, utilizatorul poate încărca imaginea prin apăsare sau drag-and-drop. Interfața afișează previzualizarea imaginii originale și oferă un comutator numit *Imagine deja preprocesată*. Acest comutator controlează dacă backend-ul aplică sau nu preprocesarea Ben Graham înainte de inferență. În partea dreaptă, aplicația afișează imaginea folosită efectiv de model; aceasta poate fi mărită cu roțița mouse-ului, deplasată prin drag și resetată prin dublu click. Pentru compararea ergonomiei vizuale, aplicația include și un comutator pentru modul light/dark.

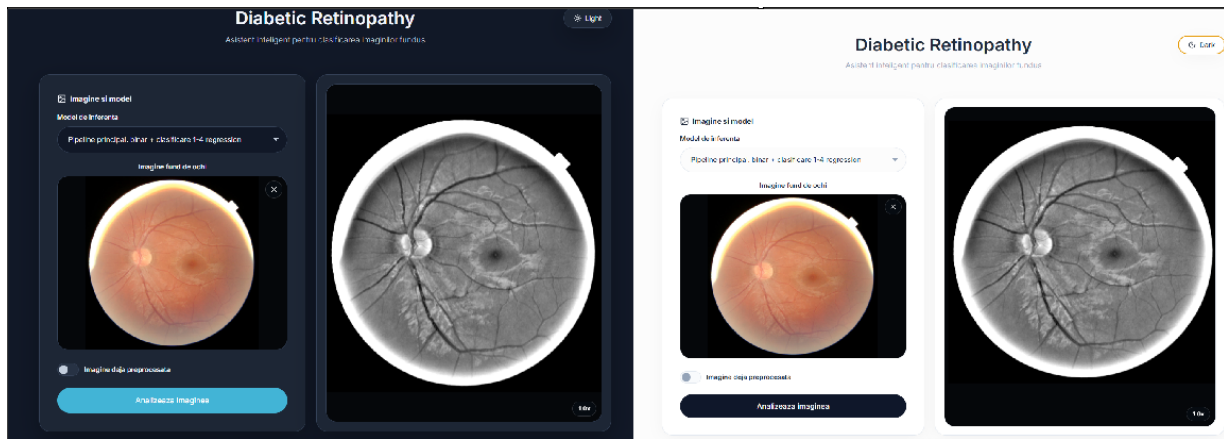


Figura 7.1: Interfața aplicației de inferență în modurile dark și light, cu selectorul de model, previzualizarea imaginii și imaginea folosită la inferență

După apăsarea butonului de analiză, frontend-ul construiește un obiect **FormData** care conține imaginea, identificatorul modelului și starea comutatorului de preprocesare. Răspunsul primit de la backend este afișat în trei carduri: rezultatul, recomandările și metadatele modelului. Cardul de rezultat conține clasa detectată, denumirea clinică a stadiului, nivelul de încredere, a doua opțiune și diferența procentuală dintre primele două opțiuni atunci când acestea sunt disponibile. Pentru pipeline-ul principal, cardul poate afișa și distribuția estimată pe gradele 1–4, derivată din scorul de regresie. Cardul de recomandări separă mesajul pentru medic de mesajul pentru pacient și include o observație tehnică despre aplicarea preprocesării. Cardul de metadate arată probabilitățile sau scorurile pipeline-ului, fișierele folosite, arhitectura, dimensiunile imaginilor, timpul de execuție, metoda de preprocesare și ramura de clasificare.

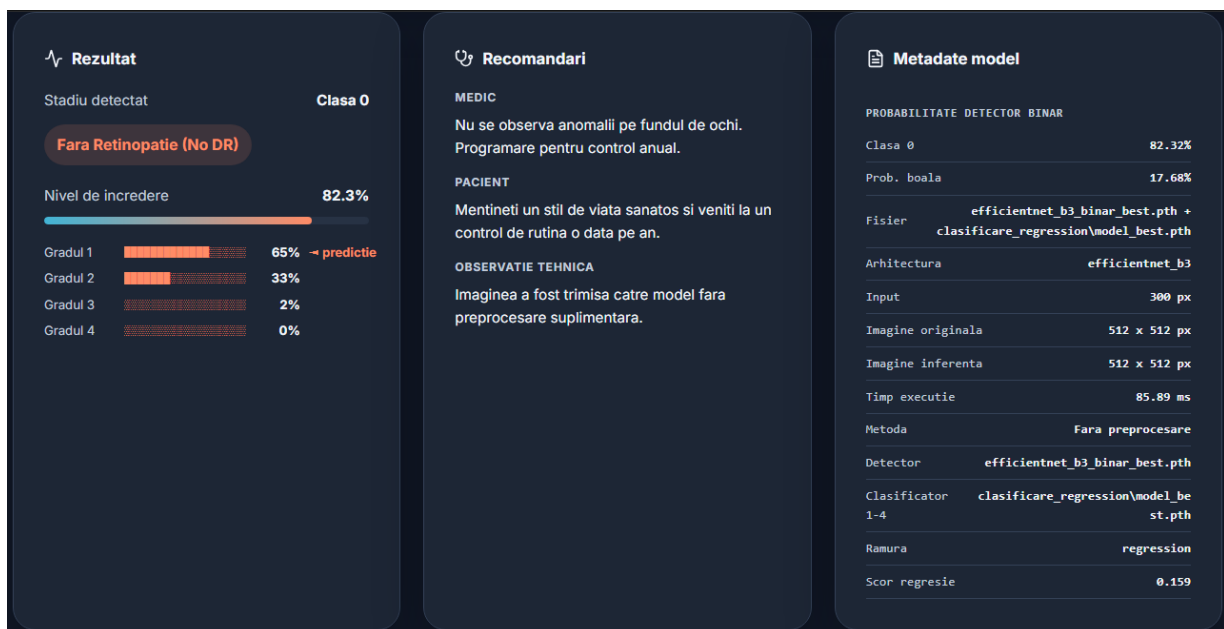


Figura 7.2: Cardurile de rezultat, recomandări și metadate tehnice afișate după rularea pipeline-ului principal

7.5 Fluxul de utilizare

Fluxul de utilizare este simplu. Utilizatorul pornește backend-ul, deschide frontend-ul, selectează modelul dorit, încarcă o imagine de fund de ochi, stabilește dacă imaginea este deja preprocesată și apasă butonul de analiză. Imaginea este trimisă către server, unde este analizată conform opțiunii selectate. Rezultatul este apoi returnat în format JSON și afișat în interfața web.

Această structură permite testarea rapidă a mai multor modele pe aceeași imagine. De exemplu, o imagine poate fi analizată mai întâi cu pipeline-ul principal în două etape, apoi cu modele experimentale ResNet50, EfficientNet-B3, InceptionV3 sau Inception-ResNet-V2. Diferențele dintre scoruri pot oferi indicii despre stabilitatea predicției și despre sensibilitatea arhitecturilor la aceeași intrare. Compararea trebuie însă făcută cu prudență, deoarece modelele experimentale afișează probabilități softmax multiclasă, în timp ce pipeline-ul principal combină probabilitatea detectorului binar cu o estimare derivată din scorul de regresie al modelului 1–4. Selectorul de modele păstrează modelele principale separate de experimente, astfel încât scenariul final al lucrării rămâne ușor de identificat.

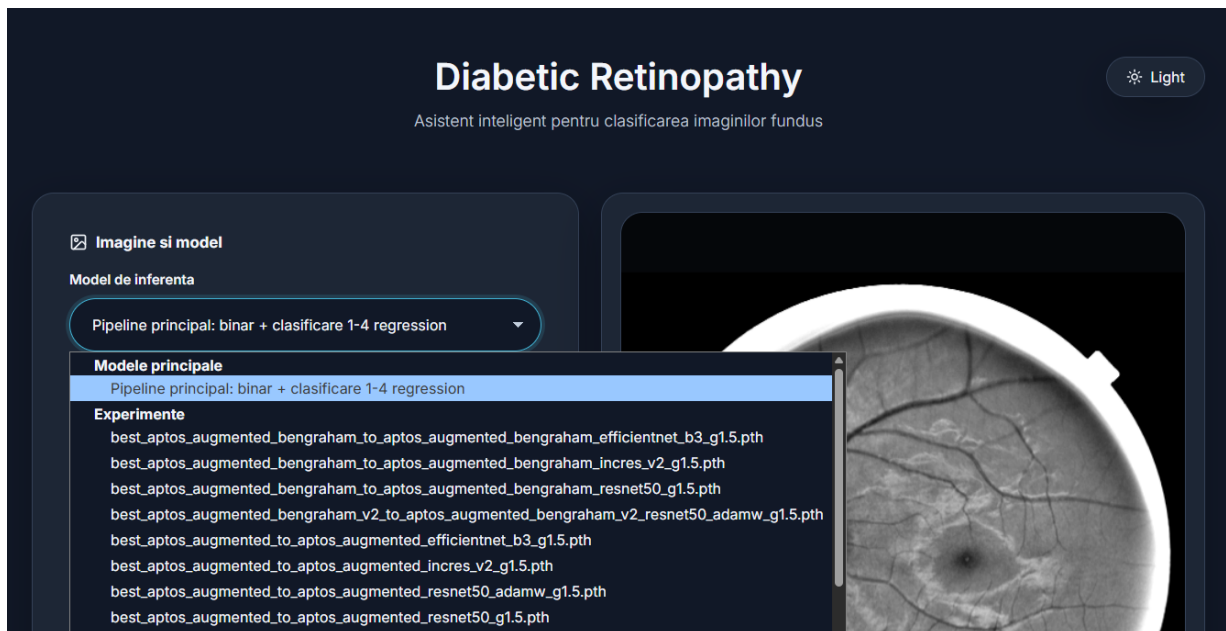


Figura 7.3: Meniul dropdown pentru selecția pipeline-ului principal și a modelelor experimentale

7.6 Integrarea modelului antrenat

Integrarea modelului antrenat diferă pentru cele două categorii din catalog. Pentru modelele experimentale, backend-ul reutilizează funcția `get_model` folosită în etapa de antrenare. Fișierul `.pth` salvează ponderile, nu întreaga logică a arhitecturii, deci modelul trebuie reconstruit înainte de încărcarea parametrilor. Backend-ul identifică arhitectura prin funcția `get_base_model_name`, analizând numele fișierului selectat.

Pentru pipeline-ul principal, arhitectura este reconstruită explicit în backend. Detectorul binar este un `EfficientNet-B3` cu un singur neuron de ieșire și funcție sigmoid, iar clasificatorul de severitate este un `EfficientNet-B3` cu ieșire scalară pentru regresie. Ponderile sunt

încărcate cu `torch.load`, folosind `map_location` pentru compatibilitate cu dispozitivul disponibil. Detectorul binar este încărcat din directorul `Rezultate_main/binar`, iar etapa de severitate din `Rezultate_main/clasificare_regression`. Pentru ramura de regresie, scorul continuu este transformat în clase prin `numpy.digitize` și pragurile salvate în `rounder_coef.npy`. În acest fel, aplicația poate rula atât scenariul final al lucrării, cât și modelele multiclasă experimentale.

7.7 Limitări practice ale aplicației

Aplicația are rol demonstrativ și prezintă mai multe limitări. În primul rând, verificarea preprocesării nu este echivalentă cu validarea medicală a imaginii și nu garantează că imaginea are o calitate suficientă pentru diagnostic. În al doilea rând, modelele sunt încărcate la fiecare cerere, ceea ce este simplu pentru testare, dar inefficient pentru producție. În cazul pipeline-ului principal, încărcarea poate implica două modele pentru a obține atât decizia binară, cât și informațiile despre severitate. Controlul calității imaginilor este o etapă importantă în evaluarea sistemelor pentru retinopatie diabetică, deoarece imaginile neclare, slab iluminate sau incorect centrate pot afecta predicția modelului [22].

Recomandările afișate sunt statice și orientative. Ele nu țin cont de istoricul pacientului, de acuitatea vizuală, de edemul macular, de tratamente existente sau de alte informații clinice. În plus, pragul operațional al detectorului binar din aplicație este o alegere tehnică pentru prototip și ar trebui calibrat riguros înainte de orice utilizare clinică. Într-o versiune reală, aplicația ar trebui să includă validare medicală, control al calității imaginilor, autentificare, audit, stocare securizată și un mecanism de marcarea a predicțiilor incerte. Integrarea într-un flux clinic ar trebui însoțită și de explicabilitate și analiză a erorilor, astfel încât deciziile modelului să poată fi interpretate de utilizatori umani [7].

Capitolul 8

Concluzii și direcții viitoare

8.1 Concluzii generale

Lucrarea a prezentat un sistem complet pentru clasificarea automată a retinopatiei diabetice pe baza imaginilor de fund de ochi. Au fost analizate fundamentele medicale ale bolii, stadiile de severitate, particularitățile imaginilor retiniene și direcțiile relevante din literatura de specialitate. Pe această bază a fost construit un pipeline experimental în PyTorch, folosind transfer learning, preprocesare dedicată imaginilor fundus și antrenare locală pe CUDA.

Metodologia include două componente complementare. Prima componentă este reprezentată de experimentele preliminare, prin care au fost comparate arhitecturi, funcții de pierdere, optimizatori și variante de augmentare. A doua componentă este sistemul propus, organizat în două etape: detecția binară a bolii și clasificarea severității pentru cazurile patologice. În plus, a fost realizată o aplicație web de inferență, care demonstrează modul în care modelele salvate pot fi utilizate într-o interfață practică.

8.2 Rezultatele obținute față de obiectivele inițiale

Obiectivele inițiale au fost îndeplinite în mai multe direcții. A fost realizată o analiză a metodelor existente pentru detecția și clasificarea retinopatiei diabetice, au fost studiate mai multe seturi de date publice, iar imaginile au fost reorganizate prin scripturi dedicate pentru split, curățare, augmentare și preprocesare. De asemenea, a fost implementată o metodă bazată pe rețele convoluționale și transfer learning, adaptată într-un sistem în două etape.

Codul permite testarea mai multor arhitecturi, funcții de pierdere și optimizatori. Au fost generate modele salvate, grafice de evaluare și rapoarte de clasificare. O contribuție importantă a lucrării este identificarea riscului de *data leakage* în utilizarea dataset-ului `Diabetic_Balanced_Data` și construirea unui split grupat care păstrează imaginea originală și augmentările ei în același subset. În plus, aplicația de inferență arată că rezultatele experimentale pot fi integrate într-un flux software accesibil, în care utilizatorul încarcă o imagine și primește o predicție.

8.3 Limitările lucrării

Principala limitare a sistemului propus este legată de propagarea erorilor în pipeline-ul în două etape. Prima etapă decide dacă imaginea este sănătoasă sau patologică, iar această decizie influențează direct interpretarea rezultatului final. Un fals negativ în etapa binară este critic, deoarece o imagine cu retinopatie diabetică poate fi oprită înainte de gradarea severității și poate fi raportată ca fiind fără boală. În sens invers, un fals pozitiv trimite o imagine sănătoasă către etapa de clasificare 1–4, unde aceasta poate primi un grad scăzut de boală. Acest compromis este acceptabil într-un context experimental orientat spre screening, deoarece

reducerea fals-negativelor este prioritară, dar ar necesita calibrare riguroasă și validare clinică înainte de utilizarea într-un scenariu real.

O altă limitare importantă este faptul că modelele au fost evaluate pe seturi de date publice, care nu reflectă complet variabilitatea clinică reală. În practică, imaginile pot proveni din camere diferite, centre medicale diferite, populații diferite și protocoale diferite de achiziție, iar distribuția claselor este de obicei dezechilibrată. Din acest motiv, performanța obținută pe un dataset public nu garantează automat aceeași performanță într-un program real de screening. De asemenea, generalizarea între dataset-uri rămâne dificilă, deoarece modelul poate învăța particularități ale sursei imaginilor, nu doar semnele patologice ale retinopatiei diabetice.

O altă limitare este faptul că unele seturi publice conțin imagini augmentate sau foarte asemănătoare între ele. În această lucrare, problema a fost investigată explicit pentru `Diabetic_Balanced_Data`, iar split-ul grupat reduce riscul ca originale și augmentări să ajungă în subseturi diferite. Totuși, pentru o validare și mai strictă, ar fi utilă o împărțire la nivel de pacient, astfel încât imagini apropiate provenite de la același pacient să nu apară simultan în antrenare și testare.

Calitatea imaginilor reprezintă, la rândul ei, o limitare. Preprocesarea Ben Graham reduce variațiile de iluminare și uniformizează parțial aspectul imaginilor, însă nu înlocuiește un modul clinic de control al calității. Imaginile neclare, prost centrate, supraexpuse, subexpuse sau afectate de artefacte pot influența predicția, mai ales în clasele apropiate vizual. Separarea dintre clasele 1, 2 și 3 rămâne dificilă, deoarece diferențele dintre stadii sunt uneori subtile chiar și pentru evaluarea umană.

Lucrarea nu include validare clinică realizată de medici oftalmologi și nu testează sistemul pe imagini provenite strict din mai multe centre medicale. Prin urmare, aplicația de inferență are rol demonstrativ și nu poate fi considerată un instrument medical. În forma actuală, aplicația este un prototip tehnic, fără autentificare, audit, stocare securizată a datelor, integrare cu sisteme clinice sau certificare medicală. În plus, explicațiile vizuale nu sunt integrate direct în fluxul aplicației, ceea ce limitează interpretabilitatea fiecărei predicții pentru un utilizator medical.

8.4 Direcții viitoare

O primă direcție viitoare este calibrarea explicită a pragului detectorului binar și analiza separată a fals-negativelor și fals-pozitivelor produse de pipeline-ul în două etape. Într-o versiune mai avansată, pragul nu ar trebui ales doar ca valoare operațională fixă, ci optimizat în funcție de scopul aplicației: sensibilitate ridicată pentru screening, specificitate mai mare pentru reducerea alarmelor false sau un compromis stabilit împreună cu specialiști clinici. De asemenea, cazurile cu probabilitate apropiată de prag ar putea fi marcate ca incerte și trimise automat către revizuire umană.

O direcție de modelare ar fi înlocuirea etapei de clasificare 1–4 cu un model multclasă 0–4, dar păstrând detectorul binar ca filtru inițial. În această variantă, imaginile pentru care detectorul binar indică foarte sigur clasa 0 ar putea fi eliminate din analiza suplimentară, iar restul imaginilor ar fi trimise către un clasificator complet 0–4. Această soluție ar permite modelului de severitate să învețe explicit și clasa sănătoasă, reducând riscul ca un fals pozitiv al detectorului binar să fie forțat automat într-una dintre clasele 1–4. În același timp, filtrarea cazurilor sigur sănătoase ar păstra avantajul sistemului în două etape, deoarece etapa multclasă s-ar concentra mai ales pe imaginile ambigue sau suspecte.

Pentru îmbunătățirea generalizării, modelele ar trebui antrenate și pe seturi externe de date, nu doar validate pe acestea. Integrarea unor imagini provenite din surse diferite ar expune rețeaua la camere, rezoluții, niveluri de iluminare și distribuții de clase mai variate. O astfel de strategie ar putea reduce dependența de particularitățile unui singur dataset și ar apropia procesul de antrenare de condițiile întâlnite într-un scenariu real.

O altă direcție este îmbunătățirea augmentărilor, astfel încât acestea să reproducă mai potrivit diversitatea imaginilor fundus. Pe lângă transformările geometrice și cromatice uzuale, ar putea fi introduse variații controlate de iluminare, contrast, claritate, câmp vizual, artefacte de achiziție și diferențe între camere. Scopul nu ar fi doar creșterea artificială a numărului de imagini, ci simularea unor condiții realiste care să ajute modelul să devină mai robust.

În aceeași direcție, poate fi explorată și generarea de imagini sintetice de fund de ochi pentru diferite grade de retinopatie diabetică, folosind modele generative de imagini. Această abordare ar putea fi utilă mai ales pentru clasele mai slab reprezentate, unde numărul de exemple reale este redus. Totuși, imaginile sintetice ar trebui folosite cu prudență și validate din punct de vedere vizual și medical, deoarece un generator poate introduce artefacte nerealiste sau leziuni care nu respectă distribuția clinică reală. Prin urmare, datele sintetice ar trebui privite ca o completare a dataset-urilor reale, nu ca un înlocuitor al acestora.

Pe partea de arhitecturi, pot fi explorate modele de ultimă generație, precum Vision Transformers (ViT) sau variante hibride CNN-Transformer. Acestea pot surprinde relații globale între regiuni îndepărtate ale imaginii, ceea ce poate fi util în analiza retinei, unde severitatea nu depinde întotdeauna de o singură leziune locală, ci de distribuția mai multor semne patologice. Compararea lor cu EfficientNet-B3, ResNet50 și Inception-ResNet-V2 ar permite evaluarea beneficiului real al acestor arhitecturi în raport cu costul computațional mai ridicat.

În completare, sistemul poate fi extins cu un modul de control al calității imaginii și cu metode de explicabilitate vizuală. Înainte de clasificare, aplicația ar putea verifica dacă fotografia este clară, bine iluminată și centrată, iar cazurile neconforme ar putea fi respinse sau marcate pentru recapturare. După predicție, integrarea unor hărți de activare sau a unei segmentări de leziuni ar face rezultatul mai ușor de interpretat. Aplicația de inferență poate fi, de asemenea, îmbunătățită prin păstrarea modelelor în memorie, definirea unor praguri de incertitudine și generarea unor rapoarte exportabile.

Bibliografie

- [1] Varun Gulshan, Lily Peng, Marc Coram, Martin C Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, Kasumi Widner, Tom Madams, Jorge Cuadros, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22):2402–2410, 2016.
- [2] Muhammad Kashif Jabbar, Jianzhuo Yan, Hongxia Xu, Zaka Ur Rehman, and Ayesha Jabbar. Transfer learning-based model for diabetic retinopathy diagnosis using retinal images. *Brain Sciences*, 12(5):535, 2022.
- [3] Xiaofei Wang, Mai Xu, Jicong Zhang, Lai Jiang, and Liu Li. Deep multi-task learning for diabetic retinopathy grading in fundus images. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 2826–2834, 2021.
- [4] Nitigya Sambyal, Poonam Saini, Rupali Syal, and Varun Gupta. Modified u-net architecture for semantic segmentation of diabetic retinopathy images. *Biocybernetics and Biomedical Engineering*, 40(3):1094–1109, 2020.
- [5] Shiqi Huang, Jianan Li, Yuze Xiao, Ning Shen, and Tingfa Xu. Rtnet: relation transformer network for diabetic retinopathy multi-lesion segmentation. *IEEE Transactions on Medical Imaging*, 41(6):1596–1607, 2022.
- [6] Hossein Shakibania, Sina Raoufi, Behnam Pourafkham, Hassan Khotanlou, and Muharram Mansoorizadeh. Dual branch deep learning network for detection and stage grading of diabetic retinopathy. *Biomedical Signal Processing and Control*, 93:106168, 2024.
- [7] Mohamed Chetoui and Moulay A Akhloufi. Explainable end-to-end deep learning for diabetic retinopathy detection across multiple datasets. *Journal of Medical Imaging*, 7(4):044503–044503, 2020.
- [8] Guanghua Zhang, Bin Sun, Zhaoxia Zhang, Jing Pan, Weihua Yang, and Yunfang Liu. Multi-model domain adaptation for diabetic retinopathy classification. *Frontiers in Physiology*, 13:918929, 2022.
- [9] Vitreum. Fund de ochi, 2026. Disponibil la: <https://vitreum.ro/dictionar-oftalmologic/fund-de-ochi/>.
- [10] Vitreum. Retinopatie diabetica, 2026. Disponibil la: <https://vitreum.ro/afectiuni/retinopatie-diabetica/>.
- [11] Master TAID. Retele convolutionale – convolutional neural networks (cnns), 2026. Disponibil la: https://www.master-taid.ro/Cursuri/MLAV_files/Retele%20Convolutionale%20-%20Convolutional%20Neural%20Networks%20%28CNNs%29.html.
- [12] ResearchGate. Block diagram of resnet-50 construction, 2026. Disponibil la: https://www.researchgate.net/figure/Block-diagram-of-ResNet-50-construction_fig1_390130860.

- [13] ResearchGate. Inceptionv3 architecture, 2026. Disponibil la: https://www.researchgate.net/figure/nceptionV3-architecture-2-Xception-Xception-20-means-the-extreme-version-of-Inception_fig2_338071209.
- [14] Jun Wang, Qianying Liu, Haotian Xie, Zhaogang Yang, and Hefeng Zhou. Boosted efficientnet: Detection of lymph node metastases in breast cancer using convolutional neural networks. *Cancers*, 13(4):661, 2021. Disponibil la: <https://www.mdpi.com/2072-6694/13/4/661>.
- [15] ResearchGate. Inception-resnet-v2 schematic diagram, 2026. Disponibil la: https://www.researchgate.net/figure/nception-ResNet-v2-schematic-diagram-25_fig3_342334669.
- [16] Joes Staal, Michael D Abràmoff, Meindert Niemeijer, Max A Viergever, and Bram Van Ginneken. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4):501–509, 2004.
- [17] Akara Sopharak, Bunyarit Uyyanonvara, and Sarah Barman. Automatic exudate detection from non-dilated diabetic retinopathy retinal images using fuzzy c-means clustering. *sensors*, 9(3):2148–2161, 2009.
- [18] Harry Pratt, Frans Coenen, Deborah M Broadbent, Simon P Harding, and Yalin Zheng. Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90:200–205, 2016.
- [19] American Diabetes Association Professional Practice Committee. 2. diagnosis and classification of diabetes: Standards of care in diabetes–2024. *Diabetes Care*, 47(Supplement_1):S20–S42, 2024.
- [20] Tien Yin Wong, Chui Ming Gemmy Cheung, Michael Larsen, Sobha Sharma, and Rafael Simó. Diabetic retinopathy. *Nature Reviews Disease Primers*, 2:16012, 2016.
- [21] Tao Li, Yingqi Gao, Kai Wang, Song Guo, Hanruo Liu, and Hong Kang. Diagnostic assessment of deep learning algorithms for diabetic retinopathy screening. *Information Sciences*, 501:511–522, 2019.
- [22] Ruhan Liu, Xiangning Wang, Qiang Wu, Ling Dai, Xi Fang, Tao Yan, Jaemin Son, Shiqi Tang, Jiang Li, Zijian Gao, et al. Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge. *Patterns*, 3(6), 2022.
- [23] Luca Giancardo, Fabrice Meriaudeau, Thomas P Karnowski, Yaqin Li, Seema Garg, Kenneth W Tobin Jr, and Edward Chaum. Exudate-based diabetic macular edema detection in fundus images using publicly available datasets. *Medical image analysis*, 16(1):216–226, 2012.
- [24] Meindert Niemeijer, Bram Van Ginneken, Joes Staal, Maria SA Suttorp-Schulten, and Michael D Abràmoff. Automatic detection of red lesions in digital color fundus photographs. *IEEE Transactions on medical imaging*, 24(5):584–592, 2005.
- [25] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [26] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.

- [27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [28] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2818–2826, 2016.
- [29] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6105–6114, 2019.
- [30] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [31] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.
- [32] Kushagra Tandon. Diabetic retinopathy balanced. Kaggle dataset, 2024. Disponibil la: <https://www.kaggle.com/datasets/kushagrandon12/diabetic-retinopathy-balanced>.
- [33] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.
- [34] Tomasz Krzywicki, Piotr Brona, Agnieszka M. Zbrzezny, and Andrzej E. Grzybowski. A global review of publicly available datasets containing fundus images: Characteristics, barriers to access, usability, and generalizability. *Journal of Clinical Medicine*, 12(10):3587, 2023. Disponibil la: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10219065/>.
- [35] Pallets Projects. Flask documentation, 2026. Disponibil la: <https://flask.palletsprojects.com/>.
- [36] Meta Open Source. React documentation, 2026. Disponibil la: <https://react.dev/>.
- [37] Axios Project. Axios documentation, 2026. Disponibil la: <https://axios-http.com/docs/intro>.
- [38] Flask-CORS Contributors. Flask-cors documentation, 2026. Disponibil la: <https://flask-cors.readthedocs.io/en/latest/>.
- [39] PyTorch Foundation. Pytorch documentation, 2026. Disponibil la: <https://docs.pytorch.org/docs/stable/>.

Anexa A

Cod

```

1 # =====
2 # Setup - foldere de rezultate (Rezultate_main/...)
3 # =====
4
5 import os, glob, shutil
6 from pathlib import Path
7
8 RESULTS_ROOT = Path("Rezultate_main")
9 DIR_BINAR = RESULTS_ROOT / "binar"
10 def dir_clasificare(mode): return RESULTS_ROOT / f"clasificare_{mode}"
11 for d in [DIR_BINAR] + [dir_clasificare(m) for m in ["crossentropy", "regression"]]:
12     d.mkdir(parents=True, exist_ok=True)
13
14 # Mutam rezultatele vechi (daca exista) in folderele dedicate
15 def _muta(pattern, dest):
16     for f in glob.glob(pattern):
17         try:
18             shutil.move(f, str(Path(dest) / os.path.basename(f)))
19             print(f" muta {f} -> {dest}")
20         except Exception as e:
21             print(f" (skip {f}: {e})")
22
23 _muta("efficientnet_b3_binar_best.pth", DIR_BINAR)
24 _muta("matrice_confuzie_binar.png", DIR_BINAR)
25 for m in ["crossentropy", "regression"]:
26     _muta(f"efficientnet_b3_multiclass_{m}.pth", dir_clasificare(m))
27     _muta(f"curbe_{m}.png", dir_clasificare(m))
28     _muta(f"matrice_confuzie_{m}.png", dir_clasificare(m))
29 print("Foldere de rezultate gata.")
30
31
32 # Functii de grafice (refolosite la binar si la clasificarea multi-clasa)
33 import matplotlib.pyplot as plt
34 import seaborn as sns
35 import numpy as np
36 from sklearn.metrics import classification_report, confusion_matrix
37
38 def plot_curbe(train_loss, val_loss, train_acc, val_acc, metric_vals, metric_name, out_dir,
39               titlu):
40     xs = range(1, len(train_loss) + 1)
41     # train loss + val loss
42     plt.figure(figsize=(8, 5))
43     plt.plot(xs, train_loss, label='Train Loss', lw=2)
44     plt.plot(xs, val_loss, label='Val Loss', lw=2, ls='--')
45     plt.title(f'Loss {titlu}', fontweight='bold'); plt.xlabel('Epoca'); plt.legend(); plt.grid(
46         alpha=.3)
47     plt.tight_layout(); plt.savefig(Path(out_dir) / 'curbe_loss.png', dpi=200, bbox_inches='
48         tight'); plt.show()
49     # train acc + val acc
50     plt.figure(figsize=(8, 5))
51     plt.plot(xs, train_acc, label='Train Acc', lw=2)
52     plt.plot(xs, val_acc, label='Val Acc', lw=2, ls='--')
53     plt.title(f'Accuracy {titlu}', fontweight='bold'); plt.xlabel('Epoca'); plt.ylim(0, 1);
54     plt.legend(); plt.grid(alpha=.3)
55     plt.tight_layout(); plt.savefig(Path(out_dir) / 'curbe_acc.png', dpi=200, bbox_inches='
56         tight'); plt.show()
57     # metrica (QWK sau AUC)
58     plt.figure(figsize=(8, 5))
59     plt.plot(xs, metric_vals, color='green', lw=2)
60     plt.title(f'Val {metric_name} {titlu}', fontweight='bold'); plt.xlabel('Epoca'); plt.ylim
61         (-0.05, 1); plt.grid(alpha=.3)
62     plt.tight_layout(); plt.savefig(Path(out_dir) / f'curba_{metric_name.lower()}.png', dpi
63         =200, bbox_inches='tight'); plt.show()

```

```

57
58 def plot_prf(y_true, y_pred, class_names, out_dir, titlu):
59     rep = classification_report(y_true, y_pred, target_names=class_names, labels=list(range(
60         len(class_names))), output_dict=True, zero_division=0)
61     prec = [rep[c]['precision'] for c in class_names]
62     rec = [rep[c]['recall'] for c in class_names]
63     f1 = [rep[c]['f1-score'] for c in class_names]
64     x = np.arange(len(class_names)); w = 0.25
65     plt.figure(figsize=(max(8, 1.8 * len(class_names)), 5))
66     plt.bar(x - w, prec, w, label='Precision')
67     plt.bar(x, rec, w, label='Recall')
68     plt.bar(x + w, f1, w, label='F1')
69     plt.xticks(x, class_names, rotation=15); plt.ylim(0, 1); plt.grid(axis='y', alpha=.3)
70     plt.title('Distributie Precision / Recall / F1 {titlu}', fontweight='bold'); plt.legend()
71     plt.tight_layout(); plt.savefig(Path(out_dir) / 'prf_bars.png', dpi=200, bbox_inches='
72         tight'); plt.show()
73
74 def plot_confuzie(y_true, y_pred, class_names, out_dir, titlu):
75     cm = confusion_matrix(y_true, y_pred, labels=list(range(len(class_names))))
76     fig, axes = plt.subplots(1, 2, figsize=(8 + 2 * len(class_names), 6))
77     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=
78         class_names,
79         ax=axes[0], annot_kws={"size": 13})
80     axes[0].set_title('Valori absolute', fontweight='bold'); axes[0].set_ylabel('Adevarata');
81     axes[0].set_xlabel('Prezisa')
82     cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True).clip(min=1)
83     sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', xticklabels=class_names,
84         yticklabels=class_names,
85         ax=axes[1], annot_kws={"size": 13}, vmin=0, vmax=1)
86     axes[1].set_title('Normalizata (recall/rand)', fontweight='bold'); axes[1].set_ylabel('
87         Adevarata'); axes[1].set_xlabel('Prezisa')
88     plt.suptitle(f'Matrice de confuzie {titlu}', fontweight='bold', y=1.02)
89     plt.tight_layout(); plt.savefig(Path(out_dir) / 'confusion.png', dpi=300, bbox_inches='
90         tight'); plt.show()
91
92 print("Funcții de grafice gata.")
93
94 # =====
95 # EfficientNet-B3 Binar - versiune corectata
96 # =====
97 #
98 # Ce s-a schimbat fata de varianta ta si de ce:
99 #
100 # 1. Antrenare in doua faze (FREEZE_BACKBONE_EPOCHS = 3). In faza 1 backbone-ul e inghetat si
101 # se antreneaza doar capul nou, cu LR mare (1e-3). In faza 2 se dezgheata tot, cu LR
102 # diferentiat: backbone 1e-4, cap 3e-4.
103 # 2. Augmentare activata (flip, rotatie, color jitter) - era complet comentata.
104 # 3. num_workers=0 - propriul tau comentariu avertiza ca pe Windows num_workers>0 ingheata la
105 # epoca 2.
106 # 4. AUC raportat si monitorizat - early stopping si checkpoint pe AUC de validare.
107 # 5. Prag de decizie ales pe validare (Youden's J) si aplicat consecvent pe test.
108 # 6. Etichete fortate la float in bucla.
109 # 7. Celula optionala de sanity-check.
110
111 import os
112 import numpy as np
113 import torch
114 import torch.nn as nn
115 import torch.optim as optim
116 from torchvision import models, transforms
117 from torch.utils.data import DataLoader, Subset
118 from tqdm import tqdm
119 from sklearn.metrics import classification_report, accuracy_score, roc_auc_score, roc_curve
120 import matplotlib.pyplot as plt
121
122 # Importam DOAR clasa de Dataset din fisierul nostru extern
123 from binary_dataset import BinaryRetinopathyDataset
124
125 print("=== INCARCARE DATE SI DATALOADERS ===")
126
127 DATA_DIR = r'B:\Projects\Disertatie\Diabetic-Retinopathy-Classifer\datasets\processed_by_me\
128     eyepacs\eyepacs_binar'
129
130 IMG_SIZE = 300 # rezolutia nativa a EfficientNet-B3
131 BATCH_SIZE = 32 # la 512: scade la 16/8 daca iei CUDA Out of Memory
132 USE_BEN_GRAHAM = False # vezi nota din primul cell inainte sa activezi

```



```

122 # --- Preprocesare optionala specifica de fundus (Ben Graham, EyePACS 2015) ---
123 class BenGrahamPreprocess:
124     def __init__(self, sigma_scale=10, crop_tol=7):
125         self.sigma_scale = sigma_scale
126         self.crop_tol = crop_tol
127
128     def _crop_to_circle(self, img):
129         import cv2
130         gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
131         mask = gray > self.crop_tol
132         if mask.sum() == 0:
133             return img
134         coords = np.argwhere(mask)
135         y0, x0 = coords.min(axis=0)
136         y1, x1 = coords.max(axis=0) + 1
137         return img[y0:y1, x0:x1]
138
139     def __call__(self, img):
140         import cv2
141         from PIL import Image
142         arr = np.array(img.convert("RGB"))
143         arr = self._crop_to_circle(arr)
144         s = max(arr.shape[0], arr.shape[1]) / self.sigma_scale
145         blurred = cv2.GaussianBlur(arr, (0, 0), s)
146         arr = cv2.addWeighted(arr, 4, blurred, -4, 128)
147         return Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8))
148
149
150 _pre = [BenGrahamPreprocess()] if USE_BEN_GRAHAM else []
151 _norm = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
152
153 # 1. Transformari - AUGMENTAREA E ACUM ACTIVA pe train
154 train_transform = transforms.Compose(_pre + [
155     transforms.Resize((IMG_SIZE, IMG_SIZE)),
156     transforms.RandomHorizontalFlip(p=0.5),
157     transforms.RandomVerticalFlip(p=0.5),
158     transforms.RandomRotation(15),
159     transforms.ColorJitter(brightness=0.1, contrast=0.1),
160     transforms.ToTensor(),
161     _norm,
162 ])
163
164 test_transform = transforms.Compose(_pre + [
165     transforms.Resize((IMG_SIZE, IMG_SIZE)),
166     transforms.ToTensor(),
167     _norm,
168 ])
169
170 # 2. Dataseturi
171 train_ds = BinaryRetinopathyDataset(os.path.join(DATA_DIR, 'train'), transform=train_transform)
172 val_ds = BinaryRetinopathyDataset(os.path.join(DATA_DIR, 'val'), transform=test_transform)
173 test_ds = BinaryRetinopathyDataset(os.path.join(DATA_DIR, 'test'), transform=test_transform)
174
175 # 3. DataLoadere - num_workers=0 pe Windows
176 NUM_WORKERS = 8
177 train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, num_workers=
    NUM_WORKERS, pin_memory=True)
178 val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=
    NUM_WORKERS, pin_memory=True)
179 test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False, num_workers=
    NUM_WORKERS, pin_memory=True)
180
181 print("DataLoaderele au fost create cu succes!")
182 print(f"Numar de batch-uri pe antrenament: {len(train_loader)}")
183 print(f"Ben Graham: {'ACTIV' if USE_BEN_GRAHAM else 'dezactivat'}")
184
185
186 # =====
187 # Model + configuratie in doua faze
188 # =====
189
190 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
191 print(f"Antrenam pe: {device}")
192
193 # --- Hiperparametri ---

```

```

194 HEAD_LR_PHASE1      = 1e-3
195 BACKBONE_LR_PHASE2  = 1e-4
196 HEAD_LR_PHASE2      = 3e-4
197 WEIGHT_DECAY        = 1e-4
198 DROPOUT              = 0.40
199 FREEZE_BACKBONE_EPOCHS = 3
200 EARLY_STOPPING_PATIENCE = 30
201 USE_POS_WEIGHT       = False
202
203 # --- EfficientNet-B3 pre-antrenat ---
204 model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
205 num_features = model.classifier[1].in_features
206 model.classifier[1] = nn.Sequential(
207     nn.Dropout(p=DROPOUT),
208     nn.Linear(num_features, 1)
209 )
210 model = model.to(device)
211
212 # Verificam distributia claselor
213 num_sanatosi = len(os.listdir(os.path.join(DATA_DIR, 'train', 'sanatos')))
214 num_bolnavi  = len(os.listdir(os.path.join(DATA_DIR, 'train', 'bolnav')))
215 print(f"Clase: sanatos={num_sanatosi}, bolnav={num_bolnavi}")
216
217 if USE_POS_WEIGHT:
218     pos_weight = torch.tensor([num_sanatosi / max(num_bolnavi, 1)], dtype=torch.float32,
219                               device=device)
219     criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
220     print(f"pos_weight activ: {pos_weight.item():.4f}")
221 else:
222     criterion = nn.BCEWithLogitsLoss()
223     print("pos_weight dezactivat (set echilibrat).")
224
225
226 # --- Helpers pentru inghet/dezghet si construirea optimizatorului pe faze ---
227 def set_backbone_trainable(m, trainable):
228     for name, p in m.named_parameters():
229         if not name.startswith("classifier"):
230             p.requires_grad = trainable
231
232 def split_params(m):
233     head = [p for n, p in m.named_parameters() if n.startswith("classifier")]
234     backbone = [p for n, p in m.named_parameters() if not n.startswith("classifier")]
235     return head, backbone
236
237 def build_optimizer_phase1(m):
238     set_backbone_trainable(m, False)
239     head, _ = split_params(m)
240     opt = optim.AdamW(head, lr=HEAD_LR_PHASE1, weight_decay=WEIGHT_DECAY)
241     sch = optim.lr_scheduler.ReduceLROnPlateau(opt, mode='max', factor=0.5, patience=2)
242     return opt, sch
243
244 def build_optimizer_phase2(m):
245     set_backbone_trainable(m, True)
246     head, backbone = split_params(m)
247     opt = optim.AdamW([
248         {"params": backbone, "lr": BACKBONE_LR_PHASE2},
249         {"params": head, "lr": HEAD_LR_PHASE2},
250     ], weight_decay=WEIGHT_DECAY)
251     sch = optim.lr_scheduler.ReduceLROnPlateau(opt, mode='max', factor=0.5, patience=2)
252     return opt, sch
253
254 scaler = torch.amp.GradScaler('cuda')
255 print("Model si configuratie gata. Faza 1 (backbone inghetat) incepe la antrenare.")
256
257
258 # =====
259 # (Optional) Sanity-check: poate modelul sa suprainvete 100 de imagini?
260 # =====
261 RUN_SANITY = True
262
263 if RUN_SANITY:
264     import copy
265     small = Subset(train_ds, list(range(100)))
266     small_loader = DataLoader(small, batch_size=16, shuffle=True, num_workers=0)
267     m = copy.deepcopy(model)
268     set_backbone_trainable(m, True)

```

```

269     opt = optim.AdamW(m.parameters(), lr=1e-4, weight_decay=0)
270     for ep in range(20):
271         m.train(); preds, tgts = [], []
272         for images, labels in small_loader:
273             images = images.to(device); labels = labels.float().to(device)
274             opt.zero_grad()
275             with torch.amp.autocast('cuda'):
276                 out = m(images).squeeze(1)
277                 loss = criterion(out, labels)
278                 scaler.scale(loss).backward(); scaler.step(opt); scaler.update()
279                 preds += ((torch.sigmoid(out) > 0.5).int().detach().cpu().numpy().tolist())
280                 tgts += labels.int().cpu().numpy().tolist()
281             acc = accuracy_score(tgts, preds)
282             print(f"sanity epoca {ep+1}: train_acc={acc:.3f}")
283     print("Daca acc ajunge ~1.0 -> pipeline OK. Daca ramane ~0.5 -> problema in date/etichete.
284         ")
285 else:
286     print("Sanity-check dezactivat (RUN_SANITY=False).")
287
288 # =====
289 # Antrenare (doua faze + early stopping pe AUC)
290 # =====
291
292 EPOCHS = 50
293 FREEZE_BACKBONE_EPOCHS = 10
294 best_val_auc = -1.0
295 bad_epochs = 0
296
297 optimizer, scheduler = build_optimizer_phase1(model)
298 print("=== INCEPERE ANTRENARE - FAZA 1 (backbone inghetat) ===")
299
300 history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": [], "val_auc": []}
301
302 for epoch in range(EPOCHS):
303     if epoch == FREEZE_BACKBONE_EPOCHS:
304         optimizer, scheduler = build_optimizer_phase2(model)
305         print("=== FAZA 2: backbone dezghetat, fine-tuning complet cu LR diferentiat ===")
306
307     # ---- TRAIN ----
308     model.train()
309     train_loss = 0.0
310     train_corrects = 0
311
312     for images, labels in tqdm(train_loader, desc=f"Epoca {epoch+1}/{EPOCHS} [TRAIN]"):
313         images = images.to(device)
314         labels = labels.float().to(device)
315         optimizer.zero_grad()
316
317         with torch.amp.autocast('cuda'):
318             outputs = model(images).squeeze(1)
319             loss = criterion(outputs, labels)
320
321             probs = torch.sigmoid(outputs.detach())
322             preds = (probs > 0.5).float()
323             train_corrects += torch.sum(preds == labels).item()
324
325             scaler.scale(loss).backward()
326             scaler.step(optimizer)
327             scaler.update()
328
329             train_loss += loss.item() * images.size(0)
330
331     # ---- VAL ----
332     model.eval()
333     val_loss = 0.0
334     val_probs, val_targets = [], []
335     with torch.no_grad():
336         for images, labels in tqdm(val_loader, desc=f"Epoca {epoch+1}/{EPOCHS} [VAL]"):
337             images = images.to(device)
338             labels = labels.float().to(device)
339             outputs = model(images).squeeze(1)
340             loss = criterion(outputs, labels)
341             val_loss += loss.item() * images.size(0)
342             val_probs.extend(torch.sigmoid(outputs).cpu().numpy())
343             val_targets.extend(labels.cpu().numpy())

```

```

344
345     avg_train_loss = train_loss / len(train_ds)
346     avg_train_acc = train_corrects / len(train_ds)
347
348     avg_val_loss = val_loss / len(val_ds)
349     val_preds = (np.array(val_probs) > 0.5).astype(int)
350     val_acc = accuracy_score(val_targets, val_preds)
351
352     try:
353         val_auc = roc_auc_score(val_targets, val_probs)
354     except ValueError:
355         val_auc = float("nan")
356
357     history["train_loss"].append(avg_train_loss)
358     history["train_acc"].append(avg_train_acc)
359     history["val_loss"].append(avg_val_loss)
360     history["val_acc"].append(val_acc)
361     history["val_auc"].append(val_auc)
362
363     scheduler.step(val_auc)
364
365     current_lr = optimizer.param_groups[0]['lr']
366
367     print(f"Epoca {epoch+1}: T_Loss={avg_train_loss:.4f} | T_Acc={avg_train_acc:.4f} | V_Loss
368           = {avg_val_loss:.4f} "
369           f"| V_Acc={val_acc:.4f} | V_AUC={val_auc:.4f} | LR={current_lr:.1e}")
370
371     # Checkpoint + early stopping pe AUC
372     if val_auc > best_val_auc + 1e-4:
373         best_val_auc = val_auc
374         bad_epochs = 0
375         torch.save(model.state_dict(), 'efficientnet_b3_binar_best.pth')
376         print(f"Model nou salvat! (Val AUC: {best_val_auc:.4f})")
377     else:
378         bad_epochs += 1
379         print(f"Fara imbunatatire pe AUC: {bad_epochs}/{EARLY_STOPPING_PATIENCE}")
380         if bad_epochs >= EARLY_STOPPING_PATIENCE:
381             print("Early stopping.")
382             break
383
384     print(f"Cel mai bun Val AUC: {best_val_auc:.4f}")
385
386     # =====
387     # Evalua pe test
388     # =====
389
390     import seaborn as sns
391     from sklearn.metrics import confusion_matrix, fbeta_score, f1_score
392
393     model.load_state_dict(torch.load(DIR_BINAR / 'efficientnet_b3_binar_best.pth')); model.eval()
394     CN_BIN = ['Sanatos (0)', 'Bolnav (1)']
395
396     # beta=2 -> FN sunt de 2x mai grave decat FP
397     beta = 2.0
398     best_thr, best_fb = 0.5, 0.0
399
400     # Prag optim pe validare
401     v_probs, v_targets = [], []
402     with torch.no_grad():
403         for images, labels in tqdm(val_loader, desc="Val (prag)"):
404             images = images.to(device)
405             v_probs.extend(torch.sigmoid(model(images)).squeeze(1)).cpu().numpy())
406             v_targets.extend(labels.numpy())
407     v_probs = np.array(v_probs)
408
409     for t in np.linspace(0.01, 0.99, 200):
410         preds = (v_probs > t).astype(int)
411         fb = fbeta_score(v_targets, preds, beta=beta, pos_label=1, zero_division=0)
412         if fb > best_fb:
413             best_fb = fb
414             best_thr = t
415
416     print(f"Prag optim pe F{beta}: {best_thr:.3f} (F{beta}={best_fb:.3f})")
417
418     # Probabilitati pe TEST
419     test_probs, test_targets = [], []

```

```

419 with torch.no_grad():
420     for images, labels in tqdm(test_loader, desc="Testare"):
421         images = images.to(device)
422         test_probs.extend(torch.sigmoid(model(images).squeeze(1)).cpu().numpy())
423         test_targets.extend(labels.numpy())
424 test_probs = np.array(test_probs)
425 test_auc = roc_auc_score(test_targets, test_probs)
426 print(f"\nTest AUC: {test_auc:.4f}")
427
428 for label, t in [("prag 0.5", 0.5), (f"prag {best_thr:.3f} (din val)", best_thr)]:
429     preds = (test_probs > t).astype(int)
430     print(f"\nRAPORT - {label}:")
431     print(classification_report(test_targets, preds, target_names=CN_BIN, labels=[0,1], digits
432                                     =3))
433
434 test_pred = (test_probs > best_thr).astype(int)
435
436 # Grafice
437 plot_curbe(history["train_loss"], history["val_loss"], history["train_acc"], history["val_acc"]
438             ],
439             history["val_auc"], 'AUC', DIR_BINAR, '- Binar')
440 plot_prf(test_targets, test_pred, CN_BIN, DIR_BINAR, '- Binar')
441 plot_confuzie(test_targets, test_pred, CN_BIN, DIR_BINAR, '- Binar')
442 print(f"Grafice salvate in {DIR_BINAR}")
443
444 # =====
445 # Clasificare 1-4 - sectiune comuna
446 # =====
447
448 import os
449 import numpy as np
450 import torch
451 import torch.nn as nn
452 import torch.nn.functional as F
453 import torch.optim as optim
454 from torchvision import models, transforms
455 from torch.utils.data import DataLoader, Subset
456 from tqdm import tqdm
457 from sklearn.metrics import classification_report, accuracy_score, confusion_matrix,
458     cohen_kappa_score
459 import scipy.optimize as opt
460 import matplotlib.pyplot as plt
461 import seaborn as sns
462 import sys
463 from pathlib import Path
464
465 DATA_DIR = r'B:\Projects\Disertatie\Diabetic-Retinopathy-Classifier\datasets\
466     processed_by_me\eyepacs\eyepacs_1_4'
467 IMG_SIZE = 300
468 BATCH_SIZE = 32
469 NUM_CLASSES = 4
470 CLASS_NAMES = ['Stadiul 1', 'Stadiul 2', 'Stadiul 3', 'Stadiul 4']
471 NUM_WORKERS = 8
472
473 _norm = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
474
475 train_transform = transforms.Compose([
476     transforms.Resize((IMG_SIZE, IMG_SIZE)),
477     transforms.RandomHorizontalFlip(p=0.5),
478     transforms.RandomVerticalFlip(p=0.5),
479     transforms.RandomRotation(15),
480     transforms.ColorJitter(brightness=0.15, contrast=0.15),
481     transforms.RandomAffine(degrees=0, translate=(0.05, 0.05)),
482     transforms.ToTensor(),
483     _norm,
484 ])
485
486 test_transform = transforms.Compose([
487     transforms.Resize((IMG_SIZE, IMG_SIZE)),
488     transforms.ToTensor(),
489     _norm,
490 ])
491
492 LOCAL_DIR = Path(r'B:\Projects\Disertatie\Diabetic-Retinopathy-Classifier\Notebooks\Main\Local
493 ')
494 if str(LOCAL_DIR) not in sys.path:

```

```

490     sys.path.insert(0, str(LOCAL_DIR))
491 from dataset_1_4 import MultiClassRetinopathyDataset
492
493 train_ds = MultiClassRetinopathyDataset(os.path.join(DATA_DIR, 'train'), transform=
    train_transform)
494 val_ds   = MultiClassRetinopathyDataset(os.path.join(DATA_DIR, 'val'),   transform=
    test_transform)
495 test_ds  = MultiClassRetinopathyDataset(os.path.join(DATA_DIR, 'test'),  transform=
    test_transform)
496
497 loader_kwargs = {'num_workers': NUM_WORKERS, 'pin_memory': torch.cuda.is_available()}
498 if NUM_WORKERS > 0:
499     loader_kwargs['prefetch_factor'] = 1
500
501 train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True, **loader_kwargs)
502 val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False, **loader_kwargs)
503 test_loader  = DataLoader(test_ds,  batch_size=BATCH_SIZE, shuffle=False, **loader_kwargs)
504
505 print("=== STATISTICI DATASET ===")
506 print(f"Mapare clase: {train_ds.class_to_idx}")
507 for split_name, ds in [("TRAIN", train_ds), ("VAL", val_ds), ("TEST", test_ds)]:
508     targets = np.array(ds.targets)
509     print(f"{split_name} ({len(ds)}): " + ", ".join(f"{c}={((targets==i).sum())}" for i,c in
        enumerate(CLASS_NAMES)))
510 print(f"Batch-uri train: {len(train_loader)}")
511
512
513 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
514 print(f"Antrenam pe: {device}")
515
516 HEAD_LR_PHASE1           = 1e-3
517 BACKBONE_LR_PHASE2      = 1e-4
518 HEAD_LR_PHASE2           = 3e-4
519 WEIGHT_DECAY             = 1e-4
520 DROPOUT                  = 0.40
521 FREEZE_BACKBONE_EPOCHS  = 5
522 TOTAL_EPOCHS             = 50
523 EARLY_STOPPING_PATIENCE = 30
524 LABEL_SMOOTHING         = 0.10
525 SEED                     = 42
526
527 torch.manual_seed(SEED); np.random.seed(SEED)
528
529 # ----- Construire model in functie de MODE -----
530 def build_model(mode):
531     m = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
532     nf = m.classifier[1].in_features
533     if mode == "crossentropy":
534         m.classifier[1] = nn.Sequential(nn.Dropout(DROPOUT), nn.Linear(nf, NUM_CLASSES))
535     elif mode == "regression":
536         m.classifier[1] = nn.Sequential(nn.Dropout(DROPOUT), nn.Linear(nf, 1))
537     else:
538         raise ValueError(f"MODE necunoscut: {mode}")
539     return m.to(device)
540
541 ce_criterion = nn.CrossEntropyLoss(label_smoothing=LABEL_SMOOTHING)
542 reg_criterion = nn.SmoothL1Loss()
543
544 def compute_loss(out, labels):
545     if MODE == "crossentropy":
546         return ce_criterion(out, labels)
547     if MODE == "regression":
548         return reg_criterion(out.squeeze(1), labels.float())
549
550 def decode(out):
551     """Predictii intregi 0..K-1 pentru monitorizare (regresie: rotunjire naiva)."""
552     if MODE == "crossentropy":
553         return out.argmax(1)
554     if MODE == "regression":
555         return out.squeeze(1).round().clamp(0, NUM_CLASSES - 1).long()
556
557 def raw_scores(out):
558     """Iesirea continua pentru regresie (necesara la rounder-ul optimizat)."""
559     return out.squeeze(1) if MODE == "regression" else None
560
561 # ----- Rounder optimizat pe validare (doar regresie) -----

```

```

562 class OptimizedRounder:
563     """Alege K-1 praguri care maximizeaza QWK pe validare (Nelder-Mead)."""
564     def __init__(self, n_classes): self.n_classes = n_classes; self.coef_ = None
565     def _classify(self, X, coef):
566         return np.clip(np.digitize(X, np.sort(coef)), 0, self.n_classes - 1)
567     def _neg_qwk(self, coef, X, y):
568         return -cohen_kappa_score(y, self._classify(X, coef), weights='quadratic')
569     def fit(self, X, y):
570         init = np.arange(self.n_classes - 1) + 0.5
571         r = opt.minimize(self._neg_qwk, init, args=(X, y), method='Nelder-Mead')
572         self.coef_ = np.sort(r.x)
573     def predict(self, X):
574         return self._classify(X, self.coef_)
575
576 # ----- Freeze/unfreeze + optimizatoare pe faze -----
577 def set_backbone_trainable(m, trainable):
578     for name, p in m.named_parameters():
579         if not name.startswith("classifier"):
580             p.requires_grad = trainable
581
582 def split_params(m):
583     head = [p for n, p in m.named_parameters() if n.startswith("classifier")]
584     backbone = [p for n, p in m.named_parameters() if not n.startswith("classifier")]
585     return head, backbone
586
587 def build_optimizer_phase1(m):
588     set_backbone_trainable(m, False)
589     head, _ = split_params(m)
590     o = optim.AdamW(head, lr=HEAD_LR_PHASE1, weight_decay=WEIGHT_DECAY)
591     s = optim.lr_scheduler.ReduceLROnPlateau(o, mode='max', factor=0.5, patience=2)
592     return o, s
593
594 def build_optimizer_phase2(m):
595     set_backbone_trainable(m, True)
596     head, backbone = split_params(m)
597     o = optim.AdamW([{"params": backbone, "lr": BACKBONE_LR_PHASE2},
598                     {"params": head, "lr": HEAD_LR_PHASE2}], weight_decay=WEIGHT_DECAY)
599     s = optim.lr_scheduler.ReduceLROnPlateau(o, mode='max', factor=0.5, patience=2)
600     return o, s
601
602 print("Helperi de clasificare gata (build_model, compute_loss, decode, ...).")
603
604
605 def ruleaza_ramura(mode):
606     """Codul tau de antrenare in 2 faze + evaluare, pentru un mod."""
607     global MODE
608     MODE = mode
609     torch.manual_seed(SEED); np.random.seed(SEED)
610     out_dir = dir_clasificare(mode)
611     CKPT = str(out_dir / 'model_best.pth')
612
613     model = build_model(mode)
614     scaler = torch.amp.GradScaler('cuda', enabled=(device.type == 'cuda'))
615     optimizer, scheduler = build_optimizer_phase1(model)
616     best_val_qwk = -1.0
617     epochs_no_improve = 0
618     history = {'t_loss': [], 'v_loss': [], 't_acc': [], 'v_acc': [], 'v_qwk': []}
619
620     print(f"=== ANTRENARE [{mode}] - FAZA 1 (backbone inghetat) ===")
621     for epoch in range(TOTAL_EPOCHS):
622         if epoch == FREEZE_BACKBONE_EPOCHS:
623             optimizer, scheduler = build_optimizer_phase2(model)
624             print("=== FAZA 2: backbone dezghetat, fine-tuning complet ===")
625
626         # ---- TRAIN ----
627         model.train(); train_loss = 0.0; train_preds, train_targets = [], []
628         for images, labels in tqdm(train_loader, desc=f"[{mode}] Epoca {epoch+1}/{TOTAL_EPOCHS} [TRAIN]"):
629             images = images.to(device, non_blocking=True)
630             labels = labels.to(device, non_blocking=True)
631             optimizer.zero_grad(set_to_none=True)
632             with torch.amp.autocast('cuda', enabled=(device.type == 'cuda')):
633                 out = model(images); loss = compute_loss(out, labels)
634             scaler.scale(loss).backward(); scaler.step(optimizer); scaler.update()
635             train_loss += loss.item() * images.size(0)
636             train_preds.extend(decode(out).detach().cpu().tolist())

```



```

637         train_targets.extend(labels.detach().cpu().tolist())
638
639     # ---- VAL ----
640     model.eval(); val_loss = 0.0; val_preds, val_targets = [], []
641     with torch.no_grad():
642         for images, labels in tqdm(val_loader, desc=f"[{mode}] Epoca {epoch+1}/{
TOTAL_EPOCHS} [VAL]"):
643             images = images.to(device, non_blocking=True)
644             labels = labels.to(device, non_blocking=True)
645             with torch.amp.autocast('cuda', enabled=(device.type == 'cuda')):
646                 out = model(images); loss = compute_loss(out, labels)
647                 val_loss += loss.item() * images.size(0)
648                 val_preds.extend(decode(out).detach().cpu().tolist())
649                 val_targets.extend(labels.detach().cpu().tolist())
650
651     avg_train_loss = train_loss / len(train_ds)
652     avg_val_loss = val_loss / len(val_ds)
653     train_acc = accuracy_score(train_targets, train_preds)
654     val_acc = accuracy_score(val_targets, val_preds)
655     val_qwk = cohen_kappa_score(val_targets, val_preds, weights='quadratic')
656     for k, v in zip(history, [avg_train_loss, avg_val_loss, train_acc, val_acc, val_qwk]):
657         history[k].append(v)
658     scheduler.step(val_qwk)
659     print(f"Epoca {epoch+1}: T_Loss={avg_train_loss:.4f} | V_Loss={avg_val_loss:.4f} "
660           f"| T_Acc={train_acc:.4f} | V_Acc={val_acc:.4f} | V_QWK={val_qwk:.4f}")
661
662     if val_qwk > best_val_qwk + 1e-4:
663         best_val_qwk = val_qwk; epochs_no_improve = 0
664         torch.save(model.state_dict(), CKPT)
665         print(f"Model nou salvat! (Val QWK: {best_val_qwk:.4f})")
666     else:
667         epochs_no_improve += 1
668         print(f"Fara imbunatatire pe QWK: {epochs_no_improve}/{EARLY_STOPPING_PATIENCE}")
669         if epochs_no_improve >= EARLY_STOPPING_PATIENCE:
670             print("Early stopping."); break
671     print(f"Cel mai bun Val QWK [{mode}]: {best_val_qwk:.4f}")
672
673     # ---- EVALUARE PE TEST ----
674     model.load_state_dict(torch.load(CKPT)); model.eval()
675     def collect(loader):
676         preds, targets, scores = [], [], []
677         with torch.no_grad():
678             for images, labels in tqdm(loader, desc=f"[{mode}] eval"):
679                 images = images.to(device)
680                 out = model(images)
681                 preds.extend(decode(out).cpu().tolist())
682                 targets.extend(labels.tolist())
683                 if mode == "regression":
684                     scores.extend(raw_scores(out).cpu().numpy())
685         return np.array(preds), np.array(targets), np.array(scores)
686
687     if mode == "regression":
688         _, val_t, val_s = collect(val_loader)
689         rounder = OptimizedRounder(NUM_CLASSES); rounder.fit(val_s, val_t)
690         np.save(out_dir / 'rounder_coef.npy', rounder.coef_)
691         print(f"Praguri optime (din val): {np.round(rounder.coef_, 3)}")
692         _, test_t, test_s = collect(test_loader)
693         test_p = rounder.predict(test_s)
694     else:
695         test_p, test_t, _ = collect(test_loader)
696
697     test_acc = accuracy_score(test_t, test_p)
698     test_qwk = cohen_kappa_score(test_t, test_p, weights='quadratic')
699     print(f"\nTest Accuracy : {test_acc:.4f}\nTest QWK : {test_qwk:.4f}\n")
700     print(classification_report(test_t, test_p, target_names=CLASS_NAMES, labels=list(range(
NUM_CLASSES)), digits=3))
701
702     # Grafice
703     plot_curbe(history['t_loss'], history['v_loss'], history['t_acc'], history['v_acc'],
704               history['v_qwk'], 'QWK', out_dir, f'- [{mode}]')
705     plot_prf(test_t, test_p, CLASS_NAMES, out_dir, f'- [{mode}]')
706     plot_confuzie(test_t, test_p, CLASS_NAMES, out_dir, f'- [{mode}] QWK={test_qwk:.3f}')
707     print(f"Salvat in {out_dir}")
708     return {'mode': mode, 'qwk': test_qwk, 'acc': test_acc}
709
710 print("Functia ruleaza_ramura_gata.")

```



```

711
712 # Ramura regression
713 res_reg = ruleaza_ramura("regression")
714
715 # =====
716 # Inferenta - cascada (binar -> clasificare)
717 # =====
718
719 from PIL import Image
720
721 INFER_MODE = "regression"      # "crossentropy" | "regression"
722 PRAG_BINAR = 0.50             # p_sick < PRAG => sanatos (grad 0)
723
724 def _build_binary():
725     m = models.efficientnet_b3(weights=None)
726     nf = m.classifier[1].in_features
727     m.classifier[1] = nn.Sequential(nn.Dropout(0.40), nn.Linear(nf, 1))
728     return m.to(device)
729
730 binm = _build_binary()
731 binm.load_state_dict(torch.load(DIR_BINAR / 'efficientnet_b3_binar_best.pth')); binm.eval()
732
733 MODE = INFER_MODE
734 clsm = build_model(INFER_MODE)
735 clsm.load_state_dict(torch.load(dir_clasificare(INFER_MODE) / 'model_best.pth')); clsm.eval()
736
737 _rounder_coef = None
738 if INFER_MODE == "regression":
739     p = dir_clasificare("regression") / 'rounder_coef.npy'
740     if p.exists():
741         _rounder_coef = np.load(p)
742
743 GRADE_NAMES = ['Grad 0 (No DR)', 'Grad 1', 'Grad 2', 'Grad 3', 'Grad 4']
744
745 @torch.no_grad()
746 def prezice_imagine(path, prag=PRAG_BINAR):
747     img = Image.open(path).convert("RGB")
748     x = test_transform(img).unsqueeze(0).to(device)
749     p_sick = torch.sigmoid(binm(x).squeeze(1)).item()
750     if p_sick < prag:
751         return {"grad_final": 0, "eticheta": GRADE_NAMES[0], "p_sick": round(p_sick, 3), "etapa": "binar"}
752     out = clsm(x)
753     if INFER_MODE == "regression":
754         s = out.squeeze(1).item()
755         if _rounder_coef is not None:
756             g = int(np.clip(np.digitize([s], np.sort(_rounder_coef))[0], 0, NUM_CLASSES - 1))
757         else:
758             g = int(np.clip(round(s), 0, NUM_CLASSES - 1))
759     else:
760         g = int(decode(out).item())
761     grad_final = g + 1 # clasificatorul da 0..3 -> gradele 1..4
762     return {"grad_final": grad_final, "eticheta": GRADE_NAMES[grad_final], "p_sick": round(p_sick, 3), "etapa": "cascada"}
763
764 import random
765 print("=== DEMO INFERENTA IN CASCADA (Binar -> Multi-Klasa) ===")
766
767 DIR_TEST_SANATOS = r'B:\Projects\Disertatie\Diabetic-Retinopathy-Classfier\datasets\processed_by_me\eyepacs\eyepacs_binar\test\0_sanatos'
768 DIR_TEST_MULTII = r'B:\Projects\Disertatie\Diabetic-Retinopathy-Classfier\datasets\processed_by_me\eyepacs\eyepacs_1_4\test'
769
770 fisiere_demo = []
771
772 if os.path.exists(DIR_TEST_SANATOS):
773     poze_0 = [os.path.join(DIR_TEST_SANATOS, f) for f in os.listdir(DIR_TEST_SANATOS) if f.endswith(('.png', '.jpg', '.jpeg'))]
774     if poze_0:
775         fisiere_demo.extend(random.sample(poze_0, min(2, len(poze_0))))
776
777 stadii = ['stadiul_1', 'stadiul_2', 'stadiul_3', 'stadiul_4']
778 for stadiu in stadii:
779     dir_stadiu = os.path.join(DIR_TEST_MULTII, stadiu)
780     if os.path.exists(dir_stadiu):

```

```

781     poze_stadiu = [os.path.join(dir_stadiu, f) for f in os.listdir(dir_stadiu) if f.
endswith(('.png', '.jpg', '.jpeg'))]
782     if poze_stadiu:
783         fisiere_demo.append(random.choice(poze_stadiu))
784
785 random.shuffle(fisiere_demo)
786 fisiere_demo = fisiere_demo[:6]
787
788 if not fisiere_demo:
789     print("[Eroare] Nu am gasit imagini. Verifica rutele folderelor de test.")
790 else:
791     fig, axes = plt.subplots(2, 3, figsize=(16, 10))
792     axes = axes.flatten()
793
794     for idx, img_path in enumerate(fisiere_demo):
795         rezultat = prezice_imagine(img_path)
796         img_display = Image.open(img_path)
797
798         ax = axes[idx]
799         ax.imshow(img_display)
800         ax.axis('off')
801
802         titlu = f"Predictie: {rezultat['eticheta']}\n"
803         titlu += f"P_bolnav: {rezultat['p_sick']:.3f} | Model decizional: {rezultat['etapa'].
upper()}"
804
805         culoare = 'green' if rezultat['grad_final'] == 0 else 'red'
806         ax.set_title(titlu, color=culoare, fontsize=11, fontweight='bold', pad=10)
807
808     for i in range(len(fisiere_demo), len(axes)):
809         axes[i].axis('off')
810
811 plt.tight_layout()
812 plt.show()

```

Listing A.1: codul de antrenare pentru sistemul principal

```

1 import os
2
3 import torch
4 from PIL import Image
5 from torch.utils.data import ConcatDataset, Dataset
6 import torchvision.transforms as T
7
8
9 IMAGE_EXTENSIONS = ('.png', '.jpg', '.jpeg')
10
11
12 def default_transform(split='train', image_size=224, train_augment='basic'):
13     split = split.lower()
14     train_augment = train_augment.lower()
15
16     if split == 'train' and train_augment == 'basic':
17         return T.Compose([
18             T.Resize((image_size, image_size)),
19             T.RandomHorizontalFlip(p=0.5),
20             T.RandomVerticalFlip(p=0.5),
21             T.RandomRotation(degrees=15),
22             T.ColorJitter(brightness=0.1, contrast=0.1),
23             T.ToTensor(),
24             T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
25         ])
26
27     return T.Compose([
28         T.Resize((image_size, image_size)),
29         T.ToTensor(),
30         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
31     ])
32
33
34 class ImageFolderDRDataset(Dataset):
35     def __init__(self, root_dir, split='train', image_size=224, transform=None, train_augment=
'basic'):

```

```

36     self.root_dir = root_dir
37     self.split = split.lower()
38     self.image_size = image_size
39     self.split_dir = os.path.join(self.root_dir, self.split)
40
41     if not os.path.exists(self.split_dir):
42         raise FileNotFoundError(f"[Eroare] Folderul nu exista: {self.split_dir}")
43
44     self.image_paths = []
45     self.labels = []
46
47     for class_id in range(5):
48         class_folder = os.path.join(self.split_dir, str(class_id))
49         if not os.path.exists(class_folder):
50             continue
51         for img_name in os.listdir(class_folder):
52             if img_name.lower().endswith(IMAGE_EXTENSIONS):
53                 self.image_paths.append(os.path.join(class_folder, img_name))
54                 self.labels.append(class_id)
55
56     self.transform = transform or default_transform(self.split, self.image_size,
57 train_augment=train_augment)
58
59 def __len__(self):
60     return len(self.image_paths)
61
62 def __getitem__(self, idx):
63     img_path = self.image_paths[idx]
64     label = self.labels[idx]
65
66     try:
67         image = Image.open(img_path).convert('RGB')
68     except Exception:
69         image = Image.new('RGB', (self.image_size, self.image_size), (0, 0, 0))
70
71     if self.transform:
72         image = self.transform(image)
73
74     return image, torch.tensor(label, dtype=torch.long)
75
76 def make_dataset(
77     root_dirs,
78     split,
79     image_size,
80     train_augment='basic',
81 ):
82     if isinstance(root_dirs, (str, os.PathLike)):
83         root_dirs = [root_dirs]
84
85     root_dirs = [str(root_dir) for root_dir in root_dirs if root_dir]
86     if not root_dirs:
87         raise ValueError("Trebuie sa furnizezi cel putin un root de dataset.")
88
89     if len(root_dirs) > 1:
90         return ConcatDataset([
91             ImageFolderDRDataset(
92                 root_dir,
93                 split=split,
94                 image_size=image_size,
95                 train_augment=train_augment,
96             )
97             for root_dir in root_dirs
98         ])
99
100     return ImageFolderDRDataset(root_dirs[0], split=split, image_size=image_size,
train_augment=train_augment)

```

Listing A.2: my_dataset.py – încărcarea setului de date echilibrat pentru experimente

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F

```

```

4 import torchvision.models as models
5 import torch.optim as optim
6
7 # Importam timm pentru Inception-ResNet-v2
8 try:
9     import timm
10 except ImportError:
11     timm = None
12
13 class InceptionWrapper(nn.Module):
14     """
15     Carcasa de protectie pentru Inception-v3.
16     Lasa modelul sa se incarce nativ, dar returneaza o singura iesire la antrenare,
17     exact cum asteapta functiile noastre de Loss.
18     """
19     def __init__(self, num_classes, pretrained=True):
20         super().__init__()
21         # Lasam Inception sa se initializeze 100% standard (fara erori de aux_logits)
22         weights = models.Inception_V3_Weights.DEFAULT if pretrained else None
23         self.model = models.inception_v3(weights=weights)
24
25         # Modificam ultimul strat
26         in_features = self.model.fc.in_features
27         self.model.fc = nn.Linear(in_features, num_classes)
28
29     def forward(self, x):
30         outputs = self.model(x)
31         # La antrenare, Inception returneaza un obiect cu 2 iesiri. Noi o dam mai departe doar
32         # pe prima.
33         if self.training:
34             return outputs[0] # outputs[0] reprezinta predictia principala (logits)
35         # La validare, returneaza oricum o singura iesire
36         return outputs
37
38 # =====
39 # 1. Implementare focal loss pentru clasificare multi-clasa
40 # =====
41 class FocalLossMultiClass(nn.Module):
42     """
43     Focal Loss adaptat pentru clasificare multi-clasa.
44     Accepta predictii de forma [Batch, Clase] si etichete standard (indici) de forma [Batch].
45     """
46     def __init__(self, alpha=None, gamma=1.5, reduction='mean', label_smoothing=0.0):
47         super(FocalLossMultiClass, self).__init__()
48         self.gamma = gamma
49         self.alpha = alpha
50         self.reduction = reduction
51         self.label_smoothing = label_smoothing
52
53     def forward(self, inputs, targets):
54         # Calculam direct Cross Entropy (care stie sa citeasca indicii claselor)
55         ce_loss = F.cross_entropy(inputs, targets, weight=self.alpha, reduction='none',
56                                   label_smoothing=self.label_smoothing)
57         pt = torch.exp(-ce_loss)
58         focal_loss = ((1 - pt) ** self.gamma) * ce_loss
59
60         if self.reduction == 'mean':
61             return focal_loss.mean()
62         elif self.reduction == 'sum':
63             return focal_loss.sum()
64         else:
65             return focal_loss
66
67 # =====
68 # 2. FABRICA DE MODELE
69 # =====
70 def get_model(model_name, num_classes=5, pretrained=True, drop_rate=0.0):
71     """
72     Fabrica de modele. Toate au strat final modificat pentru num_classes.
73     """
74     model_name = model_name.lower()
75
76     if model_name == 'resnet50':
77         weights = models.ResNet50_Weights.DEFAULT if pretrained else None
78         model = models.resnet50(weights=weights)

```

```

78         num_ftrs = model.fc.in_features
79         model.fc = nn.Sequential(
80             nn.Dropout(0.5),
81             nn.Linear(num_ftrs, num_classes)
82         )
83
84     elif model_name == 'efficientnet_b3':
85         weights = models.EfficientNet_B3_Weights.DEFAULT if pretrained else None
86         model = models.efficientnet_b3(weights=weights)
87         in_features = model.classifier[1].in_features
88         model.classifier[1] = nn.Sequential(
89             nn.Dropout(0.5),
90             nn.Linear(in_features, num_classes)
91         )
92
93     elif model_name == 'inception_v3':
94         model = InceptionWrapper(num_classes=num_classes, pretrained=pretrained)
95
96     elif model_name == 'incres_v2':
97         if timm is None:
98             raise ImportError(
99                 "Modelul incres_v2 necesita pachetul 'timm'. Instaleaza-l cu: pip install timm
100             )
101         model = timm.create_model(
102             'inception_resnet_v2',
103             pretrained=pretrained,
104             num_classes=num_classes,
105             drop_rate=drop_rate,
106         )
107
108     else:
109         raise ValueError(f"Modelul {model_name} nu este suportat!")
110
111     return model
112
113 # =====
114 # 3. FABRICA DE LOSS-URI SI PREDICTII (Totul se intampla aici)
115 # =====
116 def get_loss_function(loss_name, class_weights=None, device='cuda', gamma=1.5, label_smoothing=0.0):
117     loss_name = loss_name.lower()
118
119     # -----
120     # REPARAT: Transformam lista venita din consola intr-un Tensor
121     # -----
122     if class_weights is not None and isinstance(class_weights, list):
123         class_weights = torch.tensor(class_weights, dtype=torch.float)
124
125     if loss_name in ['bce_ordinal', 'ordinal']:
126         return nn.BCEWithLogitsLoss()
127
128     elif loss_name == 'focal_loss':
129         # Trimitem ponderile pe GPU (daca exista) pentru Focal Loss
130         alpha_tensor = class_weights.to(device) if class_weights is not None else None
131         return nn.FocalLossMultiClass(alpha=alpha_tensor, gamma=gamma, label_smoothing=label_smoothing)
132
133     elif loss_name == 'ce':
134         return nn.CrossEntropyLoss(label_smoothing=label_smoothing)
135
136     elif loss_name == 'weighted_ce':
137         if class_weights is None:
138             raise ValueError("Pentru Weighted CE, trebuie sa trimiti '--class_weights'!")
139         # Acum class_weights este oficial un Tensor, deci .to(device) va functiona perfect!
140         return nn.CrossEntropyLoss(weight=class_weights.to(device), label_smoothing=label_smoothing)
141
142     else:
143         raise ValueError(f"Loss-ul '{loss_name}' nu este suportat.")
144
145
146 def compute_loss(criterion, outputs, labels, loss_name, device):
147     """
148     Calculeaza valoarea erorii. Gestioneaza automat transformarile necesare
149     pentru etichete (ex. transformarea in vectori ordinali).

```

```

150     """
151     loss_name = loss_name.lower()
152
153     if loss_name in ['bce_ordinal', 'ordinal']:
154         # Ordinal Regression necesita transformarea [2] -> [1, 1, 1, 0, 0]
155         levels = torch.arange(5).to(device)
156         labels_ordinal = (labels.unsqueeze(1) >= levels).float()
157         return criterion(outputs, labels_ordinal)
158
159     else:
160         # Pentru CE, Weighted CE si noul Focal Loss, putem da etichetele exact asa cum vin (
161         # indici 0-4)
162         return criterion(outputs, labels)
163
164 def get_predictions(outputs, loss_name):
165     """
166     Transforma iesirea modelului (tensor de probabilitati/logits) intr-o eticheta finala (0,
167     1, 2, 3, 4).
168     """
169     loss_name = loss_name.lower()
170
171     if loss_name in ['bce_ordinal', 'ordinal']:
172         # Adunam valorile care trec pragul de 0.0 (pragul implicit dupa BCEWithLogits)
173         preds = (outputs > 0.0).sum(dim=1) - 1
174         return torch.clamp(preds, min=0, max=4)
175
176     else:
177         # Pentru CE, Weighted CE si Focal Loss, extragem direct valoarea maxima
178         return torch.argmax(outputs, dim=1)
179
180 HEAD_PARAM_KEYWORDS = (
181     'fc',
182     'classifier',
183     'classif',
184     'head',
185     'last_linear',
186     'logits',
187 )
188
189 def is_head_parameter(name):
190     lowered = name.lower()
191     return any(keyword in lowered for keyword in HEAD_PARAM_KEYWORDS)
192
193
194 def set_backbone_trainable(model, trainable):
195     for name, param in model.named_parameters():
196         if not is_head_parameter(name):
197             param.requires_grad = trainable
198
199
200 def count_trainable_parameters(model):
201     return sum(param.numel() for param in model.parameters() if param.requires_grad)
202
203
204 def get_optimizer_params(model, lr, backbone_lr_mult=1.0):
205     if backbone_lr_mult == 1.0:
206         return model.parameters()
207
208     head_params = []
209     backbone_params = []
210     for name, param in model.named_parameters():
211         if is_head_parameter(name):
212             head_params.append(param)
213         else:
214             backbone_params.append(param)
215
216     return [
217         {'params': backbone_params, 'lr': lr * backbone_lr_mult},
218         {'params': head_params, 'lr': lr},
219     ]
220
221
222 def format_lrs(optimizer):
223     return ', '.join([f"{group['lr']:.8f}" for group in optimizer.param_groups])

```

```

224
225
226 # =====
227 # 4. OPTIMIZATORII
228 # =====
229 def get_optimizer(model, optimizer_name='adam', lr=1e-4, backbone_lr_mult=1.0):
230     optimizer_name = optimizer_name.lower()
231     params = get_optimizer_params(model, lr, backbone_lr_mult=backbone_lr_mult)
232
233     if optimizer_name == 'adam':
234         return optim.Adam(params, lr=lr, weight_decay=1e-4)
235     elif optimizer_name == 'adamw':
236         return optim.AdamW(params, lr=lr, weight_decay=1e-4)
237     elif optimizer_name == 'sgd':
238         return optim.SGD(params, lr=lr, momentum=0.9, weight_decay=1e-5)
239     else:
240         raise ValueError(f"Optimizatorul '{optimizer_name}' nu este suportat.")

```

Listing A.3: builder.py – construirea modelelor, funcțiilor de pierdere și optimizatorilor

```

1 import os
2 import random
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import seaborn as sns
6 import torch
7 import torch.nn.functional as F
8 from sklearn.metrics import (
9     auc,
10     balanced_accuracy_score,
11     classification_report,
12     cohen_kappa_score,
13     confusion_matrix,
14     f1_score,
15     precision_recall_fscore_support,
16     roc_auc_score,
17     roc_curve,
18 )
19 from sklearn.preprocessing import label_binarize
20 from tqdm import tqdm
21 from builder import compute_loss, get_predictions
22
23 # Constante
24 CLASS_NAMES = ['0', '1', '2', '3', '4']
25 IMAGENET_MEAN = np.array([0.485, 0.456, 0.406])
26 IMAGENET_STD = np.array([0.229, 0.224, 0.225])
27
28 RESULT_SUBDIRS = {
29     'history': 'istoric', 'loss': 'loss', 'accuracy': 'acuratete', 'auc': 'auc', 'qwk': 'qwk',
30     'macro_f1': 'macro_f1', 'balanced_acc': 'balanced_accuracy', 'confusion': 'matrici_confuzie',
31     'classification_reports': 'rapoarte_clasificare', 'precision_recall_f1': 'precision_recall_f1',
32     'roc': 'roc', 'gradcam': 'gradcam', 'errors': 'erori', 'other': 'altele',
33 }
34
35 # =====
36 # Functii utilitare
37 # =====
38 def resolve_dataset_path(path, project_root):
39     if path is None: return None
40     path = os.path.expanduser(path)
41     return os.path.abspath(path) if os.path.isabs(path) else os.path.abspath(os.path.join(
42         project_root, path))
43
44 def safe_source_name(name):
45     return name.replace(' ', '_').replace('/', '_').replace('\\', '_').lower()
46
47 def make_results_dirs(grafice_dir):
48     dirs = {}
49     os.makedirs(grafice_dir, exist_ok=True)
50     for key, folder_name in RESULT_SUBDIRS.items():
51         path = os.path.join(grafice_dir, folder_name)

```

```

51         os.makedirs(path, exist_ok=True)
52         dirs[key] = path
53     return dirs
54
55 def make_experiment_base_name(experiment_name, args):
56     parts = [experiment_name, args.model]
57     if args.loss_name != 'focal_loss': parts.append(args.loss_name)
58     if args.optimizer != 'adam': parts.append(args.optimizer)
59     if args.loss_name == 'focal_loss': parts.append(f"g{args.gamma:g}")
60     return "_".join(parts)
61
62 def make_result_base_name(base_name, test_source, use_test_suffix):
63     if not use_test_suffix: return base_name
64     return f"{base_name}_test_{test_source.replace('+', '_').replace('/', '_').replace('\\', '_')}"
65
66 class ModelEMA:
67     def __init__(self, model, decay):
68         self.decay = decay
69         self.shadow = {n: p.detach().clone() for n, p in model.state_dict().items() if torch.
is_floating_point(p)}
70         self.backup = {}
71
72     @torch.no_grad()
73     def update(self, model):
74         model_state = model.state_dict()
75         for name, shadow_param in self.shadow.items():
76             shadow_param.mul_(self.decay).add_(model_state[name].detach(), alpha=1.0 - self.
decay)
77
78     def apply_shadow(self, model):
79         self.backup = {}
80         model_state = model.state_dict()
81         for name, shadow_param in self.shadow.items():
82             self.backup[name] = model_state[name].detach().clone()
83             model_state[name].copy_(shadow_param)
84
85     def restore(self, model):
86         model_state = model.state_dict()
87         for name, backup_param in self.backup.items():
88             model_state[name].copy_(backup_param)
89         self.backup = {}
90
91 # =====
92 # Functii de vizualizare (Grafice, Grad-CAM, Dataset Grid)
93 # =====
94 def denormalize_image(tensor):
95     image = tensor.detach().cpu().permute(1, 2, 0).numpy()
96     return np.clip((image * IMAGENET_STD) + IMAGENET_MEAN, 0, 1)
97
98 def show_dataset_samples_grid(dataset_dict, title_prefix=""):
99     num_datasets = len(dataset_dict)
100     if num_datasets == 0: return
101     fig, axes = plt.subplots(num_datasets, 5, figsize=(15, 3 * num_datasets))
102     if num_datasets == 1: axes = np.expand_dims(axes, axis=0)
103     for row_idx, (source_name, ds) in enumerate(dataset_dict.items()):
104         found_classes = {}
105         indices = list(range(len(ds)))
106         random.shuffle(indices)
107         for i in indices[:min(len(ds), 500)]:
108             img, label = ds[i]
109             lbl = int(label)
110             if lbl not in found_classes: found_classes[lbl] = denormalize_image(img)
111             if len(found_classes) == 5: break
112         for col_idx in range(5):
113             ax = axes[row_idx, col_idx]
114             if col_idx in found_classes: ax.imshow(found_classes[col_idx])
115             else: ax.text(0.5, 0.5, 'Lipsa', ha='center', va='center')
116             if row_idx == 0: ax.set_title(f"Clasa {col_idx}")
117             ax.axis('off')
118             if col_idx == 0: ax.text(-0.1, 0.5, source_name, va='center', ha='right',
transform=ax.transAxes, fontsize=12, fontweight='bold', rotation=90)
119     plt.suptitle(f"Esantioane - {title_prefix}", fontsize=16)
120     plt.tight_layout()
121     plt.show()
122

```



```

123 def save_combined_history(history, path, show=False):
124     fig, axes = plt.subplots(2, 3, figsize=(18, 10))
125     axes = axes.ravel()
126     metrics = [('Train Loss', 'Val Loss', 'train_loss', 'val_loss', 'Istoric Loss', 'Loss'),
127               ('Train Accuracy', 'Val Accuracy', 'train_acc', 'val_acc', 'Istoric Acuratete',
128                'Acuratete')]
129     for i, (l1, l2, k1, k2, tit, ylab) in enumerate(metrics):
130         axes[i].plot(history[k1], label=l1, color='blue')
131         axes[i].plot(history[k2], label=l2, color='orange')
132         axes[i].set_title(tit); axes[i].set_xlabel('Epoca'); axes[i].set_ylabel(ylab); axes[i]
133         ].legend()
134     for i, (l1, k1, tit, ylab, col) in enumerate([('Val AUC', 'val_auc', 'Istoric AUC (Val)',
135          'AUC', 'green'),
136          ('Val QWK', 'val_qwk', 'Istoric QWK (Val)',
137          'QWK', 'purple'),
138          ('Val Macro F1', 'val_macro_f1', 'Istoric
139          Macro F1 (Val)', 'Macro F1', 'red'),
140          ('Val Balanced Acc', 'val_balanced_acc', '
141          Istoric Balanced Accuracy (Val)', 'Balanced Accuracy', 'brown')], start=2):
142         axes[i].plot(history[k1], label=l1, color=col)
143         axes[i].set_title(tit); axes[i].set_xlabel('Epoca'); axes[i].set_ylabel(ylab); axes[i]
144         ].legend()
145     plt.tight_layout()
146     plt.savefig(path, bbox_inches='tight')
147     if show: plt.show()
148     plt.close()
149
150 def save_combined_confusion_matrices(labels, preds, path, show=False):
151     fig, axes = plt.subplots(1, 2, figsize=(16, 6))
152     cm = confusion_matrix(labels, preds, labels=list(range(5)))
153     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=CLASS_NAMES, yticklabels=
154     CLASS_NAMES, ax=axes[0], annot_kws={"size": 16, "weight": "bold"})
155     axes[0].set_title('Matrice Confuzie (Absoluta)'); axes[0].set_xlabel('Predictie (Model)');
156     axes[0].set_ylabel('Realitate (Adevar)')
157     cm_norm = confusion_matrix(labels, preds, labels=list(range(5)), normalize='true')
158     sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', xticklabels=CLASS_NAMES,
159     yticklabels=CLASS_NAMES, vmin=0, vmax=1, ax=axes[1], annot_kws={"size": 16, "weight": "
160     bold"})
161     axes[1].set_title('Matrice Confuzie (Normalizata)'); axes[1].set_xlabel('Predictie (Model)
162     '); axes[1].set_ylabel('Realitate (Adevar)')
163     plt.tight_layout()
164     plt.savefig(path, bbox_inches='tight')
165     if show: plt.show()
166     plt.close()
167
168 def save_precision_recall_f1(labels, preds, base_name, results_dirs, show=False):
169     precision, recall, f1, support = precision_recall_fscore_support(labels, preds, labels=
170     list(range(5)), zero_division=0)
171     x = np.arange(len(CLASS_NAMES)); width = 0.25
172     plt.figure(figsize=(10, 6))
173     plt.bar(x - width, precision, width, label='Precision'); plt.bar(x, recall, width, label='
174     Recall'); plt.bar(x + width, f1, width, label='F1')
175     plt.xticks(x, CLASS_NAMES); plt.ylim(0, 1); plt.xlabel('Clasa'); plt.ylabel('Scor'); plt.
176     title('Precision / Recall / F1 pe clase'); plt.legend(); plt.grid(axis='y', alpha=0.25)
177     prf1_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.png")
178     plt.savefig(prf1_path, bbox_inches='tight')
179     if show: plt.show()
180     plt.close()
181     report = classification_report(labels, preds, labels=list(range(5)), target_names=
182     CLASS_NAMES, zero_division=0)
183     report_path = os.path.join(results_dirs['classification_reports'], f"
184     classification_report_{base_name}.txt")
185     with open(report_path, 'w', encoding='utf-8') as f: f.write(report)
186     metrics_path = os.path.join(results_dirs['precision_recall_f1'], f"prf1_{base_name}.csv")
187     with open(metrics_path, 'w', encoding='utf-8') as f:
188         f.write('class,precision,recall,f1,support\n')
189         for c, p, r, f1_val, s in zip(CLASS_NAMES, precision, recall, f1, support): f.write(f'
190         {c},{p:.6f},{r:.6f},{f1_val:.6f},{int(s)}\n')
191     return prf1_path, report_path, metrics_path, report
192
193 def save_roc_curve(labels, probs, base_name, results_dirs, show=False):
194     labels = np.asarray(labels); probs = np.asarray(probs); y_true = label_binarize(labels,
195     classes=list(range(5)))
196     plt.figure(figsize=(8, 6)); plotted_any = False
197     for i, class_name in enumerate(CLASS_NAMES):
198         if y_true[:, i].sum() in [0, len(y_true)]: continue

```

```

180         fpr, tpr, _ = roc_curve(y_true[:, i], probs[:, i])
181         plt.plot(fpr, tpr, label=f'Clasa {class_name} (AUC={auc(fpr, tpr):.3f})')
182         plotted_any = True
183     if plotted_any:
184         try: plt.plot(*roc_curve(y_true.ravel(), probs.ravel())[:2], linestyle='--', color='
black', label=f'Micro (AUC={auc(*roc_curve(y_true.ravel(), probs.ravel())[:2]):.3f})')
185         except Exception: pass
186     plt.plot([0, 1], [0, 1], linestyle=':', color='gray'); plt.xlabel('False Positive Rate');
plt.ylabel('True Positive Rate'); plt.title('Curbe ROC multiclasa'); plt.legend(loc='lower
right'); plt.grid(alpha=0.25)
187     roc_path = os.path.join(results_dirs['roc'], f"roc_{base_name}.png")
188     plt.savefig(roc_path, bbox_inches='tight')
189     if show: plt.show()
190     plt.close()
191     return roc_path
192
193 def find_last_conv_layer(model):
194     for name, module in model.named_modules():
195         if isinstance(module, torch.nn.Conv2d): last_name, last_module = name, module
196     try: return last_name, last_module
197     except NameError: raise RuntimeError("Nu am gasit niciun strat Conv2d pentru Grad-CAM.")
198
199 def save_gradcam_overlay_all_classes(model, data_loader, loss_name, device, base_name,
results_dirs, show=False):
200     try: layer_name, target_layer = find_last_conv_layer(model)
201     except RuntimeError as e: print(f"Grad-CAM sarit: {e}"); return None
202     model.eval()
203     class_images = {}
204     for imgs, labels in data_loader:
205         for i, lbl in enumerate(labels.tolist()):
206             if lbl not in class_images: class_images[lbl] = imgs[i].unsqueeze(0)
207             if len(class_images) == 5: break
208     if not class_images: return None
209
210     fig, axes = plt.subplots(2, 5, figsize=(20, 8))
211     fig.suptitle(f"Grad-CAM 5 Clase (Layer: {layer_name})", fontsize=16)
212
213     for c in range(5):
214         ax_orig, ax_cam = axes[0, c], axes[1, c]
215         if c not in class_images:
216             ax_orig.axis('off'); ax_cam.axis('off'); continue
217
218         img_tensor = class_images[c].to(device)
219         activations, gradients = [], []
220         fh = target_layer.register_forward_hook(lambda _, __, out: activations.append(out.
detach()))
221         bh = target_layer.register_full_backward_hook(lambda _, __, gout: gradients.append(
gout[0].detach()))
222
223         model.zero_grad(set_to_none=True)
224         outputs = model(img_tensor)
225         preds = get_predictions(outputs, loss_name)
226         outputs[0, c].backward()
227
228         cam = F.relu(torch.sum(gradients[-1][0].mean(dim=(1, 2), keepdim=True) * activations
[-1][0], dim=0))
229         cam = ((cam - cam.min()) / (cam.max() - cam.min())) if cam.max() > cam.min() else
torch.zeros_like(cam)
230         cam = F.interpolate(cam.unsqueeze(0).unsqueeze(0), size=img_tensor.shape[-2:], mode='
bilinear', align_corners=False).squeeze().cpu().numpy()
231
232         image = denormalize_image(img_tensor[0])
233         overlay = np.clip((0.55 * image) + (0.45 * plt.cm.jet(cam)[..., :3]), 0, 1)
234
235         ax_orig.imshow(image); ax_orig.set_title(f'Original (Adevarat: {c})'); ax_orig.axis('
off')
236         ax_cam.imshow(overlay); ax_cam.set_title(f'Grad-CAM (Predictie: {preds[0].item()})');
ax_cam.axis('off')
237         fh.remove(); bh.remove()
238
239     plt.tight_layout()
240     gradcam_path = os.path.join(results_dirs['gradcam'], f"gradcam_5clase_{base_name}.png")
241     plt.savefig(gradcam_path, bbox_inches='tight')
242     if show: plt.show()
243     plt.close()
244     return gradcam_path

```

```

245
246 # =====
247 # Inima antrenamentului / evaluarii - run_one_epoch
248 # =====
249 def compute_extra_metrics(labels, preds):
250     try: qwk = cohen_kappa_score(labels, preds, weights='quadratic')
251     except: qwk = 0.0
252     try: macro_f1 = f1_score(labels, preds, labels=list(range(5)), average='macro',
253                               zero_division=0)
254     except: macro_f1 = 0.0
255     try: balanced_acc = balanced_accuracy_score(labels, preds)
256     except: balanced_acc = 0.0
257     return qwk, macro_f1, balanced_acc
258
259 def run_one_epoch(model, loader, criterion, optimizer, device, loss_name, train=True, desc="",
260                  scaler=None, use_amp=False, grad_clip=0.0, ema=None, use_tta=False):
261     model.train() if train else model.eval()
262
263     total_loss, correct, total = 0.0, 0, 0
264     all_preds, all_labels, all_probs = [], [], []
265     tta_active = use_tta and not train and loss_name.lower() not in ['bce_ordinal', 'ordinal']
266
267     with torch.set_grad_enabled(train):
268         progress = tqdm(loader, desc=desc, leave=False)
269         for imgs, labels in progress:
270             imgs, labels = imgs.to(device), labels.to(device)
271
272             if train and optimizer is not None:
273                 optimizer.zero_grad(set_to_none=True)
274
275             with torch.amp.autocast(device.type, enabled=use_amp):
276                 if tta_active:
277                     outputs = (model(imgs) + model(torch.flip(imgs, dims=[3]))) / 2.0
278                 else:
279                     outputs = model(imgs)
280                 loss = compute_loss(criterion, outputs, labels, loss_name, device)
281
282             if train and optimizer is not None and scaler is not None:
283                 scaler.scale(loss).backward()
284                 if grad_clip > 0:
285                     scaler.unscale_(optimizer)
286                     torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip)
287                     scaler.step(optimizer)
288                     scaler.update()
289                     if ema is not None: ema.update(model)
290
291             batch_size = imgs.size(0)
292             total_loss += loss.item() * batch_size
293             preds = get_predictions(outputs, loss_name)
294             correct += (preds == labels).sum().item()
295             total += batch_size
296
297             if not train:
298                 all_preds.extend(preds.cpu().numpy())
299                 all_labels.extend(labels.cpu().numpy())
300                 all_probs.extend(F.softmax(outputs, dim=1).cpu().numpy())
301
302             progress.set_postfix({'loss': f"{loss.item():.4f}"})
303
304     avg_loss = total_loss / max(total, 1)
305     acc = correct / max(total, 1)
306
307     if train:
308         return avg_loss, acc
309     else:
310         try: auc_val = roc_auc_score(all_labels, all_probs, multi_class='ovr', average='macro'
311                                     )
312         except: auc_val = 0.0
313         qwk, macro_f1, balanced_acc = compute_extra_metrics(all_labels, all_preds)
314         return avg_loss, acc, auc_val, qwk, macro_f1, balanced_acc, all_preds, all_labels,
315         all_probs
316
317 # =====
318 # Utilitare Metric Monitor / Final Test
319 # =====
320 def select_metric(metrics, monitor_metric):

```

```

318     return metrics.get(monitor_metric, metrics['val_loss'])
319
320 def is_lower_better_metric(monitor_metric):
321     return monitor_metric == 'val_loss'
322
323 def metric_improved(metric, best_metric, monitor_metric, min_delta):
324     return metric < best_metric - min_delta if is_lower_better_metric(monitor_metric) else
325         metric > best_metric + min_delta
326
327 def run_final_test(model, test_source, test_loader, criterion, args, device, base_name,
328     results_dirs, use_test_suffix):
329     result_base_name = make_result_base_name(base_name, test_source, use_test_suffix)
330     print("\n" + "=" * 50); print(f"START TEST FINAL: {test_source}"); print("=" * 50)
331
332     test_loss, test_acc, test_auc, test_qwk, test_macro_f1, test_balanced_acc, test_preds,
333     test_labels, test_probs = run_one_epoch(
334         model, test_loader, criterion, None, device, args.loss_name, train=False, desc=f"
335         Testare model [{test_source}]", use_tta=args.tta_eval
336     )
337
338     print("Rezultate testare finala:")
339     print(f"Dataset test: {test_source}\n  Acuratete: {test_acc * 100:.2f}%\n  Scor AUC:
340         {test_auc:.4f}\n  QWK: {test_qwk:.4f}")
341     print(f"Macro F1: {test_macro_f1:.4f}\n  Bal Acc: {test_balanced_acc:.4f}\n  Loss:
342         {test_loss:.4f}\n")
343
344     print("Classification report:")
345     prf1_path, report_path, metrics_path, report = save_precision_recall_f1(test_labels,
346         test_preds, result_base_name, results_dirs, show=True)
347     print(report)
348
349     cm_path = os.path.join(results_dirs['confusion'], f"cm_combined_{result_base_name}.png")
350     save_combined_confusion_matrices(test_labels, test_preds, cm_path, show=True)
351     roc_path = save_roc_curve(test_labels, test_probs, result_base_name, results_dirs, show=
352         True)
353     gradcam_path = save_gradcam_overlay_all_classes(model, test_loader, args.loss_name, device
354         , result_base_name, results_dirs, show=True)
355
356     print(f"Rezultate pentru test={test_source}:")
357     print(f"Matrice confuzie combinata: {cm_path}\nPrecision/Recall/F1: {prf1_path}\nRaport
358         clasificare: {report_path}\nMetrici CSV: {metrics_path}\nROC curve: {roc_path}")
359     if gradcam_path: print(f"Grad-CAM 5 clase overlay: {gradcam_path}")
360
361     return {
362         'source': test_source, 'loss': test_loss, 'acc': test_acc, 'auc': test_auc, 'qwk':
363         test_qwk,
364         'macro_f1': test_macro_f1, 'balanced_acc': test_balanced_acc, 'cm_path': cm_path,
365         'prf1_path': prf1_path, 'report_path': report_path, 'metrics_path': metrics_path,
366         'roc_path': roc_path, 'gradcam_path': gradcam_path,
367     }

```

Listing A.4: utils.py – funcții de vizualizare și pentru antrenare

```

1 import argparse
2 import os
3 import sys
4 import time
5 import torch
6 from torch.utils.data import DataLoader
7
8 try: CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
9 except NameError: CURRENT_DIR = os.getcwd()
10 PARENT_DIR = os.path.abspath(os.path.join(CURRENT_DIR, '..'))
11 PROJECT_ROOT = PARENT_DIR
12 if PARENT_DIR not in sys.path: sys.path.insert(0, PARENT_DIR)
13
14 from builder import count_trainable_parameters, format_lrs, get_loss_function, get_model,
15     get_optimizer, set_backbone_trainable
16 from my_dataset import make_dataset
17 import utils
18
19 def parse_args():

```

```

19 parser = argparse.ArgumentParser(description="Antrenament DR - dataseturi custom
ImageFolder")
20 parser.add_argument('--model', type=str, default='resnet50')
21 parser.add_argument('--loss_name', type=str, default='ce')
22 parser.add_argument('--optimizer', type=str, default='adam')
23 parser.add_argument('--epochs', type=int, default=20)
24 parser.add_argument('--batch_size', type=int, default=32)
25 parser.add_argument('--lr', type=float, default=0.0001)
26 parser.add_argument('--img_size', type=int, default=224)
27 parser.add_argument('--num_workers', type=int, default=0)
28 parser.add_argument('--class_weights', type=float, nargs='+', default=None)
29 parser.add_argument('--gamma', type=float, default=1.5)
30 parser.add_argument('--label_smoothing', type=float, default=0.0)
31 parser.add_argument('--monitor_metric', type=str, default='qwk', choices=['qwk', 'macro_f1',
', 'balanced_acc', 'auc', 'acc', 'val_loss'])
32 parser.add_argument('--scheduler', type=str, default='plateau', choices=['plateau', '
cosine', 'none'])
33 parser.add_argument('--amp', action='store_true')
34 parser.add_argument('--grad_clip', type=float, default=0.0)
35 parser.add_argument('--ema_decay', type=float, default=0.0)
36 parser.add_argument('--model_dropout', type=float, default=0.0)
37 parser.add_argument('--freeze_backbone_epochs', type=int, default=0)
38 parser.add_argument('--backbone_lr_mult', type=float, default=1.0)
39 parser.add_argument('--tta_eval', action='store_true')
40 parser.add_argument('--train_augment', type=str, default='basic', choices=['basic', 'none'
])
41 parser.add_argument('--early_stopping_patience', type=int, default=0)
42 parser.add_argument('--early_stopping_min_delta', type=float, default=0.0)
43 parser.add_argument('--train_dataset_roots', type=str, nargs='+', required=True)
44 parser.add_argument('--train_dataset_names', type=str, nargs='+', default=None)
45 parser.add_argument('--test_dataset_roots', type=str, nargs='+', required=True)
46 parser.add_argument('--test_dataset_names', type=str, nargs='+', default=None)
47 return parser.parse_args()
48
49 def main():
50     args = parse_args()
51     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
52
53     train_roots = [utils.resolve_dataset_path(path, PROJECT_ROOT) for path in args.
train_dataset_roots]
54     test_roots = [utils.resolve_dataset_path(path, PROJECT_ROOT) for path in args.
test_dataset_roots]
55     train_names = [utils.safe_source_name(args.train_dataset_names[i] if args.
train_dataset_names else f"train_{i+1}") for i in range(len(train_roots))]
56     test_sources = [utils.safe_source_name(args.test_dataset_names[i] if args.
test_dataset_names else f"test_{i+1}") for i in range(len(test_roots))]
57     test_roots_by_source = dict(zip(test_sources, test_roots))
58     train_source = '+'.join(train_names)
59     experiment_name = f"{train_source}_to_{'.'.join(test_sources)}"
60
61     train_datasets_viz = {name: make_dataset(root, 'train', args.img_size, args.train_augment)
for name, root in zip(train_names, train_roots)}
62     test_datasets = {src: make_dataset(root, 'test', args.img_size, args.train_augment) for
src, root in test_roots_by_source.items()}
63
64     print("Vizualizam esantioane din seturile de antrenament:")
65     utils.show_dataset_samples_grid(train_datasets_viz, title_prefix="Date Antrenament (TRAIN)
")
66     print("Vizualizam esantioane din seturile de testare:")
67     utils.show_dataset_samples_grid(test_datasets, title_prefix="Date Testare (TEST)")
68
69     rezultate_dir = os.path.abspath(os.path.join(CURRENT_DIR, "..", "Rezultate", "New"))
70     modele_dir = os.path.join(rezultate_dir, "modele")
71     os.makedirs(modele_dir, exist_ok=True)
72     results_dirs = utils.make_results_dirs(os.path.join(rezultate_dir, "grafice"))
73
74     print(f"Config: {args.model} | Loss: {args.loss_name} | Opt: {args.optimizer} | LR: {args.
lr}")
75     print(f"Experiment: {experiment_name} | TRAIN/VAL={train_source} | TEST={'.'.join(
test_sources)}")
76     print(f"Train augment: {args.train_augment} | Monitor metric: {args.monitor_metric} |
Scheduler: {args.scheduler}")
77     print(f"AMP: {args.amp} | Grad clip: {args.grad_clip} | EMA decay: {args.ema_decay} | TTA
eval: {args.tta_eval}")
78     print(f"Model dropout: {args.model_dropout} | Freeze backbone epochs: {args.
freeze_backbone_epochs} | Backbone LR mult: {args.backbone_lr_mult}")

```

```

79
80 train_ds = make_dataset(train_roots, split='train', image_size=args.img_size,
81 train_augment=args.train_augment)
82 val_ds = make_dataset(train_roots, split='val', image_size=args.img_size, train_augment=
83 args.train_augment)
84 train_loader = DataLoader(train_ds, batch_size=args.batch_size, shuffle=True, num_workers=
85 args.num_workers)
86 val_loader = DataLoader(val_ds, batch_size=args.batch_size, shuffle=False, num_workers=
87 args.num_workers)
88 test_loaders = {src: DataLoader(ds, batch_size=args.batch_size, shuffle=False, num_workers
89 =args.num_workers) for src, ds in test_datasets.items()}
90
91 if args.loss_name == 'focal_loss': print(f"Focal Loss activat cu gamma={args.gamma}")
92 test_sizes = ' | '.join([f"TESTARE[{src}]={len(ds)}" for src, ds in test_datasets.items()
93 ])
94 print(f"Distributie date: TRAIN={len(train_ds)} | VALIDARE={len(val_ds)} | {test_sizes}")
95
96 model = get_model(args.model, num_classes=5, drop_rate=args.model_dropout).to(device)
97 if args.freeze_backbone_epochs > 0:
98     set_backbone_trainable(model, trainable=False)
99     print(f"Backbone inghetat pentru primele {args.freeze_backbone_epochs} epoci.
100 Parametri antrenabili initial: {count_trainable_parameters(model):,}")
101
102 criterion = get_loss_function(args.loss_name, class_weights=args.class_weights, device=
103 device, gamma=args.gamma, label_smoothing=args.label_smoothing)
104 optimizer = get_optimizer(model, optimizer_name=args.optimizer, lr=args.lr,
105 backbone_lr_mult=args.backbone_lr_mult)
106
107 if args.scheduler == 'plateau':
108     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min' if utils.
109 is_lower_better_metric(args.monitor_metric) else 'max', factor=0.75, patience=5)
110 elif args.scheduler == 'cosine':
111     scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=max(args.
112 epochs, 1), eta_min=args.lr * 0.01)
113 else: scheduler = None
114
115 use_amp = args.amp and device.type == 'cuda'
116 scaler = torch.amp.GradScaler(device.type, enabled=use_amp)
117 ema = utils.ModelEMA(model, args.ema_decay) if args.ema_decay > 0 else None
118
119 history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': [], 'val_auc':
120 [], 'val_qwk': [], 'val_macro_f1': [], 'val_balanced_acc': []}
121 best_metric = float('inf') if utils.is_lower_better_metric(args.monitor_metric) else -
122 float('inf')
123 epochs_without_improvement, stopped_early = 0, False
124 base_name = utils.make_experiment_base_name(experiment_name, args)
125 model_path = os.path.join(modele_dir, f"best_{base_name}.pth")
126
127 # =====
128 # BUCLA DE ANTRENAMENT (Curata, bazata exclusiv pe run_one_epoch)
129 # =====
130 for epoch in range(args.epochs):
131     start_time = time.time()
132     if args.freeze_backbone_epochs > 0 and epoch == args.freeze_backbone_epochs:
133         set_backbone_trainable(model, trainable=True)
134         print(f"Backbone deblocat la epoca {epoch + 1}.")
135
136     # 1. ANTRENAMENT
137     epoch_train_loss, epoch_train_acc = utils.run_one_epoch(
138         model, train_loader, criterion, optimizer, device, args.loss_name, train=True,
139         desc=f"Ep {epoch + 1:02d} [TRAIN]", scaler=scaler, use_amp=use_amp, grad_clip=args
140 .grad_clip, ema=ema
141     )
142
143     # 2. VALIDARE
144     if ema is not None: ema.apply_shadow(model)
145     val_loss, val_acc, val_auc, val_qwk, val_macro_f1, val_balanced_acc, _, _, _ = utils.
146 run_one_epoch(
147     model, val_loader, criterion, None, device, args.loss_name, train=False,
148     desc=f"Ep {epoch + 1:02d} [VALID]", use_tta=args.tta_eval
149 )
150     if ema is not None: ema.restore(model)
151
152     history['train_loss'].append(epoch_train_loss); history['train_acc'].append(
153 epoch_train_acc)

```

```

138     history['val_loss'].append(val_loss); history['val_acc'].append(val_acc); history['
val_auc'].append(val_auc)
139     history['val_qwk'].append(val_qwk); history['val_macro_f1'].append(val_macro_f1);
history['val_balanced_acc'].append(val_balanced_acc)
140
141     metrics = {'acc': val_acc, 'auc': val_auc, 'qwk': val_qwk, 'macro_f1': val_macro_f1, '
balanced_acc': val_balanced_acc, 'val_loss': val_loss}
142     metric = utils.select_metric(metrics, args.monitor_metric)
143     if scheduler is not None: scheduler.step(metric) if args.scheduler == 'plateau' else
scheduler.step()
144
145     marker = ""
146     if utils.metric_improved(metric, best_metric, args.monitor_metric, args.
early_stopping_min_delta):
147         best_metric = metric; epochs_without_improvement = 0
148         if ema is not None: ema.apply_shadow(model)
149         torch.save(model.state_dict(), model_path)
150         if ema is not None: ema.restore(model)
151         marker = "NOU BEST"
152     else: epochs_without_improvement += 1
153
154     print(f"Ep {epoch + 1:02d}/{args.epochs} | T-Loss: {epoch_train_loss:.4f} | V-Loss: {
val_loss:.4f} | "
155           f"T-Acc: {epoch_train_acc * 100:.1f}% | V-Acc: {val_acc * 100:.1f}% | V-AUC: {
val_auc:.4f} | "
156           f"V-QWK: {val_qwk:.4f} | V-F1: {val_macro_f1:.4f} | V-BalAcc: {val_balanced_acc
:.4f} | "
157           f"Best[{args.monitor_metric}]=[{best_metric:.4f}] {marker}")
158     print(f"LR curent: {format_lrs(optimizer)}\nDurata epoca: {(time.time() - start_time)
/ 60:.2f} min")
159
160     if args.early_stopping_patience > 0:
161         print(f"Early stopping: {epochs_without_improvement}/{args.early_stopping_patience
} epoci fara imbunatatire")
162         if epochs_without_improvement >= args.early_stopping_patience:
163             stopped_early = True
164             print(f"Early stopping activat la epoca {epoch + 1}.")
165             break
166
167     # =====
168     # TESTARE FINALA
169     # =====
170     model.load_state_dict(torch.load(model_path, map_location=device))
171     history_path = os.path.join(results_dirs['history'], f"istoric_combinat_{base_name}.png")
172     utils.save_combined_history(history, history_path, show=True)
173
174     use_test_suffix = len(test_loaders) > 1
175     test_results = [utils.run_final_test(model, src, loader, criterion, args, device,
base_name, results_dirs, use_test_suffix) for src, loader in test_loaders.items()]
176
177     print(f"\nGata. Model: {model_path}\nGrafic evolutie combinat: {history_path}")
178     if stopped_early: print(f"Antrenare oprita anticipat la epoca {epoch + 1}/{args.epochs}.")
179     if len(test_results) > 1:
180         print("Rezumat testari finale:")
181         for r in test_results: print(f"    {r['source']}: Acc={r['acc'] * 100:.2f}% | AUC={r['
auc']:.4f} | QWK={r['qwk']:.4f} | F1={r['macro_f1']:.4f}")
182
183 if __name__ == '__main__':
184     if not hasattr(sys.modules['__main__'], '__spec__'): sys.modules['__main__'].__spec__ =
None
185     main()

```

Listing A.5: train.py – antrenarea, evaluarea și generarea graficelor